POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa inżynierska

SYSTEM PROCEDURALNEGO GENEROWANIA FUNKCJONALNOŚCI W GRACH TYPU "TOWER DEFENSE".

Jakub Kolasiński, 155985
Krzysztof Mańczak, 155939
Sebastian Marcinek, 155914
Jakub Szybura, 155951

Promotor
dr inż. Andrzej Stroiński

POZNAŃ 2026

Spis treści

1 Wstęp **1**

2 Przegląd rozwiązań i wstęp teoretyczny **4**

2.1 Proceduralne generowanie treści w grach 4

2.2 Specyfika gatunku Tower Defense (balans, ścieżki, fale przeciwników) 5

2.3 Przegląd istniejących rozwiązań w grach 7

2.4 Przegląd literatury i innych źródeł wiedzy 8

2.5 Analiza i wybór algorytmów/rozwiązań 9

3 Moduł biblioteki do dynamicznego dopasowywania rozgrywki w grach **13**

3.1 Wykorzystane technologie i biblioteki zewnętrzne 13

3.2 Założenia projektowe i architektura 13

3.3 Biblioteka 16

3.3.1 Generatory RNG 16

Liczby 17

Ziarno 20

Schematy ataków 26

3.3.2 Moduł generacji środowiska gry 28

Siatka/Wewnętrzna mapa 28

Pathfinding (Szukanie ścieżki) 30

Generator Mapy 38

3.3.3 System efektów gry (moduł stanów) 48

Logika 48

Rejestr 48

Planer 50

3.3.4 Moduły generacji i modyfikacji treści 52

Generator tekstu oparty na łańcuchach Markowa 52

Zmiany palet barw obiektów wizualnych (Palette Swap) 56

Zmiany na obiektach dźwiękowych 61

3.3.5 Szablony kluczowych elementów gry typu "Tower Defense" 64

4 Środowisko Testowe (Gra) **66**

4.1 Integracja biblioteki i architektura gry 66

4.2 Systemy tylko w grze (UI, Save/Load, Input system, Kamera) 67

4.3 Game Loop 68

4.4 Przykład działania generatorów na poszczególnych etapach rozgrywki 69

5 Testy, parametry metod i rezultaty **72**

6 Zakończenie **78**
6.1 Podsumowanie 78

Literatura **80**

Rozdział 1

Wstęp

Gry komputerowe są przedsięwzięciami, które wymagają znacznych nakładów pracy, by końcowy efekt był zadowalający. Począwszy od etapu projektowania konieczne jest, aby planowana w grze zawartość, w postaci między innymi udostępnianych mechanik rozgrywki oraz elementów audiowizualnych, stanowiła spójną całość, która będzie dla graczy źródłem satysfakcjonującej rozrywki. Nie zawsze jednak oczekiwanym efektem jest, aby gra komputerowa stawiała przed graczem identycznie wyglądające wyzwania. Wprowadzenie do gry elementów, które docelowo z każdą rozgrywką przybierałyby różną formę i wpływałyby na sposób przejścia gry, stanowi dla dewelopera jedną z możliwości utworzenia unikatowego doświadczenia dla gracza. Do zrealizowania tego celu implementowane są w grach mechaniki generowania proceduralnego, oznaczającego tworzenie wykorzystywanych później obiektów na podstawie przygotowanych do tego celu algorytmów, aniżeli tworzenia ich ręcznie. Do wygenerowania w sposób proceduralny możliwe są nie tylko tekstury i modele, ale też statystyki przedmiotów, czy ułożenie poziomów. Pozwala to zmniejszyć nakład pracy włożony w projektowanie tych aspektów gry. Zapewnienie w ten sposób graczom odpowiednio zaimplementowanych elementów generowanych proceduralnie pozwala zwiększa poziom regrywalności i dynamiczności danych tytułów, a w konsekwencji zachęca graczy do kolejnych rozgrywek.

Wykorzystanie mechanik generowania proceduralnego znalazło swoje zastosowanie na rynku gier komputerowych, podejście to cieszy się niesłabnącą popularnością, a uznanie graczy zdobyły tytuły, które były w stanie oprzeć liczne strony rozgrywki o elementy generowane w sposób algorytmiczny. W związku z tym, motywacją do podjęcia tematu pracy była chęć wsparcia deweloperów w implementowaniu mechanik proceduralnej generacji w swoich grach komputerowych.

Zgłębiana tematyka dotyczy szerokiego wachlarza gatunków i typów gier, gdzie można zastosować metody związane z proceduralnym generowaniem. Praca skupiła się wokół gatunku *"Tower Defense"* ze względu na dostrzeżenie potencjału implementacji tych metod w celu urozmaicenia oferowanych doświadczeń.

Charakterystycznymi gatunkami, które implementują proceduralne generowanie są między innymi: *"Rogue-like"*, gry takie jak Rogue [5], Spelunky [20], Hades [8]. W ich przypadku generowane są ułożenia poziomów, rozmieszczenie przeciwników i wszelakie wartości numeryczne użyte w rozgrywce. Generacja proceduralna znajduje też zastosowanie w innych gatunkach, takich jak *"gry strategiczne"*, czy też *action role-playing*. Przykładem gry strategicznej z PCG jest np. XCOM2 [6], który zawiera generację poziomów oraz postaci grywalnych, na które składa się nie tylko generacja wyglądu postaci, ale też ich imion oraz biografii.

Implementacja proceduralnego generowania nie jest jednak aż tak popularnym zabiegiem w grach typu tower defense (TD). Założenia tego gatunku są wyjaśnione w rozdziale 2.2. W grach tego ga-

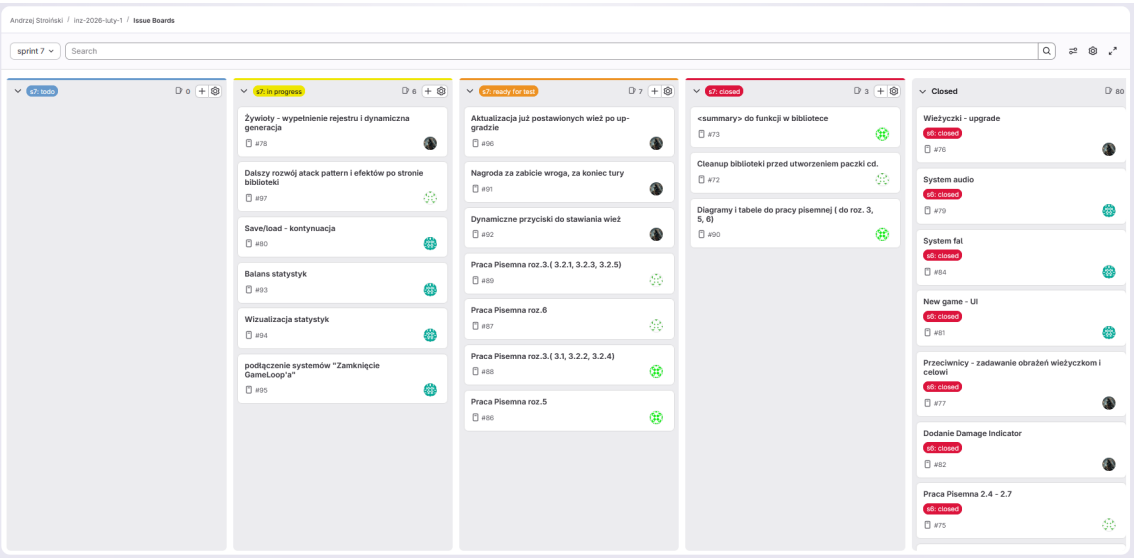
tunku tkwi jednak dużo potencjału na wprowadzenie rozwiązań wykorzystujących proceduralną generację, podobnych do tych z gier strategicznych lub rogue-like.

Autorzy projektu wychodzą na przeciw deweloperom, oferując możliwość prostego wprowadzenia tych idei pod postacią tej pracy. Jej celem było utworzenie pakietu narzędzi ułatwiającego tworzenie gier, ze szczególnym naciskiem na gry typu tower defense, mogących charakteryzować się dynamiczną generacją. Narzędzia te mają umożliwić automatyzację generacji, celem zwiększonej różnorodności w tworzonych grach. Aby tego dokonać napisana została biblioteka dla silnika do tworzenia gier komputerowych, *Unity*.

Do funkcji oferowanych przez bibliotekę należą między innymi: generacja liczb pseudolosowych (skalarów oraz wektorów), map, funkcjonalności związanych z wieżyczkami oraz przeciwnikami (ich podstawowe implementacje składające się ze statystyk oraz schematu działania), ale także generacje związane z aspektami audiowizualnymi (modyfikacja kolorów obiektu, granego dźwięku oraz tekstu). Funkcjonalności te zostały przedstawione w rozdziale 3.2, a szczegóły ich implementacji zawarto w rozdziale 3.3.

Wybór silnika oraz uzasadnienie implementacji zostanie przybliżone w kolejnych rozdziałach. Konieczne było znalezienie sposobu na przetestowanie i prezentację możliwości biblioteki oraz poprawności oferowanych przez nią mechaniki. Aby rozwiązać ten problem, przygotowane zostało środowisko testowe w formie gry, dema technologicznego gry na silniku *Unity*, która korzysta z przygotowanej biblioteki. W ramach pracy zdecydowano się na następujący podział obowiązków: połowa członków grupy miała skupić się na pisaniu biblioteki, natomiast druga połowa tworzyć miała środowisko testowe. Mimo tak ustalonego podziału, każdy z autorów wniósł ostatecznie wkład w oba środowiska. Praca inżynierska była realizowana z wykorzystaniem metodyki Scrum [12], gdzie długość sprintów wynosiła 2 tygodnie. Pod koniec każdego sprintu następowało spotkanie z promotorem pełniące funkcję przeglądu, retrospekcji oraz planowania następnego okresu.

Projekt był utrzymywany na w systemie gitlab [9] udostępnionym przez Politechnikę Poznańską. Planowanie było utrzymywane poprzez zadania zapisane w formie "ticket" oraz kontrolowane na tablicy zadań. Średnio na jeden okres dwutygodniowy, na pojedynczego autora, przypadały ok. 3-4 zadania.



Rysunek 1.1: Tablica zadań podczas 7 sprintu, podział na etapy: todo, in progress, ready for test, closed

Praca pisemna natomiast została napisana z użyciem technologi LaTeX [13], zgodnie z szablonem Politechniki Poznańskiej.

Autorzy:

- **Jakub Kolasiński** w ramach niniejszej pracy był jednym z dwóch autorów gry. Był on odpowiedzialny za interfejs użytkownika, mechaniki zapisów gry i zasobów oraz warstwę graficzną. Utworzył też systemy zarządzające klasami przeciwników, jak również wieżyczek.
- Drugim autorem gry był **Sebastian Marcinek**, do jego zadań należały między innymi: implementacja kamery, postaci gracza, mapy po stronie gry, z której następnie powstała pierwsza forma mapy po stronie biblioteki (siatka i jej zarządca) a także pozostałe elementy gry, takie jak implementacja rzeczywistych przeciwników i wieżyczek wewnątrz pętli gry.
- **Krzysztof Mańczak** należał do zespołu autorów biblioteki. W ramach zakresu swojej pracy był odpowiedzialny za wykonanie systemu ziaren generatorów (określanych jako seed) i mechanik z nimi związanych. Do niego należała też odpowiedzialność za generację mapy po stronie biblioteki, na co składały się jej logiczna forma oraz system szukania na niej ścieżek. Odpowiadał także za rejestr w systemie efektów oraz proceduralną generację tekstu, elementów audio, jak i wideo.
- Kolejnym odpowiedzialnym za przygotowanie biblioteki projektowej był **Jakub Szybura**. Wśród jego obowiązków znalazło się: przygotowanie systemu generacji wartości losowych (entropii, wartości skalarnych i wektorowych oraz schematów ataków), rozbudowanie systemu efektów i elementów (planer i logika).

Praca została podzielona na 6 rozdziałów. *Rozdział 1: "Wstęp"* stanowi wyjaśnienie struktury i motywacji stojących za pracą. *W rozdziale 2: "Przegląd rozwiązań i wstęp teoretyczny"*, przedstawiono przegląd wykorzystanych algorytmów, pojęć w pracy oraz inne źródła wiedzy. Wszystkie systemy utworzone w ramach biblioteki i jej struktura zostały wyjaśnione w *rozdziale 3: "Moduł biblioteki do dynamicznego dopasowywania rozgrywki w grach"*. Następnie w *rozdziale 4: "Środowisko Testowe"* opisane zostało krótko jak zostało napisane środowisko testowe biblioteki i co ono wykorzystuje. Podczas pracy zetknięto się z sytuacjami, w których konieczne było dobranie odpowiednich wartości dla parametrów, takich jak np.: rozmiar rzędu dla generatora tekstu, czy parametry odcięć dla poziomów generatora map. W ramach odnalezienia wartości tych parametrów, które zwracałyby jak najkorzystniejsze rezultaty, przeprowadzono szereg testów, których szczegółowy opis zawarto w *rozdziale 5: "Testy i wyniki"*. Ostatni rozdział *"Zakończenie"*, stanowi podsumowanie pracy, w którym opisano dodatkowo możliwe kontynuacje biblioteki, jakie widzą autorzy. Praca zakończona jest spisem cytowanej literatury oraz słownikiem skrótów.

Do utworzenia diagramów zaprezentowanych w pracy zostało wykorzystane narzędzie draw.io [17]. Wykresy pokazane w sekcji 5 zostały natomiast przygotowane z użyciem języku Python[28] i bibliotek pandas [24], seaborn [35], numpy [22], matplotlib [19] oraz scipy [32].

Rozdział 2

Przegląd rozwiązań i wstęp teoretyczny

2.1 Proceduralne generowanie treści w grach

Zbiór metod i narzędzi, wykorzystywany w celu zapewnienia proceduralnej generacji można zebrać pod skrótem PCG (Procedural Content Generation). Celem tego zbioru jest oszczędzenie czasu dewelopera przy maksymalizacji regrywalności poprzez zapewnianie zmiennych zdarzeń o jednolitym charakterze, np. Świat w którym odbywa się rozgrywka jest inny dla każdego gracza poprzez zmianę ułożenia pokoi w budynku. Do jej działania wykorzystuje się ograniczony zbiór wcześniej przygotowanych zasobów do generacji nowych w czasie działania programu, najczęściej we wstępnych fazach wykonania, jako forma inicjalizacji. Dany element jest pre-generowany przed jego wykorzystaniem. Generacja jest ograniczona przez zamysł i przygotowanie dewelopera, system nie wygeneruje czegoś, czego części składowe nie zostaną wcześniej przygotowane [21]. Głównymi zaletami są: wysoka skalowalność, adaptowanie już istniejących danych do nowych potrzeb oraz zapewnianie zróżnicowanych doświadczeń. Klasycznym przykładem użycia takich metod jest generowanie poziomów, statystyk lub elementów audiowizualnych. Deweloper może np. użyć napisanego proceduralnego generatora do stworzenia określonej liczby wariantów przeciwnika, bez potrzeby manualnego ustawiania każdego parametru (punktów życia, modelu, prędkości, ataku, itd.), generator sam ustawi je z puli dostępnych opcji. Tak długo, jak został przygotowany generator i pula części składowych potrzebnych do jego generacji, deweloper jest wstanie zastosować PCG do jakiegokolwiek podsystemu gry. Różne przykłady zastosowania PCG oraz te wybrane do biblioteki zostały przedstawione w rozdziale 2.3.

Kluczowym elementem tych systemów jest pojęcie ziarna *(ang. Seed)*. W inżynierii gier chcemy aby przy małym wejściu algorytm był w stanie wygenerować nową zawartość. Ziarno można rozumieć jako analog do *DNA*, znając je można przewidzieć jak przebiegnie proces tworzenia. Różne ziarna skutkują innymi wynikami, a wykorzystując to samo ziarno uzyskiwany jest determinizm. Najczęściej ziarno stanowi niewielkie źródło entropii, np. ciąg bitów. Cały ten system można rozumieć jako tłumaczenie niewielkich wartości wejściowych na bardziej skomplikowane obiekty o określonej strukturze. W pewnych przypadkach jedno ziarno to za mało do sensownej generacji i wymagane będzie użycie większej ilości dla dostarczenia wystarczającej entropii [18].

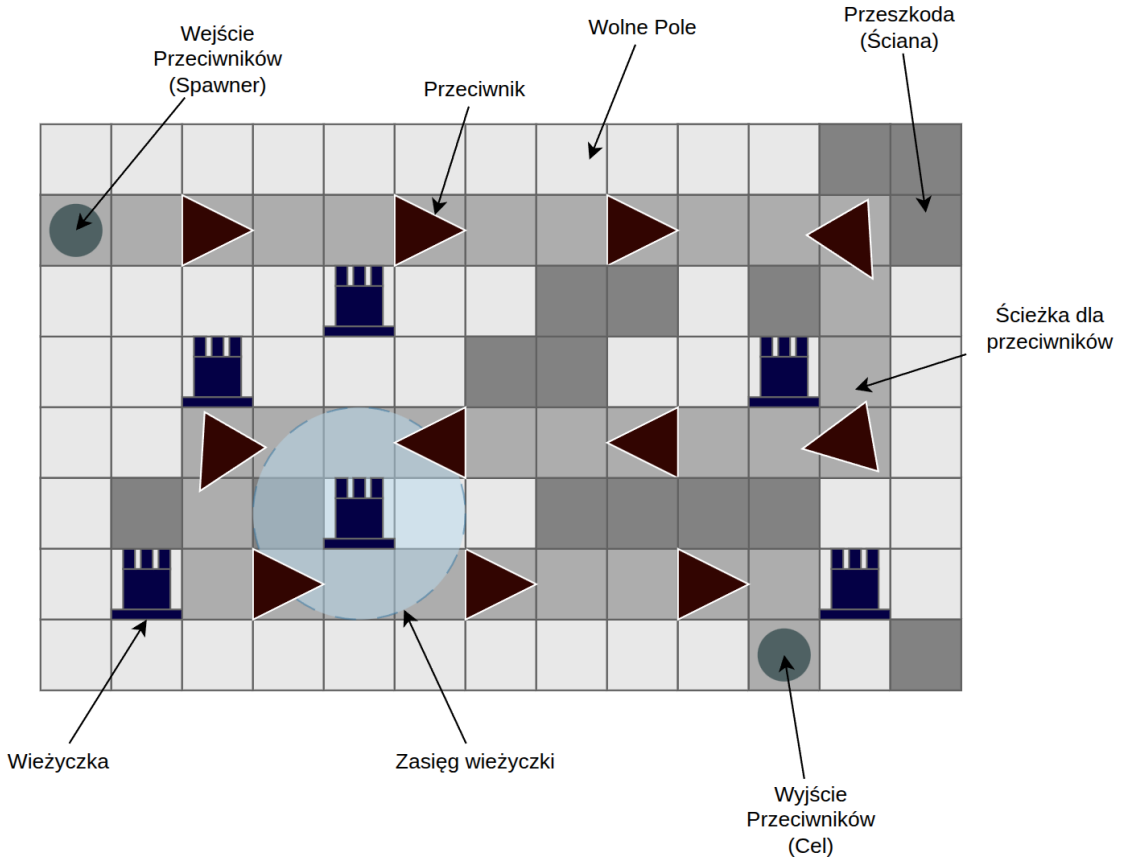
Efekt po stronie gracza jest taki, iż otrzymuje on większą liczbę obiektów w grze, kosztem ich postrzeganej jakości. Używamy przygotowanych zasobów do generacji potencjalnie ogromnej liczby możliwych do uzyskania obiektów w grze. Stanowi to podejście celujące w ilość ponad jakość. Przy ewentualnym nadużyciu generacji jakość doświadczenia jest zatracana ze względu na zauważalny zanik wrażenia unikalności w oczach gracza. Mając za dużą liczbę proceduralnie wygenerowanych obiektów, zaczynają się rzucać w oczy ich podobieństwa, a różnice tracą na wartości [21].

PCG znajduje duże zastosowanie do tworzenia obiektów występujących wielokrotnie, takich jak przeciwnicy, elementy świata lub do rozmieszczenia elementów w świecie. Generowane elementy są ograniczone tylko dostępnymi elementami składowymi, więc mają duże pole na rozszerzenie przestrzeni możliwych wyników. Kontynuując przykład generacji przeciwnika, przy dobrze przygotowanym generatorze wroga można zauważalnie wpłynąć na wynikowych przeciwników poprzez dodanie nowych modeli lub sprite'ów do puli dostępnych. Generator sam w sobie nie został zmieniony, ale poprzez dodanie jednego nowego elementu, pula możliwych wynikowych wariantów została zwiększona o iloczyn wszystkich innych możliwości na pozostałych pulach parametrów.

2.2 Specyfika gatunku Tower Defense (balans, ścieżki, fale przeciwników)

Tower Defense (co można przetłumaczyć na język polski jako "Obrona Wież"), jest gatunkiem gier komputerowych, w którego najczystszej formie celem gracza jest obrona określonego punktu przed zbliżającymi się przeciwnikami. Jak wskazuje nazwa, gracz ma do dyspozycji tak zwane "wieżyczki", czyli obiekty służące do obrony przed atakującymi. Taka wieżyczka najczęściej zwalcza wrogów poprzez strzelanie do nich, jeśli znajdą się w jego zasięgu. Przegraną stanowi sytuacja, w której dojdzie do zniszczenia punktu przez wrogów, a wygrana następuje po obronie celu przez wystarczający czas. Tradycyjnie przeciwnicy poruszają się po jednej predefiniowanej trasie, na końcu której znajduje się wcześniej wspomniany cel obrony. Ważny w tym gatunku jest balans między rosnącym zagrożeniem ze strony przeciwników, a wzrastającymi możliwościami obronnymi.

Gatunek jest na tyle popularny, że istnieje wiele wariantów urozmaicających podstawową formę. Częstymi elementami pętli gry są: podział na fale przeciwników, ulepszenia posiadanych wież oraz zasoby wykorzystywane do budowy wieżyczek jako forma ekonomii. Można też spotkać się z rzadziej występującymi urozmaiceniami rozgrywki, takimi jak np. dynamiczna ścieżka przeciwników. Popularność gatunku miała szczyt we wczesnych latach dwutysięcznych, wraz z popularyzacją technologii Adobe Flash oraz później z upowszechnieniem się urządzeń mobilnych.



Rysunek 2.1: Schematyczna ilustracja podstawowych mechanik gatunku Tower Defense: ścieżka ruchu przeciwników (od punktu początkowego do punktu docelowego), rozmieszczenie wież obronnych oraz ich zasięg rażenia w układzie siatkowym.

Rysunek 2.1 prezentuje przykładową formę standardowej gry w tym gatunku. Przeciwnicy poruszają się od tak zwanego "Spawnera" do ich celu, którego gracz ma bronić. Na siatce postawione są wieże, które będą atakować wrogów. Dana wieżyczka ma zasięg wewnątrz, którego będzie mogła strzelać. Na mapie jest ścieżka dla wrogów, pola puste na których można kłaść wieżyczki, oraz pola na których nie można nic zrobić. Jak opisano w celu pracy (rozdział 1), gatunek TD nie korzysta często z generacji, lecz istnieją przykłady implementacji PCG w grach tego typu. Dla tego gatunku generowane mogą być wieże (dokładniej mówiąc ich parametry, takie jak zasięg, rodzaj strzelanego pocisku, rate of fire itd.), ścieżka dla przeciwników, ale też przeciwnicy sami w sobie. Dokładne porównanie znajduje się w rozdziale 2.3.

2.3 Przegląd istniejących rozwiązań w grach

Aby dokonać wyboru algorytmów zaimplementowanych w pracy inżynierskiej, autorzy najpierw sprawdzili już istniejące rozwiązania w innych grach. Gry Rogue Tower [23] oraz Infinitode 2 [27] są przykładami PCG w grach z gatunku tower defense, służą jako przykłady gier w tym samym gatunku, co demo techniczne biblioteki i pełniły na etapie projektowania funkcję porównania oraz minimum funkcjonalności, które biblioteka powinna wspomagać. Do porównania zostały też uwzględnione Rogue [5] oraz No Man's Sky [7]. Rogue [5] jest starszą grą, na podstawie której powstał cały gatunek, został wybrany do porównania ze względu na bycie klasycznym i pionierskim przykładem PCG. NMS [7] natomiast, ze względu na bycie bardzo ambitnym projektem zawierającym szerokie zastosowanie generacji, jest grą symulującą cały wszechświat z licznymi planetami, przedmiotami i stworzeniami, więc służył za inspirację mechanik nieznajdujących wcześniej zastosowania w proceduralnym generowaniu w obronie wież. Inspiracje do zaimplementowanego systemu audiowizualnego w bibliotece znaleziono właśnie poprzez rozszerzenie horyzontów poza gatunek Tower Defense.

Tabela 2.1: Porównanie generacji proceduralnej w wybranych grach oraz autorskiej biblioteki.

Funkcjonalność	**Rogue Tower**	**Infinitode 2**	**TDPG**	**Rogue**	**No Man's Sky**
Gatunki	TD	TD	**TD**	Roguelike	Survival
Generowanie mapy	✓	✗	✓	✓	✓
Dynamiczne szukanie ścieżek	✓	✓	✓	✓	✓
Proceduralne efekty	✗	✓	✓	✗	✗
Proceduralne audio	✗	✗	✓	✗	✓
Proceduralni przeciwnicy	✗	✗	✓	✗	✓
Zmiana palet barw	✗	✗	✓	✗	✓
Proceduralne obiekty gracza	✓	✓	✓	✓	✓

Jak przedstawiono w tabeli 2.1, rozwiązania proceduralne są niejednokrotnie wykorzystywane w grach do kilku szczególnych zagadnień, takich jak generowanie mapy, czy wytwarzanie efektów, jakie charakteryzują dostępne dla gracza i przeciwników ataki. Autorzy zdecydowali się uwzględnić w bibliotece narzędzia do generacji wszystkich przedstawionych tam funkcjonalności.

Poprzez generowanie mapy rozumie się jakąkolwiek formę generacji środowiska, w którym toczy się rozgrywka, co osiągane jest z wykorzystaniem systemu. Wszystkie gry z wyjątkiem Infinitode 2 [27] implementują generację proceduralną, ta gra zaś zamiast generacji oferuje system manualnego tworzenia map oraz udostępniania ich między graczami. Rogue Tower [23] generuje mapę poprzez dokładanie nowych fragmentów ścieżek i rozszerzanie mapy, Rogue [5] poprzez pre-generowanie sieci pokoi i połączeń między nimi, a No Man's Sky [7] posiada zaawansowany i skomplikowany generator planet oraz systemów w kosmosie, budujący nie tylko florę i faunę, ale też podziemne systemy jaskiń. Każda z rozpatrywanych gier oferuje jakąś formę szukania ścieżki, Rogue [5] ze względu na wiek i limity systemów operacyjnych, na które został wypuszczony, implementuje najprostszą formę pathfindingu, zmodyfikowany wariant algorytmu zachłannego [33]. Co do pozostałych gier nie ma możliwości zweryfikowania używanego wewnątrz algorytmu.

Jako funkcjonalność proceduralnych efektów szukane było, czy generacja zwraca więcej niż zaledwie wypełnianie statystyk. Oczekiwane było, aby dochodziło np. do niepredefiniowanych interakcji między podsystemami.

Obiekt gracza natomiast odwołuje się do obiektów, które służą graczowi w trakcie rozgrywki lub nad którymi może panować, np. wieżyczka, broń, ulepszenia itd. Infinitode 2 [27] implementuje

taki system przez prawie nieskończone drzewo rozszerzeń do wieżyczek. W grze No Man's Sky [7] znajduje się bardzo zaawansowany algorytm tworzenia zwierząt i innych żyjących NPC. Ten system stanowił dla autorów pracy inspirację odnośnie implementacji zmiennego wyglądu i dźwięków w grze. Implementacja różni się od tej zawartej w grze, ponieważ zamiast wytwarzać modele 3D poprzez mieszanie mniejszych modeli ze sobą, biblioteka jest przygotowywana z myślą o grach dwuwymiarowych.

Każda gra uwzględniona w tym porównaniu posiada pewne obiekty, które są generowane proceduralnie. Minimalnie są to proste przedmioty, takie jak ilość złota lub broń o innych statystykach, jak w Rogue [5], wieże o innym zasięgu lub statystykach w Rogue Tower [23], czy skomplikowana generacja wieży w Infinitode 2 [27], zawierająca system efektów. No Man's Sky [7] w tej kategorii oferuje np. generację statków kosmicznych dostępnych dla gracza.

Biblioteka TDPG oferuje szeroką gamę metod PCG, dając deweloperowi więcej możliwych systemów do generowania treści, niż w przypadku znalezionych przedstawicieli gatunku tower defense, które implementują PCG.

2.4 Przegląd literatury i innych źródeł wiedzy

W książce "Procedural Content Generation in Games" [21] znajdują się opisy metod PCG w grach wideo, stanowiła ona początek szukania algorytmów, które mogłyby być zaimplementowane. Wewnątrz książki znajduje się podstawowy opis czym jest PCG (1.1), przykłady gier (1.2) oraz właściwości, które powinien mieć dobry system PCG (1.5). Dodatkowo w rozdziale 4 książki poruszony jest temat szumów i generacji terenu bazując na nich, ten mechanizm został zaimplementowany w bibliotece. Wielokrotnie w dziele jest też poruszony temat ziarna (seed).

Rozważana była generacja mapy na bazie szumu, predefiniowanych kształtów lub fraktali, tak, jak zostało pokazane w książce [21]. Autorzy skłaniali się ku implementacji szumów, do ostatecznej decyzji doszło jednak po analizie kolejnych artykułów na ten temat. Artykuł "Perlin Noise Pixel Shaders" [10] opisuje dokładnie działanie i badane w tamtym czasie implementacje szumu Perlina, jego zakres jest zatem węższy od zakresu książki, ale dobrze opisuje tę tematykę. Natomiast artykuł "Perlin noise and its improvements: A literature review" [36] został wybrany, ponieważ zawiera opis nowoczesnych metod generacji szumu i ich optymalizacji. Wybór sposobu generacji szumu w bibliotece bazuje właśnie na tym artykule i został opisany dokładniej w tabeli 2.3.

Aby system mógł sam generować formę pseudo losowości, należało zaimplementować ziarna oraz stojący za nimi system PRNG. Wiedza na ten temat została rozszerzona poprzez artykuły naukowe "Seeds for Random Number Generators" [18] oraz "Pseudorandom Number Generators for Cryptographic Applications" [31]. Praca Andrei Röck stanowi podstawę teoretyczną oraz opisy praktycznych implementacji generatorów liczb losowych do kryptografii. Autorzy wybrali to dzieło ze względu na część teoretyczną oraz wyjaśnienie różnicy między generatorami losowymi a pseudolosowymi. Tekst "Seeds for Random Number Generators" dotyczył natomiast tematu ziaren. Jest on szczególnie ważny ze względu na poruszenie problemu generatorów z jednym ziarnem i możliwego zapotrzebowania na większą entropię. W szukaniu wiedzy na temat algorytmów Pathfinding wybrane zostały dzieła akademickie "A Guide to Heuristic-based Path Planning" [3] oraz "Generalized Best-First Search Strategies and the Optimality of A*" [30], ale przydatna okazała się też strona internetowa zawierająca implementacje i wizualizacje wielu algorytmów www.redblobgames.com [25]. Pierwszy artykuł stanowi przegląd i porównanie algorytmów stosowanych w robotyce do wyznaczania optymalnych tras w czasie rzeczywistym oraz wskazówki dotyczące dobrania odpowiedniego algorytmu w zależności od środowiska oraz czasu obliczeniowego, analiza dzieła była podstawą do utworzenia tabeli 2.2 zawierającej zestawienie algorytmów.

Następnie dla poszerzenia tematu algorytmu zachłannego oraz A*, sprawdzono dzieło autorstwa Riny Dechter i Judea Pearla. Artykuł opisuje poddany analizie matematycznej algorytm A*, pod względem "optymalności A*", rozumianej w kontekście artykułu jako przekonanie, że ten algorytm rozwija minimalną liczbę węzłów spośród wszystkich algorytmów korzystających z tej samej wiedzy heurystycznej. Ze strony internetowej natomiast wykorzystano przykład implementacji trzech algorytmów, zachłannego, A* oraz Algorytmu Dijkstry, ponieważ zawiera środowisko interaktywne pokazujące wszystkie trzy z tym samym problemem, co pomogło w zrozumieniu ich trzech oraz ostatecznym wyborze.

2.5 Analiza i wybór algorytmów/rozwiązań

Przed rozpoczęciem pracy nad projektem należało dokonać wyboru odpowiednich rozwiązań. Kluczowymi do rozważenia były te, których struktura jest nietrywialna.

Jak zaznaczono w sekcji 2.3, jednym z ważniejszych algorytmów dla biblioteki jest szukanie ścieżek, polegające na wyznaczeniu listy kroków lub pozycji przejściowych pomiędzy dwoma punktami. Po przeglądzie literatury autorzy skłaniali się ku algorytmowi A*, ale badane były jeszcze 3 inne: natywne rozwiązanie w silniku Unity, Algorytm Dijkstry oraz zachłanny algorytm Flow Field (BFS). Najważniejsze w wyborze jest, aby algorytm był opłacalny pod względem obliczeniowym, aby nie zakłócał reszty systemu poprzez opóźnienia oraz aby uzyskiwał jak najlepszy wynik ścieżki pod względem ilości kroków.

Wyszukiwanie ścieżek:

Tabela 2.2: Porównanie rozważanych algorytmów wyszukiwania ścieżki.

Kryterium oceny	**Unity NavMesh**	**A***	**Dijkstra**	**Flow Field**
Optymalność wyznaczonej trasy	✓	✓	✓	✗
Szybkie wyznaczanie ścieżki	✓	✓	✗	✓
Obsługa kosztów terenu	✓	✓	✓	✗
Łatwość implementacji na siatce	✗	✓	✓	✓
Działa dobrze z dziesiątkami przeciwników	✓	✓	✓	✓
Działa dobrze z setkami przeciwników	✗	✗	✗	✓
Mały koszt dynamicznego odświeżania	✗	✓	✗	✓

Źródło: Opracowanie własne na podstawie analizy literatury i implementacji [34] [3] [30] [25]

Rozwiązanie w silniku Unity automatycznie generuje siatkę nawigacyjną na geometrii poziomu, pozwalając znajdującym się tam agentom na poprawne poruszanie się w środowisku 3D. Algorytm A* polega na wykorzystaniu heurystyki oraz listy priorytetów w celu wyznaczenia kolejnego do odwiedzenia punktu w przeszukiwaniu. Algorytm Dijkstry to klasyczny algorytm gwarantujący znalezienie najkrótszej ścieżki w grafie, który może służyć do szukania ścieżek w grze przy transformacji mapy na graf. Flow Field to rozwiązanie zachłanne, najlepiej sprawdzające się przy ogromnej liczbie agentów na mapie, generuje na całej scenie pole wektorowe wskazujące kierunek poruszania się zależnie od aktualnie zajmowanej pozycji.

Wszystkie algorytmy z wyjątkiem zachłannego potrafią wyznaczyć optymalne ścieżki pod względem odległości, nawet na skomplikowanych mapach. W wielu przypadkach jest to dokładnie ta sama trasa. Algorytm Flow Field natomiast poprzez swoją prostotę zwraca pierwszą (nieoptymalną) odkrytą trasę.

Pod względem prędkości obliczania zadowalające są A*, NavMesh oraz Flow Field. Najszybszy jest algorytm zachłanny, następnie NavMesh oraz A*. NavMesh jest szybki w obliczaniu, natomiast wymaga, aby przed obliczeniem została utworzona dla niego specjalna siatka, osobna od siatki mapy. Gdyby doliczyć czas tworzenia siatki, wówczas wynik byłby mniej zadowalający. Dijkstra jest wolniejszy od reszty, ale nie spowalniałby systemu aż tak, aby odrzucić go tylko z tego powodu.

Ważnym elementem rozważanym w porównaniu jest możliwość obsługi kosztów terenu przez algorytm. Algorytm ma być świadomy, że nie wszystkie pola są równe sobie, pozwala to np. na implementację gór, które są trudniejsze do pokonania. W tym kryterium ocena jest binarna, algorytm albo jest do tego zdolny, albo nie. Jedynie Flow Field nie zawiera tego w swoich możliwościach.

Rozwiązanie wbudowane wymaga dodatkowej siatki, co jest dużym dodatkiem pracy i dla systemu gry, ale także dla dewelopera oraz autorów biblioteki, więc zostało to wzięte pod uwagę w porównaniu.

Algorytmy są też porównane pod względem obsługi liczby przeciwników. W gatunkach takich, jak TD występować może znaczna liczba przeciwników. Chcąc dać każdemu z nich własną ścieżkę stanowi duże wyzwanie. Każdy z rozważanych algorytmów potrafi poradzić sobie z małą ilością niezależnych agentów, jednakże przy setkach, lub większej liczbie przeciwników, jedynie algorytm zachłanny nie będzie znacząco tracił na czasie obliczeń, ze względu na potrzebę ustalenia pola wektorowego tylko jednokrotnie.

Ostatnie porównanie polega na koszcie odświeżania. W bibliotece ważnym było, aby mapa mogła się zmieniać dynamicznie podczas rozgrywki, więc aby tego dokonać należało wybrać algorytm mogący szybko poprawić lub wygenerować nową ścieżkę. Flow Field wymagałby regenerowania ścieżki, lecz koszt tego jest niewielki. A* natomiast może utworzyć nową ścieżkę, a ze względu na to, że aktor przeszedł już kawałek poprzedniej, przy ponownej generacji fragment mapy do sprawdzenia będzie mniejszy.

Wybrany został algorytm A*, który jest też standardem na rynku gatunku TD [25]. Najważniejszymi kryteriami w ocenie były optymalność (najmniejszy możliwy koszt) trasy, obsługa kosztu terenu (różne pola na siatce mają inne wagi, które są dodatkowym kosztem do uwzględnienia poza samą odległością) oraz niska złożoność obliczeniowa odświeżania ścieżki. A* jest jedynym spełniającym wszystkie te 3 kryteria.

Generacja szumu:

Generowanie szumu Perlinowskiego jest skomplikowanym procesem i wymagałoby dużego nakładu pracy, aby zaimplementować je własnoręcznie, dlatego w bibliotece tworzonej do pracy inżynierskiej wykorzystywane jest już gotowe rozwiązanie w postaci biblioteki FastNoiseLite [26]. Rozwiązanie to wspomaga kilka metod generacji szumu, wybór został podjęty na bazie literatury dotyczącej szumu (opisanej w rozdziale 2.4) oraz na bazie dokumentacji biblioteki.

Tabela 2.3: Porównanie cech algorytmów generowania szumu dostępnych w bibliotece FastNoiseLite (ranking 1–4, gdzie 1 oznacza wynik najlepszy).

Kryterium oceny	**Perlin**	**OpenSimplex2**	**OpenSimplex2S**	**Value**
Brak artefaktów siatki (grid bias)	2	1	1	3
Naturalność wizualna (organiczność)	3	2	1	4
Wydajność obliczeniowa	2	1	3	1
Ciągłość gradientu (gładkość)	3	2	1	4
Złożoność obliczeniowa	2	2	3	1
Suma punktów (im mniej, tym lepiej)	**12**	**8**	**9**	**13**

Źródło: Opracowanie własne na podstawie testów biblioteki, zweryfikowane literaturą [36]

Metoda "Perlin" dotyczy klasycznego szumu gradientowego, tworząc płynne naturalne struktury, jednak ma znaną wadę w postaci tak zwanych artefaktów osiowych (układanie się wzorów wzdłuż kolumn macierzy). OpenSimplex2 oraz OpenSimplex2S to nowoczesne, szybkie alternatywy dla szumu Perlina, eliminujące artefakt kierunkowy. Wariant OpenSimplex2S dodatkowo "wygładza" (Smoothed) jeszcze bardziej przejścia między wartościami. Value to najprostsza metoda tworzenia szumu, wykorzystuje interpolacje losowych wartości punktów.

Metody OpenSimplex2 sprawdzają się najlepiej pod względem eliminacji artefaktów. Perlin oraz Value zostawiają widoczne nienaturalne wzory przy dużych mapach.

OpenSimplex2S zwraca najbardziej naturalne szumy, między ich wartościami znajdują się mają małe, gładkie przejścia. OpenSimplex2 jest mniej gładki, lecz wciąż naturalny, natomiast Perlin i Value są bardziej blokowe. Kryterium to dotyczy tego, jak bardzo naturalnie wygląda końcowy szum oraz gładkości przejść między pojedynczymi pikselami.

Czas generacji szumu jest najgorszy w OpenSimplex2S ze względu na dodatkowe wygładzanie, natomiast najlepsze pod tym względem są Value oraz OpenSimplex2, Value będąc metodą minimalnie szybszą ze względu na mniejszą złożoność.

Dzięki porównaniu kryteriów typów szumu, wybrany został *OpenSimplex2*, który stanowi złoty środek między czasem generacji, a jakością wynikowej mapy. *OpenSimplex2S* daje bardziej gładki wynik (mniejsze różnice między sąsiadującymi wartościami w macierzy), jednak jest mniej optymalny pod innymi parametrami. Z powodu późniejszej normalizacji szumu do tylko 3 wartości, czas generacji jest ważniejszy.

Struktura danych do rejestru elementów:

Biblioteka zawiera system elementów (opisany w rozdziale 3.3.3), który wymaga struktury danych do przechowywania rejestru elementów. Rejestr ma formę grafu, gdzie zapisana jest relacja między różnymi elementami, dla działania pseudo-genetyki elementów. Do implementacji grafu wykorzystana jest biblioteka QuikGraph [29] oferująca wiele form reprezentacji grafów w kodzie.

Tabela 2.4: Porównanie struktur danych do przechowywania rejestru elementów i ich wzajemnych relacji.

Kryterium oceny	**Macierz sąsiedztwa**	**Graf skierowany**	**Graf dwukierunkowy**
Dostęp do obiektu po ID	$O(1)$ (indeks tablicy)	$O(1)$ (klucz słownika)	$O(1)$ (klucz słownika)
Wyszukiwanie potomków (Out-edges)	$O(n)$	$O(k)$	$O(k)$
Wyszukiwanie rodziców (In-edges)	$O(n)$	$O(n+e)$	$O(k)$
Złożoność nawigacji wstecznej	Średnia	Wysoka	Bardzo niska
Efektywność wykorzystania pamięci	Niska ($O(n^2)$)	Wysoka ($O(n+e)$)	Wysoka ($O(n+2e)$)
Elastyczność i skalowalność struktury	Niska	Wysoka	Wysoka
Decyzja	**Odrzucono**	**Odrzucono**	**Wybrana**

Oznaczenia: n – liczba elementów, e – liczba relacji, k – stopień (liczba powiązań) danego węzła.

Rozważane były trzy reprezentacje. Macierz sąsiedztwa polega na dwuwymiarowej macierzy, gdzie dane wierzchołki są indeksami, a w komórkach są zapisane połączenia z innymi wierzchołkami. Graf skierowany bazujący wewnętrznie na strukturze słownikowej do przechowywania wierzchołków oraz list krawędzi wychodzących z wierzchołka. Graf dwukierunkowy jest rozszerzeniem grafu skierowanego, zawierający relacje w obie strony krawędzi. Ze względu na wewnętrzne użycie słownika przez bibliotekę, wszystkie metody mają stały czas dostępu do obiektu w wierzchołku poprzez ID.

Wyszukanie potomka, czyli wierzchołka, który znajduje się na końcu krawędzi wychodzącej, jest zależne od liczby połączeń dla grafów, a dla macierzy od liczby elementów w niej zawartych. Nie są one więc pod tym względem porównywalne. Szukanie rodzica, czyli relacji odwrotnej do potomka, jest natomiast takie samo w przypadku macierzy oraz grafu dwukierunkowego. Graf skierowany wymaga przeszukania potencjalnie każdego elementu i jego relacji, aż do znalezienia takiego, który zawiera w liście potomków szukaną relację.

Nawigacja wsteczna jest silnie zależna od odszukania rodzica i jest najszybsza w grafie dwukierunkowym, ze względu na strukturę przeszukiwania grafu, która jest już wbudowana w taką reprezentację.

Macierz zajmuje najmniej pamięci, aby być przechowywana, ponieważ rozmiar jest zależny tylko od liczby elementów, która w rejestrze jest zawsze mniejsza od liczby relacji. Natomiast najwięcej miejsca wymaga graf dwukierunkowy, bo musi zapisać element i jego relacje wchodzące oraz wychodzące. Grafy są strukturami łatwymi do rozszerzenia, dodania nowych elementów nieprzewidzianych na starcie, macierz natomiast wymaga rozszerzenia i potencjalnego przeniesienia elementów wewnątrz struktury. Wybrana implementacja *BidirectionalGraph* z biblioteki QuikGraph [29] wykorzystuje wewnętrznie strukturę słownikową do przechowywania wierzchołków oraz list krawędzi wchodzących i wychodzących, dzięki czemu dostęp po indeksach jest porównywalnie szybki do macierzy. Ponieważ ważna jest relacja potomka do rodziców, ale też rodzica do potomków, graf skierowany w obie strony okazał się najlepszym rozwiązaniem do projektu.

Rozdział 3

Moduł biblioteki do dynamicznego dopasowywania rozgrywki w grach

3.1 Wykorzystane technologie i biblioteki zewnętrzne

Jak wspomniano wcześniej w rozdziale (1), biblioteka TDPG została przygotowana do silnika *Unity* [34]. Na samym początku pracy rozważany był także silnik "Godot" [4], jednakże autorzy zdecydowali się przygotować pracę celującą w narzędzie z większą bazą deweloperów.

Do rejestru elementów tak, jak wspomniano w sekcji wyborów rozwiązań (tabela 2.4), został wybrany graf, aby zarządzać grafem została wykorzystana biblioteka *QuikGraph* [29] ze względu na wydajność i łatwość integracji ze względu na implementacje w C#.

Szum natomiast jest generowany z wykorzystaniem biblioteki *Fast Noise Lite* [26]. Do jej wyboru przekonała ilość parametrów udostępnionych w produkcji szumu, dając większą kontrole.

Dodatkowo aby zapewnić możliwość zapisu i odczytu danych wygenerowanych w bibliotece, autorzy chcieli zapewnić serializację. Do tego popularnym wyborem na rynku jest biblioteka *JSON.Net* [15]. Przy importowaniu biblioteki TDPG do własnego projektu, należy dodać też com.unity.nuget.newtonsoft-json.

3.2 Założenia projektowe i architektura

Biblioteka została przygotowana pod Unity, w natywnych dla silnika rozwiązaniach: C#, High Level Shader Language (HLSL), Shader Lab oraz testowana z użyciem NUnit. Biblioteka jest podzielona na sektory, korzystając z przestrzeni nazw (namespace). Do plików wewnątrz każdej z przestrzeni nazw można się odwoływać trzymając się określonego schematu:

TDPG	.	[Nazwa przestrzeni]	.	[Nazwa podprzestrzeni]
nazwa biblioteki		np. "Generators"		np. "Seed"

Tabela 3.1: Wzór przestrzeni nazw funkcjonujących wewnątrz biblioteki, trzymający się standardów języka C# [2] z pominięciem nazwy firmy.

Dzięki takiej strukturze biblioteka jest bardziej czytelna. Przestrzenie grupują ze sobą algorytmy i rozwiązania zbliżone do siebie celem, każda z nich pełni w bibliotece określoną rolę, umożliwiając skorzystanie z funkcjonalności zawartych wewnątrz niej.

W bibliotece znalazły się przestrzenie odpowiadające za:

- Generatory RNG (TDPG.Generators)
  Przestrzeń zawiera w sobie funkcjonalności generowania liczb pseudolosowych oraz ziaren. Służy ona jako serce całego projektu, pełniąc rolę źródła, ewolucji i zarządzania entropią w pozostałych systemach. Poza integracją z resztą przestrzeni nazw, zawarte w Generatorach RNG klasy same mogą obsługiwać entropię. Są w stanie wykorzystać ją samodzielnie do tworzenia deterministycznych wartości liczbowych. Przestrzeń wykorzystuje istniejące już rozwiązanie "SplitMix64" [16] jako źródło entropii, oraz implementuje koncept ziaren jako formę wartości inicjalizującej pomiędzy systemami. Wewnętrzna implementacja systemów jest kontrybucją autorów.

- System efektów (TDPG.EffectSystem)
  Biblioteka wspomaga autorską implementację pomysłu ewoluującego systemu efektów. Celem tego systemu jest udostępnienie obsługi struktur obiektów, pozwalających na wprowadzenie modyfikatorów zapewniających różnorodność rozgrywki poprzez formę "żywiołów"/"elementów" tak, jak jest to typowe w grach wideo. System zawiera klasę elementu, który składa się z efektów wpływających na rozgrywkę. Dany efekt jest zdefiniowany i zaimplementowany wewnątrz tej przestrzeni. Elementy są przechowywane oraz generowane poprzez dedykowany im rejestr zawarty w bibliotece. Struktura rejestru jest wewnętrznie utrzymywana jako graf, korzystając z biblioteki *QuikGraph* [29], natomiast wszystkie funkcjonalności związane z implementacją efektów, elementów oraz ich mieszania i generacji są autorskie.

- Generacja tekstu (TDPG.TextGeneration)
  Generacja tekstu została dodana do biblioteki, aby zapewnić większą różnorodność i indywidualność tworzonych obiektów, przeciwnik z własnym imieniem prezentuje się inaczej, niż taki, którego od innych odróżnia wyłącznie inny numer. System może być wykorzystany do nadania nazw generowanym obiektom, ale także jako zalążek do sytemu generowania tzw. environmental storytelling, nadając imitację ludzkiego pierwiastka w proceduralnie wygenerowanym przedmiocie. Generacja tekstu opiera się na istniejącym już stochastycznym procesie łańcuchów Markowa, autorzy rozszerzyli ten mechanizm o dodatkowe funkcje w formie listy zabronionych wyrazów oraz konkatenacji wymuszonych początków lub końców do wygenerowanego wyrazu.

- Generacja wideo (TDPG.VideoGeneration)
  Aby zapewnić efekt różnorodności na poziomie wizualnym, zaimplementowany jest mechanizm tworzenia tzw. palette swapów. System ten istnieje, aby zapewnić różnicę w generowanych obiektach na pierwszym etapie widocznym dla gracza, czyli jak obiekt wygląda. Generacja wideo może być wykorzystana dla jakiegokolwiek obiektu ze spritem w grze, więc może być nałożony na np. elementy świata, postać gracza, przeciwników itd. Palette swap jest rozwiązaniem wykorzystanym w wielu grach, nie jest to autorskie rozwiązanie, jedynie implementacja. System został zaimplementowany poprzez napisany przez autorów program cieniujący, wspomagany przez silnik Unity.

- Generacja Audio (TDPG.AudioModulation)
  W ramach biblioteki oferowany jest generator audio, aby zapewnić większą liczbę dźwięków i muzyki w grze. Generacja audio polega na modyfikacji podanego klipu audio przez listę różnych modyfikatorów, działających na głośności, tonie, panoramie itd. danego klipu. Dzięki temu systemowi mając limitowaną liczbę przygotowanych klipów audio oraz modyfikatorów można zapewnić unikalny dźwięk dla danego zdarzenia w grze, np. inny dźwięk "krzyku" dla przeciwnika. Sam pomysł takiego systemu nie jest oryginalny, został zaimplementowany już w innych grach (np. we wcześniej wspomnianym No Man's Sky [7]), natomiast sama implementacja do modyfikacji dźwięku w silniku Unity została wykonana w całości przez autorów, nie bazując na żadnym gotowym rozwiązaniu.

- Szablony elementów gry (TDPG.Templates)
  Ostatnia z przestrzeni nazw ma na celu zapewnić deweloperowi prosty interfejs do implementacji elementów z biblioteki. W tej przestrzeni znajdują się szablony oraz podsystemy elementów typowych do gry typu tower defense, takich jak: generacja mapy, szukanie ścieżek, ale też podstawy implementacji wieżyczek oraz przeciwników. Te klasy zostały stworzone aby ułatwić implementacje biblioteki w silniku Unity, może zostać wykorzystana jako "most" między grą, a wewnętrznymi częściami biblioteki. Implementowane systemy bazują na gotowych rozwiązaniach takich jak algorytm A*, FastNoiseLite [26] oraz pomysł translacji szumu na mapę w grze.

3.3 Biblioteka

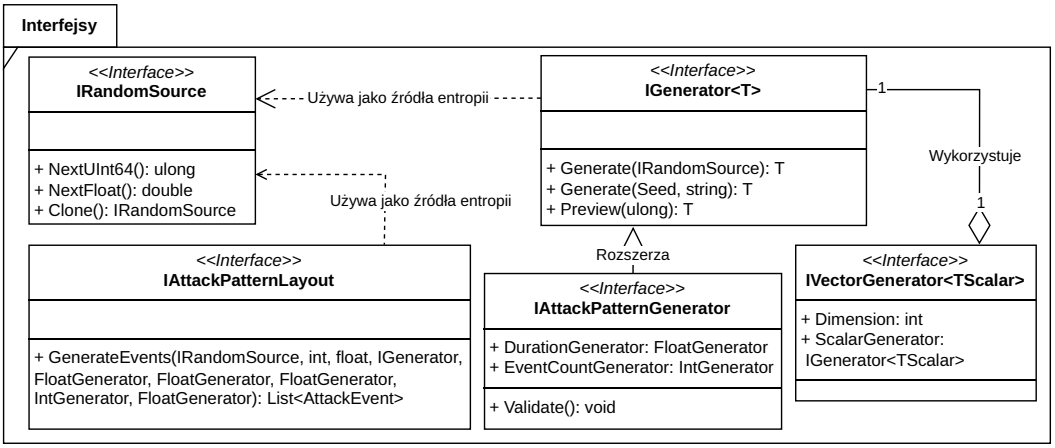
Poniższa dokumentacja biblioteki zawiera diagramy UML w celu wizualizacji struktury rozwiązań. W pewnych miejscach diagramy są uproszczeniami rzeczywistej biblioteki, niepełności są oznaczone (jako "...") . Pełne diagramy z każdą relacją i każdą funkcją nie byłyby czytelne. Pominięta na każdym diagramie UML jest klasa konwersji do serializacji, ze względu na schematyczną budowę.

3.3.1 Generatory RNG

Kluczową funkcjonalnością oferowaną przez bibliotekę jest generacja liczb pseudolosowych w celu obsługi dalszych systemów w bibliotece oraz kontroli entropii. System ten składa się z trzech generowanych struktur: liczb, ziaren oraz schematów ataków. Wszystkie z tych typów są przydatne przy tworzeniu gier, czy to jako wartości statystyk, czy też źródło entropii innych generatorów, bądź też wzorzec modelujący zachowanie jednostek w trakcie ataku.

Generacja liczb pseudolosowych obsługuje generacje *liczb skalarnych* oraz *wektorów liczb skalarnych*. Generacja ziarna jest bardziej niezależna w porównaniu z pozostałymi trzema podprzestrzeniami RNG.

Do tych trzech zostały przygotowane interfejsy widoczne na rysunku 3.1. Interfejs *IRandomSource* stanowi kluczowy element modułu generatorów, jako, że określa funkcje zwracające wartości pseudolosowe o określonych typach. Zawarta jest w nim również funkcja tworząca kopię zawartej entropii w danym stanie, celem umożliwienia tych samych rezultatów w różnych kontekstach. Funkcjonalności w nim zawarte wykorzystywane są jako źródła entropii wewnątrz interfejsów generatorów po stronie biblioteki, które jej bezpośrednio wymagają do działania: *IGenerator* oraz *IAttackPatternLayout*. Pierwszy z nich zapewnia funkcje pozwalające przy użyciu określonego źródła zwrócić deterministyczną wartość pseudolosową wybranego typu (co było osiągalne przy użyciu mechanizmu szablonów), natomiast drugi określa funkcję generującą oraz porządkującą otrzymane wartości w listę struktur typu *AttackEvent*, będących pojedynczymi składowymi klasy *AttackPattern*, opisanej w podprzestrzeni AttackPatterns. Rozszerzeniami interfejsu *IGenerator* są *IAttackPatternGenerator*, dostarczający funkcji, które zwracają pozostałe wartości wykorzystywane w *AttackPattern*, a także *IVectorGenerator*, służący do otrzymywania nie pojedynczych wartości odpowiadających określonym zmiennym, jak wcześniej, lecz sekwencji określonego typu o wybranym rozmiarze.

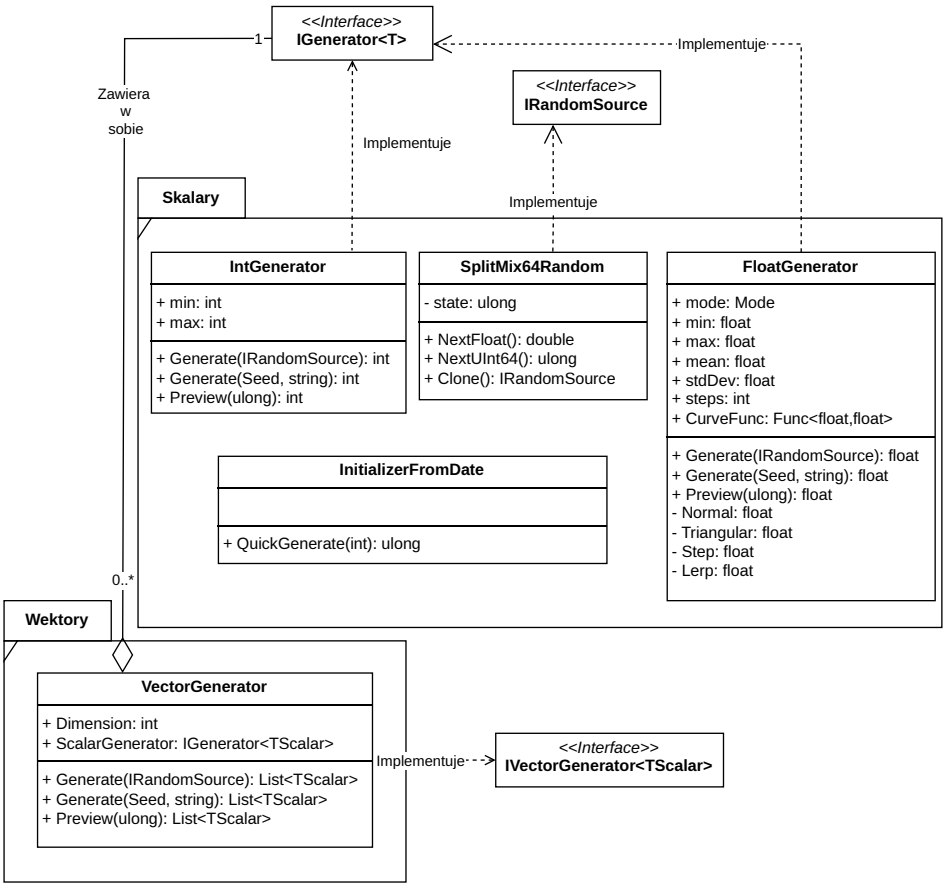


Rysunek 3.1: Diagram UML TDPG.Generators.Interfaces, przedstawiający relację między klasami interfejsów generatorów. Składa się z 5 klas interfejsów, głównym jest IRandomSource, od którego wychodzi dalsza generacja.

Liczby

Interfejs *IRandomSource* jest implementowany wewnątrz podprzestrzeni TDPG.Generators.Scalars w klasie *SplitMix64Random*, która jest implementacją istniejącego rozwiązania. Wykorzystywana jest ona jako generator 64-bitowej pseudolosowej entropii na podstawie wartości inicjującej. Podprzestrzeń skalarów zawiera ponadto dwie klasy implementujące interfejs *IGenerator*: *IntGenerator* oraz *FloatGenerator*. Służą one do generacji dwóch istotnych typów wartości liczbowych: liczb całkowitych oraz zmiennoprzecinkowych.

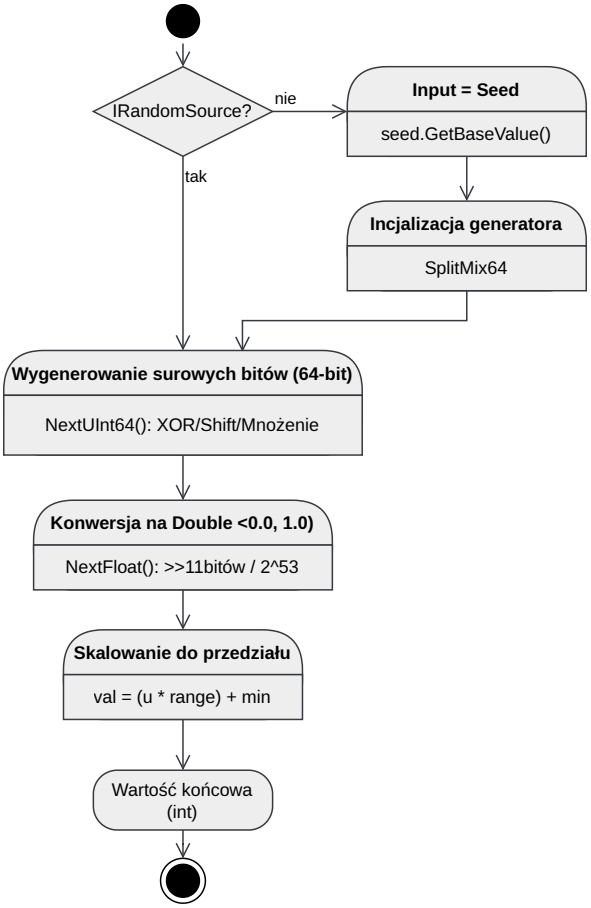
Zdefiniowana oprócz nich jest tam także pomocnicza klasa *InitializerFromDate*, używana do pozyskania wartości liczbowej na podstawie odczytu zegara systemowego, jest to szybkie wstrzyknięcie entropii zewnętrznej. *IGenerator* zawarty jest również wewnątrz klasy *VectorGenerator*, znajdującej się jako jedyna w podprzestrzeni TDPG.Generators.Vectors. Implementuje ona interfejs IVectorGenerator, umożliwiając tym samym pozyskanie określonej liczby wartości danego typu.



Rysunek 3.2: Diagram UML TDPG.Generators.Scalars oraz TDPG.Generators.Vectors. Pokazuje zależność IntGenerator, FloatGenerator oraz VectorGenerator od interfejsu IGenerator. Dodatkowo zilustrowana jest implementacja IRandomSource poprzez klase SplitMix64Random, oraz implementacja IVectorGenerator przez rzeczywisty generator wektorów liczb.

Aby wygenerować liczbę całkowitą należy podać pewną wartość wejściową stanowiącą źródło entropii w formie obiektu IRandomSource (Rysunek 3.1). Możliwe jest również zainicjowanie procesu generacji poprzez ziarno, wówczas wybierana jest z ziarna entropia. Na diagramie prezentującym tą generacje (Rysunek 3.3) jest to przy pomocy SplitMix64Random. Po inicjalizacji generatora najpierw generowane są bity, korzystając z rezultatu SplitMix64. Nawet jeden bit w da-

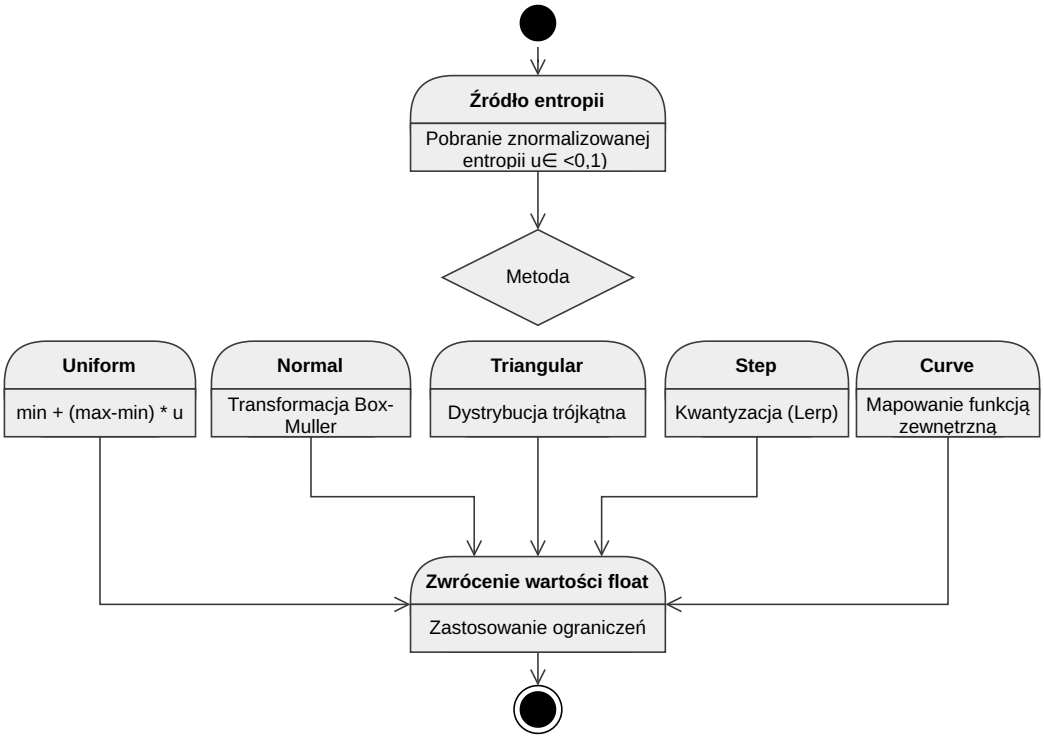
nych wejściowych skutkuje diametralnie różnymi danymi wyjściowymi. Następnie otrzymane bity tłumaczone są na double, który finalnie jest skalowany do przedziału wartości ustalonych dla generatora. Generator wektorowy wykona tą operację wtórnie aż do uzyskania pełnego wektora liczb.



Rysunek 3.3: Algorytm generacji liczby całkowitej, bazujący na rozwiązaniu SplitMix64

Rysunek 3.3 pokazuje krok po kroku przejście poprzez proces generowania liczby całkowitej. Proces rozpoczyna się od wywołania funkcji *Generate()* albo z parametrem ziarna, albo IRandomSource. W wypadku użycia ziarna, będzie ono przetransformowane do formy źródła entropii, będącego instancją klasy SplitMix64Random. Niezależnie od wywołanej metody, reszta generacji wygląda tak samo, Wykorzystywana jest metoda *NextUInt64*, która występuje w każdej klasie implementującej IRandomSource i zwraca wartość ulong, czyli 64 bitową nieujemną liczbę całkowitą. W przypadku prezentowanym na rysunku 3.3 używany jest SplitMix64Random, który tworzy tę wartość poprzez implementację istniejącego algorytmu SplitMix64 [16]. Następnie otrzymana wartość całkowita jest konwertowana na liczbę zmiennoprzecinkową, w celu zapewnienia wartości w przedziale od 0 do 1 oraz precyzji 53 bitowej. Ostatnie kroki to skalowanie liczby do przedziałów ustalonych w danej instancji generatora, między minimum, a maksimum. Końcowa wartość jest zwracana jako int.

Generacja liczby zmiennoprzecinkowej w wykorzystaniu tego samego źródła entropii jest bardzo podobna do generacji liczb całkowitych. Do przeprowadzenia poprawnego procesu należy sprecyzować tryb generacji wybrany spośród dostępnych. Umożliwia to wymuszenie określonego rozkładu prawdopodobieństwa tak, aby generator mógł zachowywać się w sposób zgodny z narzuconą konwencją, co może być pomocne kiedy deweloper oczekiwałby, że zwracane wartości miałyby określoną tendencję.



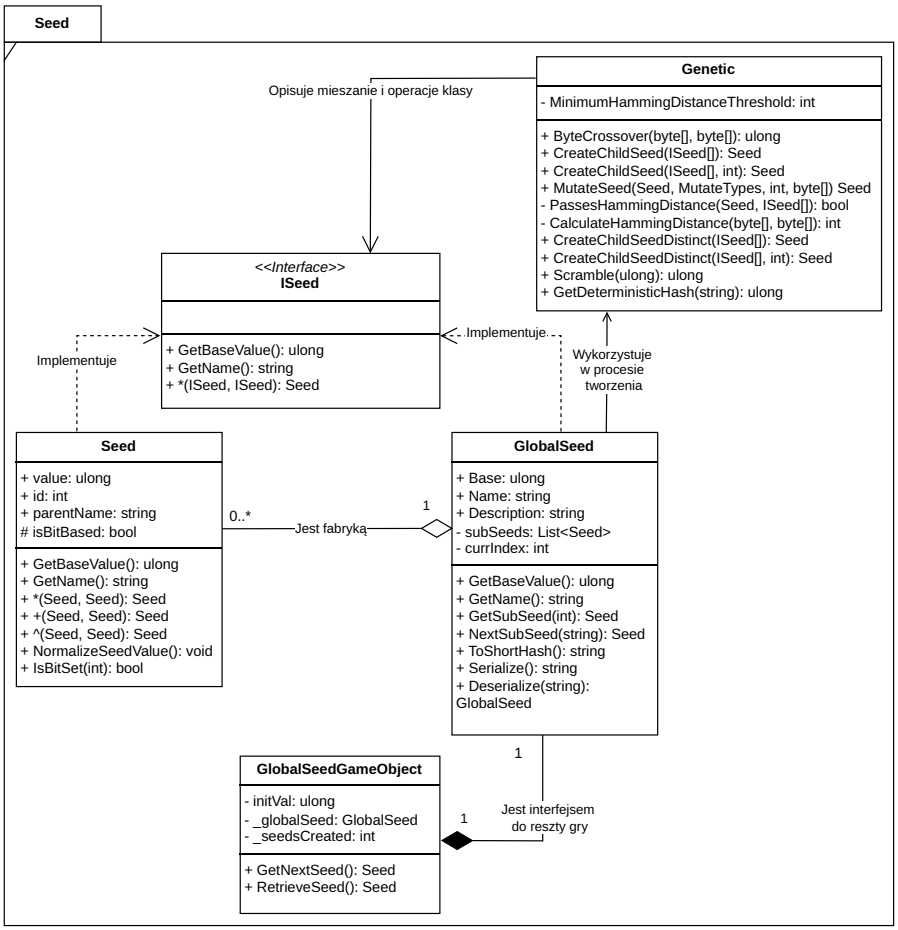
Rysunek 3.4: Algorytm generacji liczby zmiennoprzecinkowej implementowanej w klasie FloatGenerator. Prezentujący etap wyboru metody generacji liczby.

Na rysunku 3.4 przedstawiono schemat blokowy, ilustrujący działanie generatora w klasie FloatGenerator. Po ujednoliceniu źródła entropii podejmowana jest akcja zgodna z ustalonym trybem generacji, do których zaliczają się: Uniform (równe prawdopodobieństwo uzyskania wyniku na całym przedziale dostępnych wartości), Normal (rozkład prawdopodobieństwa uzyskania losowanych wartości jest zgodny z rozkładem normalnym), Triangular (rozkład prawdopodobieństwa jest przesunięty w stronę środka), Step (wartość jest zawężana do zbioru skwantyzowanych wartości na losowanym przedziale) oraz Curve (losowanie odbywa się zgodnie z przekazanym wcześniej klasie rozkładem prawdopodobieństwa). Dostępne domyślnie w bibliotece rozkłady są implementacją istniejących koncepcji modelowania prawdopodobieństwa, a nie były autorskimi pomysłami. Niezależnie od metody, wartość jest następnie rzutowana na wartość zmiennoprzecinkową z ustalonego przedziału.

Ziarno

Generacja ziarna jest całkowicie zawarta wewnątrz przestrzeni o tej samej nazwie (TDPG.Generators.Seed). Interfejs *ISeed* jest wspólny dla klasy ziarna (Seed), ale też dla klasy ziarna globalnego (Global Seed). Ziarno globalne jest wariantem ziarna pełniącym funkcję fabryki kolejnych ziaren, którego w danym środowisku powinna występować tylko jedna instancja. Duża część operacji pseudogenetycznych związanych z ziarnami znajduje się w statycznej klasie *"Genetic"*. Ziarno globalne jest implementowane wewnątrz obiektu *"GlobalSeedGameObject"*, aby ułatwić interakcje z innymi obiektami wewnątrz sceny utworzonej w silniku Unity.

Ziarna są wykorzystywane w dalszych systemach w bibliotece do "wstrzyknięcia" lub inicjalizacji pseudolosowości. Ich działanie bazuje na tym przedstawionym w literaturze (rodział 2.4). Autorzy rozszerzyli koncept o operacje inspirowane propagacją genów, mające na celu zwiększenie losowości i zabezpieczenie przed powtarzaniem się sekwencji tworzonych wartości. W przypadku tworzenia ziarna na bazie już istniejących (potomek listy rodziców) wykonywane są operacje logiczne na bitach, dodatkowo dla ujednolicenia funkcji haszującej między systemami operacyjnymi oraz wersjami .Net, przygotowana została własna funkcja "GetDeterministicHash".

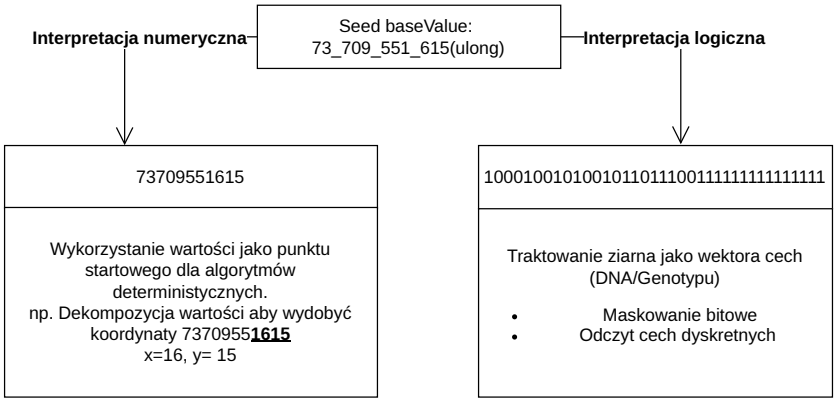


Rysunek 3.5: Diagram UML TDPG.Generators.Seed. ISeed jest centralnym obiektem, z którego dziedziczą Seed oraz GlobalSeed. GlobalSeed tworzy oraz przechowuje instancje ziaren zwykłych oraz jest implementowany wewnątrz gry poprzez obiekt GlobalSeedGameObject. Istnieje też statyczna klasa Genetic, w której znajdują się operacje arytmetyczne i pseudogenetyczne na obiektach ISeed.

Najważniejszym składnikiem ziarna jest jego wartość, składowana pod atrybutem "Value". Jest to źródło entropii, które ma dwie możliwe formy interpretacji wewnątrz biblioteki.

Zależnie od potrzeby generacji proceduralnej, ziarno może być traktowane jako postać numeryczna, bądź strumień bitów (rysunek 3.6). Flaga "isBitBased" jest wewnętrznie ustawiana dla każdej instancji ziarna i ogranicza funkcję normalizacji. Normalizacja ma na celu zapewnienie wystarczająco dużej wartości numerycznej, więc przekształciłaby wartości bitowe.

Celem flagi zatem jest kontrola użytej metody interpretacji ziarna, aby nie popsuć zawartej w nim entropii w przypadku ewentualnego ponownego użycia.



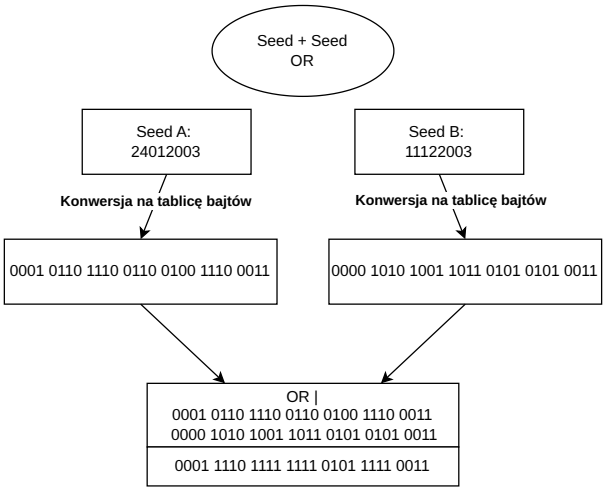
Rysunek 3.6: Dwie możliwe interpretacje ziarna, numeryczna lub binarna. Wartość numeryczna służąca w przykładzie do generacji pozycji na mapie oraz ciąg binarny i jego możliwe zastosowania.

Przykład różnych interpretacji tego samego ziarna jest zilustrowany na rysunku 3.6. Przykładowe ziarno = 73709551615 może zostać użyte do uzyskania wartości numerycznej lub boolowskiej. Stałe wymuszenie ziarna do tych dwóch interpretacji wynika z implementacji autorów.

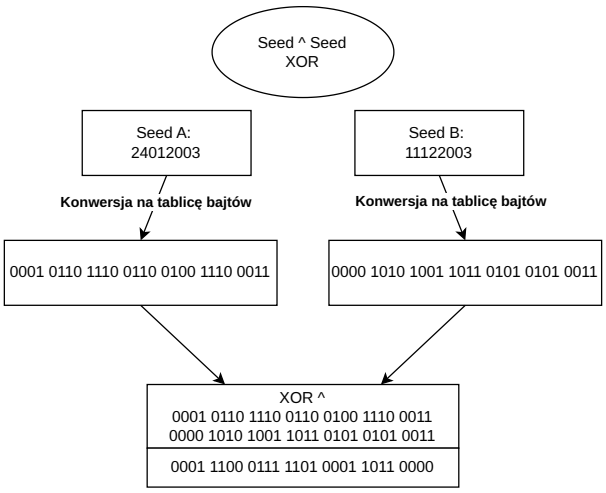
Biblioteka zawiera metody mieszania oraz dziedziczenia cech na podstawie ziaren. Wynika to z obserwacji istniejących metod implementacji ziaren w innych systemach (jak przedstawiono w literaturze 2.4), gdzie ziarna są wartościami czysto pseudolosowymi. Autorzy wyszli z pomysłem rozszerzenia konceptu ziaren o metody inspirowane propagacją genów, aby zapewnić pewne celowe podobieństwa i powtarzalne elementy między ziarnami.

Takie rozszerzenie może zostać wykorzystane jako analogiczne do DNA. Czytając bity w ziarnie, można rozczytać utworzony na jego podstawie obiekt. Dzięki pseudogenetyce można natomiast utworzyć ziarno, które stworzy obiekt podobny, ale nie identyczny do rodzica.

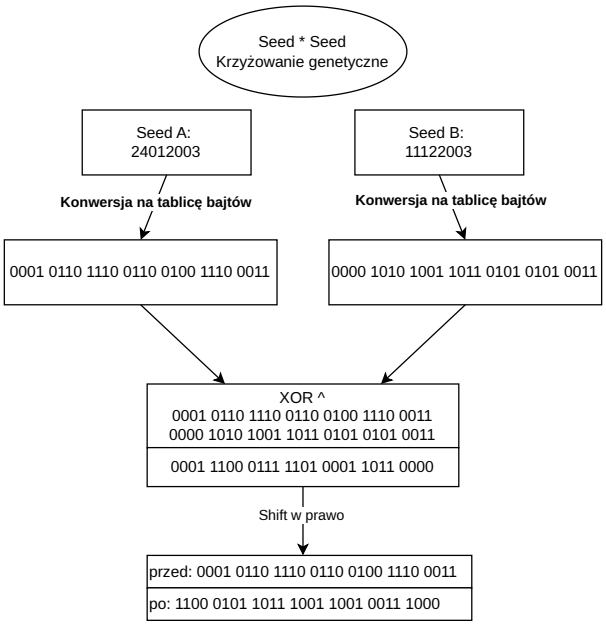
Klasa ziarna wewnątrz biblioteki TDPG ma zapewnione operacje pseudogenetyczne, mające na celu tworzenie ziaren na bazie innych (potomek). Aby to zapewnić, ziarna można sumować ze sobą (rysunek 3.7), mnożyć (rysunek 3.9) lub dokonać na nich operacji XOR (rysunek 3.8). Zapewnione są też mutacje (rysunek 3.10) mające na celu zabezpieczyć populację przed stagnacją bądź dążeniem do ziaren wypełnionych (czyli posiadających wszystkie bity ustawione na 1, co oznacza wartość zmiennej typu ulong równą 18,446,744,073,709,551,615).



Rysunek 3.7: Sumowanie dwóch ziaren. Do operacji przyjmowane są dwa lub więcej ziaren. Przykład prezentuje operację sumy dwóch. Z ziaren rodzicielskich pobierane są wartości *Value*, na których wykonana jest operacja OR. Wynikowy ciąg bitów jest entropią nowego ziarna (potomka). Zawiera wszystkie cechy z obu rodziców.



Rysunek 3.8: Alternatywa rozłączna dwóch ziaren. Operacja tworzy potomka, który dziedziczy tylko geny (bity) występujące u jednego z rodziców, ale nie u obu.

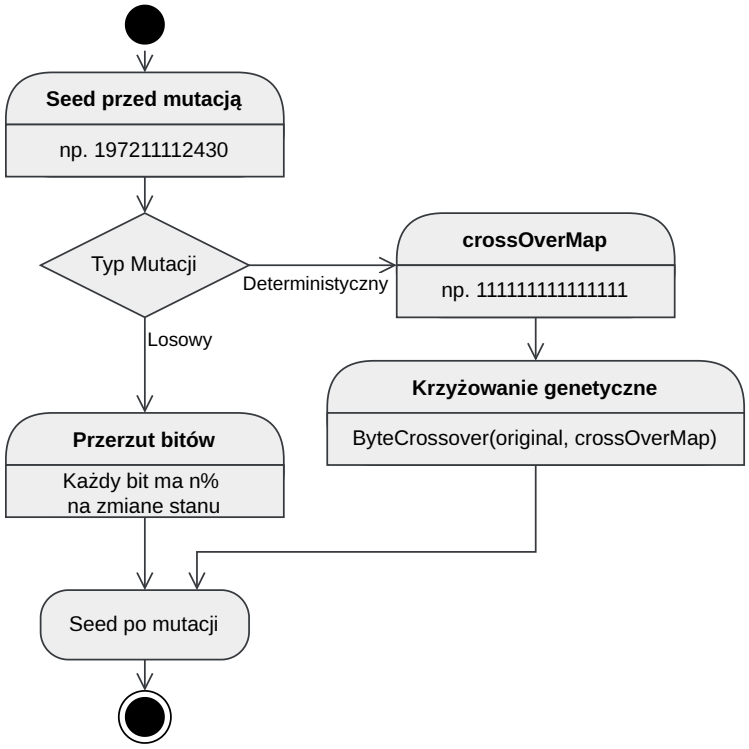


Rysunek 3.9: Mnożenie dwóch ziaren. Wewnętrznie operacja polega na wykonaniu najpierw alternatywy rozłącznej, następnie przesuwa otrzymane bity w prawo o 2 pozycje. Celem takiego łączenia jest wprowadzenie różnorodności w ziarnach, utworzenie genów, które nie występowały wcześniej poprzez przesunięcie 1 na inne pozycje.

Dodawanie i alternatywa rozłączna (XOR) ziaren są operacjami logicznymi na bitach, krzyżowanie genetyczne jest bardziej skomplikowanym procesem mającym na celu utworzenie potomka w większym stopniu różniącego się od przodków. Ponieważ jest to operacja o podobnej semantyce do dwóch poprzednich, została też sprowadzona do operacji arytmetycznej(*), jako uproszczenie w bibliotece.

Operacje te są implementowane w bibliotece jako arytmetyka ziaren dla uproszczenia dostępu do nich oraz budowaniu intuicji dewelopera korzystającego z tego rozwiązania.

W zamyśle autorów ważną rolę pełniło zapewnienie, że wśród istniejących wartości seedów ryzyko wystąpienia stagnacji wartości powinno być możliwie jak najbardziej minimalizowane. Aby wprowadzić zabezpieczenie przed taką sytuacją, udostępniona została funkcjonalność mutacji, wprowadzających zmiany bitów w przekazywanych wartościach ziaren. Jest to kolejna cecha wzorowana na własnościach występujących w genetyce, implementacja tej funkcjonalności znajduje się w klasie Genetic.



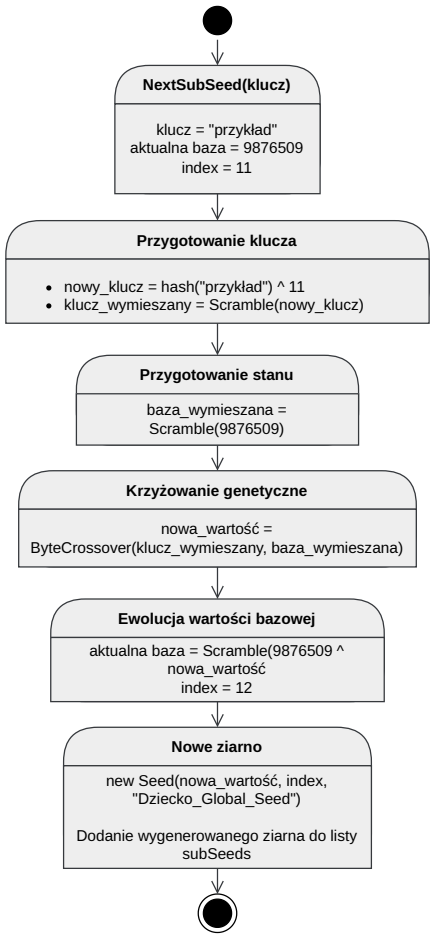
Rysunek 3.10: Algorytm mutacji wartości ziarna. Istnieją dwa tryby mutacji - losowy oraz deterministyczny. Losowy przyjmuje prawdopodobieństwo z jakim każdy z bitów może zostać odwrócony. Deterministyczny natomiast polega na operacji analogicznej do mnożenia ziaren (rysunek 3.9). Ziarno mutowane oraz zadana maska bitowa podawane są do funkcji prywatnej *crossOverMap*, która przechowuje wewnętrzną logikę mnożenia. Ostatecznie zwracane jest nowe ziarno, mające nową entropię, ale reszta jego parametrów jest skopiowana z ziarna przed mutacją

Mutacja może przebiegać na dwa różne sposoby. Deterministyczna użyje podanej maski bitowej do wykonania operacji krzyżowania wartości entropii. Natomiast przy mutacji losowej odchodzimy od pseudolosowości generowanej przez TDPG, System.Random przyjmuje procentową szansę na odwrócenie każdego z bitów w wartości ziarna. TDPG nie ma żadnego niedeterministycznego generatora, który byłby potrzebny do takiej operacji. Ziarno po mutacji zachowuje to samo imię oraz id, zmieniana jest tylko wartość entropii.

Generacja ziaren powinna być dokonana wewnątrz ziarna globalnego, które pełni rolę fabryki. Ma to zapewnić centralne źródło generacji ziaren, aby można się odwoływać do ziaren wygenerowanych wcześniej. Ziarno globalne posiada wartość bazową, która jest podstawą do generowania następnych entropii. Stosowana jest metoda gdzie po każdej generacji, wartość bazowa ewoluuje. Jest to inspirowane już istniejącymi rozwiązaniami do fabryk wartości losowych.

Dodatkowo, przy wywołaniu generacji podawany jest klucz służący do posolenia (dołączenia do oryginalnej wartości pewnego ciągu bitów, celem zwiększenia unikalności tej iteracji zapytania), co jest inspirowane metodami z kryptografii, mając na celu dodanie wartości od użytkownika do procesu generacji. Takie rozwiązanie powoduje, że system proceduralnej generacji jest częściowo zasilany wartościami spoza systemu.

Determinizm fabryki zależy od początkowej bazy oraz ciągu wywołań generacji i ich kluczy. W porównaniu z determinizmem bezstanowym zapewnia to, że nawet jeśli generacje dzielą klucz, to zwrócona entropia będzie inna. Ziarno po generacji jest dodawane do wewnętrznej listy w fabryce, na pozycji odpowiadającej jego indeksowi, w celu odzyskania entropii, gdyż następne podejście z tym samym kluczem zwróci inną wartość ze względu na nowy stan bazowy.



Rysunek 3.11: Wygenerowanie ziarna za pomocą fabryki(GlobalSeed). Nowe ziarno tworzone jest poprzez metodę *NextSubSeed()*.

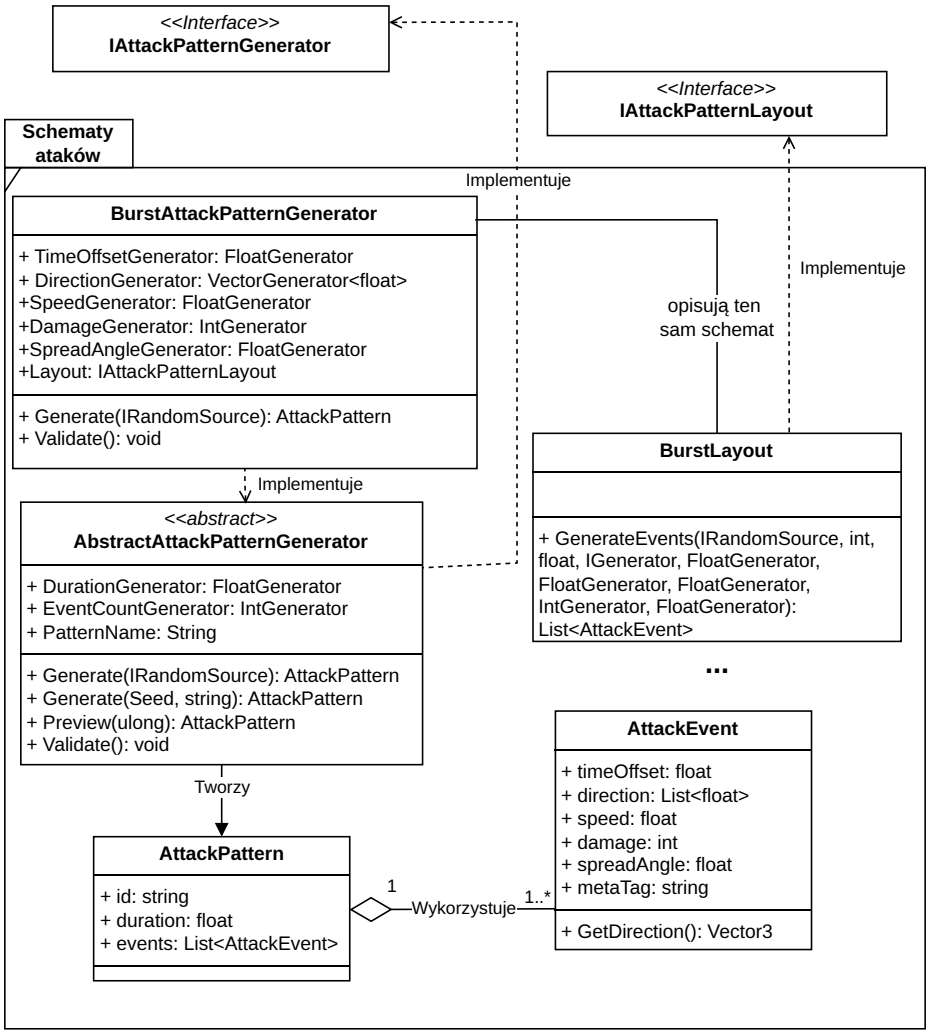
Wedle standardu przyjętego przez autorów, jeżeli ziarno powstało poza fabryką lub jest tymczasowe, to przyjmuje indeks -1.

Generacja nowego ziarna jest wykonana poprzez *NextSubSeed()*, w której argumentem jest klucz. Fabryka ma swój aktualny stan wartości Base, tutaj równy 9876509. Pierwszym krokiem generacji jest przygotowanie ciągu bitów na bazie klucza, poprzez haszowanie oraz mieszanie jego wartości. Następnie mieszana jest wartość Base, celem przygotowania drugiego ciągu bitów. Mając te dwa ciągi, tworzony jest nowy ciąg, metodą ByteCrossover, tak jak przy mutacji (rysunek 3.10) oraz mnożeniu ziaren (rysunek 3.9). Po utworzeniu nowego ciągu bitów, dochodzi do inkrementacji indeksu celem przygotowania do kolejnej generacji oraz ewolucji wartości bazowej wewnątrz fabryki. To drugie jest osiągalne przez mieszanie aktualnej bazy z nowo otrzymanym ciągiem. Na koniec tworzone jest nowe ziarno, zawierające wygenerowany ciąg bitów oraz indeks otrzymany z fabryki. To ziarno jest dodane do listy utrzymywanej w ziarnie globalnym i zwracane jako wyjście metody

Schematy ataków

W przestrzeni generatorów znajduje się również generacja schematów ataków, czyli posiadających określoną strukturę instrukcji wskazujących na formę, w jakiej po stronie programu mają być przeprowadzone ataki, ich podzielona na poszczególne kroki choreografia. W grze daje to możliwość wprowadzenia niezdefiniowanych manualnie zachowań. Celem tego systemu jest zapewnienie nieprzewidywalnych wzorców strzelania, które charakteryzować mogą poszczególne jednostki. Różnorodne schematy strzelania są częstym elementem w grach typu tower defense, więc autorzy chcieli zapewnić ich funkcjonalność w bibliotece, pomysł nie jest autorski jednakże nie bazuje dokładnie na żadnym istniejącym rozwiązaniu.

Przestrzeń ta składa się z serii klas implementujących dany wzór ataków. Wzorzec (AttackPattern) określa i wykorzystuje listę obiektów klasy AttackEvent, które są atomowymi częściami składowymi schematu, opisującymi pojedynczy atak. Do poprawnego działania potrzebne są dwie klasy: jedna implementująca interfejs IAttackPatternLayout, służąca do wygenerowania (przy wykorzystaniu zestawu składowych generatorów) listy obiektów AttackEvent, czyli samego wzorca, a także druga, implementująca interfejs IAttackPatternGenerator, która przechowuje potrzebne składowe generatory, a także przekazuje je obiektowi wcześniejszej klasy i przechwytuje powstały w wyniku jej działania AttackPattern.



Rysunek 3.12: Diagram UML TDPG.Generators.AttackPatterns, pokazujący implementację schematu "burst". Generator oraz Layout implementują interfejsy przedstawione na rysunku 3.1. Skutkiem generatora jest AttackPattern składający się z AttackEvents.

Na diagramie 3.12 dla uproszczenia została ukazana jedynie sieć powiązań dotycząca schematu ataku "burst". Przestrzeń zawiera też inne rodzaje schematów, o podobnej strukturze do "burst", ale o innej funkcjonalności. Na całość schematu składają się dwa pliki: *BurstAttackPatternGenerator* oraz *BurstLayout*. Oba implementują korespondujące z nimi interfejsy (rysunek 3.1), przy czym *BurstAttackPatternGenerator* osiąga to poprzez wykorzystanie klasy abstrakcyjnej *AbstractAttackPatternGenerator*. Oba generatory służą wypełnieniu wszystkich wartości, koniecznych do uzyskania kompletnego schematu ataku, jaki jest zawarty w klasie *AttackPattern*. W rezultacie otrzymuje się listę atomowych akcji, będących obiektami *AttackEvent*, których całość tworzy schemat.

3.3.2 Moduł generacji środowiska gry

Obiekty związane z planszą gry są jednym z najbardziej rozwiniętych systemów biblioteki, a generacja map jest klasycznym przypadkiem generacji proceduralnej. W tej przestrzeni zawarte są: generator map (podsekcja Generator Mapy), algorytm szukania ścieżek (podsekcja Pathfinding (Szukanie ścieżki)) oraz klasy związane z mapą po stronie biblioteki (podsekcja Siatka/Wewnętrzna mapa). Zostało to przygotowane w ten sposób aby zapewnić deweloperom rozbudowany system wykonujący już przygotowane funkcjonalności. Sekcja dotycząca generacji mapy jest podprzestrzenią szablonu siatki. Kluczowym elementem jest "GridManager" (Zarządca siatki), który jest punktem między resztą gry, a logiczną siatką biblioteki.

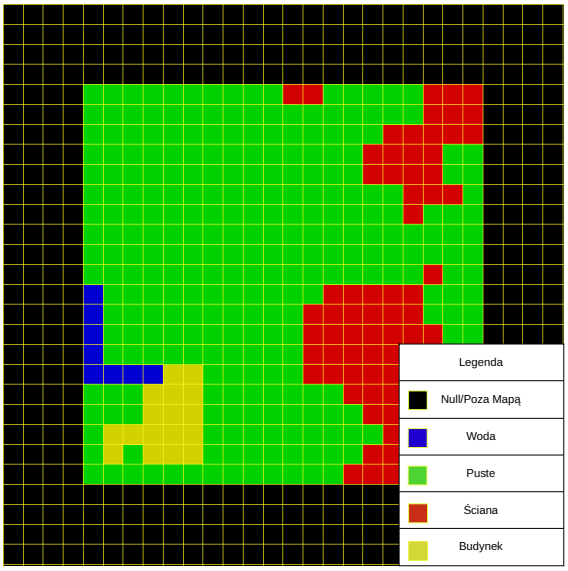
Siatka/Wewnętrzna mapa

Ponieważ silnik Unity sam w sobie oferuje obiekt siatki, dla ominięcia nieporozumień używane będą określenia TDPG.Grid dla siatki projektu oraz UnityEngine.Grid dla komponentu siatki silnika.

Osobne rozwiązanie w bibliotece zostało stworzone ze względu na potrzeby dalszych systemów, natomiast TDPG.Grid został napisany tak, aby uwzględniać współpracę z siatką UnityEngine.

Siatka wewnątrz biblioteki składa się z macierzy pól. Każde pole może być jednym z 5 rodzajów (typ wyliczeniowy TileType).

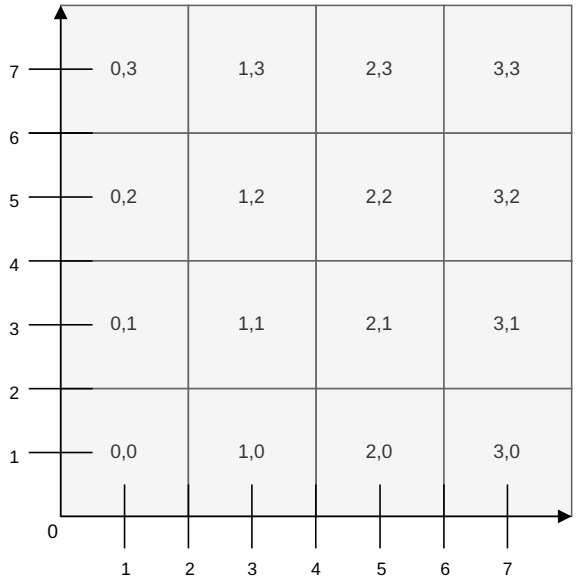
- Null - oznaczające pole poza grywalną strefą mapy lub błąd.
- Puste - pole, przez które można przechodzić oraz na którym można stawiać budynki.
- Ściana - przeciwieństwo pola pustego, pole stale wykluczone z gry, np. góry.
- Woda - specjalny rodzaj pola, pewni przeciwnicy mogą się poruszać po tym polu. Budowa jest zawsze niemożliwa lub ograniczona.
- Budynek - pole oznacza, że jest zajęte już istniejącą strukturą.



Rysunek 3.13: Przykładowa mapa logiczna, reprezentująca macierz TileType z legendą.

Każde pole na siatce jest tego samego rozmiaru i przyjmuje kształt kwadratu. Mapa sama w sobie nie musi być kwadratem, może mieć różną wysokość od szerokości. Rozmiar pojedynczego pola jest określony w parametrze "cellSize" i może być dowolną liczbą całkowitą.

Istotna jest różnica między pozycją na siatce, a pozycją na scenie gry. Siatka, tak jak wspomniano wcześniej, może być uproszczona do macierzy liczb całkowitych, natomiast "świat" (rozmieszczenie obiektów na scenie) składa się z koordynatów liczb zmiennoprzecinkowych, a więc pozycje na siatce i w świecie nie są równoznaczne.



Rysunek 3.14: Reprezentacja umieszczenia siatki w świecie przy szerokości pola=2.

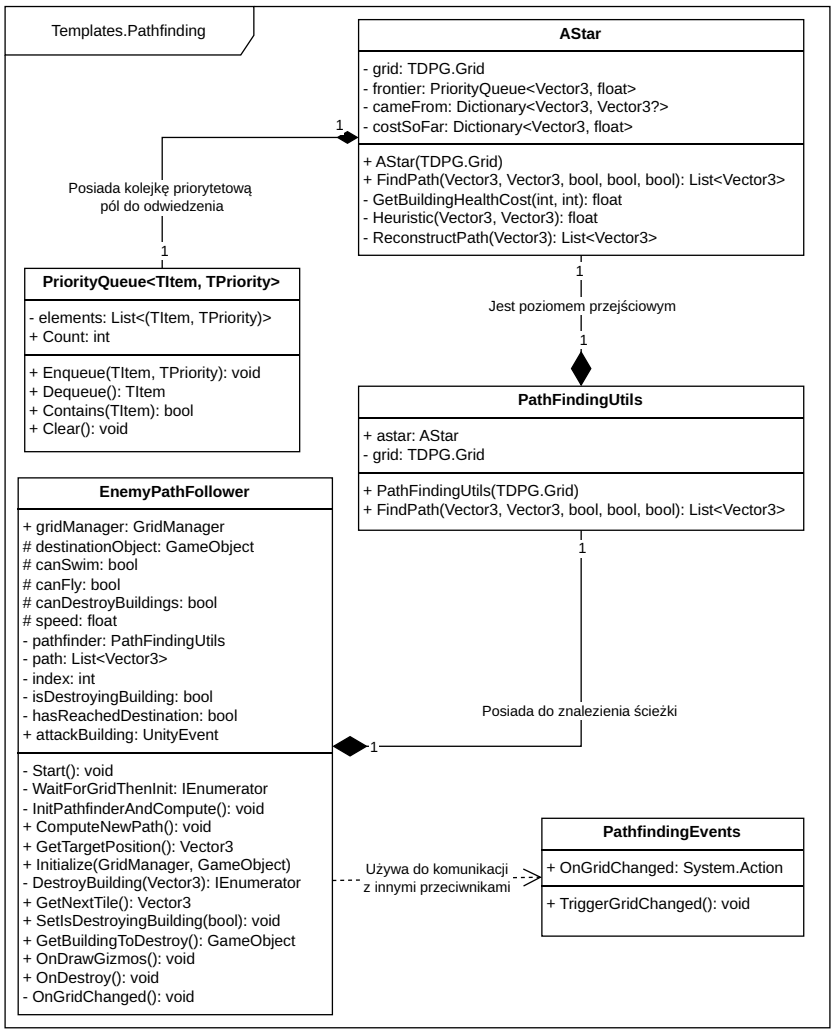
Znajdując się na pozycji (2,0) na siatce, obiekt może być między koordynatami (4,0) a (6,2) na scenie. Siatka jest uproszczoną i znormalizowaną formą rzeczywistego świata, tak jak w przypadku pionka leżącego na szachownicy, A5 jest jego pozycją w grze, lecz niekoniecznie znajduje się idealnie pośrodku zamalowanego pola. Pozycję na siatce można dzięki temu przetrzymywać w strukturze wektora dwuwymiarowego liczb całkowitych (Vector2Int), natomiast do realnych pozycji wymagane są liczby zmiennoprzecinkowe (Vector2). Tłumaczenie i utrzymywanie zgody między dwoma rodzajami pozycji jest bardzo ważnym elementem działania na siatce, w bibliotece jest to zapewniane poprzez zarządce mapy, który zawiera funkcję tłumaczące w obie strony, z siatki na świat oraz na odwrót.

Pathfinding (Szukanie ścieżki)

Biblioteka posiada funkcjonalność odpowiadającą za wyszukanie ścieżki i kierowanie się nią. Implementacja bazuje na algorytmie A* (wybór uzasadniony w rozdziale 2.5).

W bibliotece znajduje się także implementacja kolejki priorytetów, potrzebnej do wybranego algorytmu szukania ścieżek. Właściwa implementacja algorytmu znajduje się w klasie o jej nazwie. Przeciwnik będzie implementował podążanie na ścieżce poprzez klasę "EnemyPathFollower", która odpowiada za referencję do innych obiektów oraz wywoływanie szukania ścieżki. W tej klasie także stawiane są flagi określające możliwości transportu przeciwnika. O zmianach na siatce przeciwnicy informują się wzajemnie poprzez zdarzenie dedykowane im do tego.

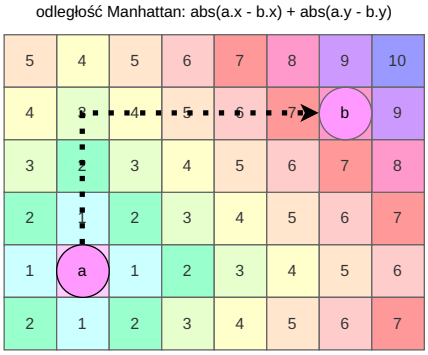
Przygotowane rozwiązanie z biblioteki należy dodać do obiektu przeciwnika implementowanego w grze, przykładowe działające rozwiązanie jest zaprezentowane w rozdziale 4.



Rysunek 3.15: Diagram UML TDPG.Templates.Pathfinding. W przestrzeni znajduje się implementacja wybranego algorytmu A*, który wewnętrznie posiada kolejkę priorytetów PriorityQueue. Klasa AStar zawiera samą logikę szukania i odtwarzania ścieżki, natomiast do przekazania niej informacji o świecie używana jest klasa PathFindingUtils. Aby przeciwnik mógł podążać za ścieżką, musi on użyć EnemyPathFollower, które zawiera wspomniane klasy oraz komunikację między przeciwnikami poprzez zdarzenie PathFindingEvents.

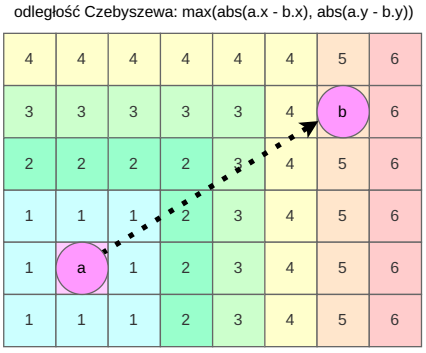
W gatunku, którym jest obrona wież, kluczowe jest, aby przeciwnicy zbliżali się do celu. Są na to dwie metody: wyznaczenie statycznej ścieżki (ewentualnie ścieżek) lub pozwolenie, aby każdy przeciwnik miał własną ścieżkę. W TDPG zaimplementowano algorytm A*, jak wspomniano w tabeli 2.2. Ten algorytm jest na tyle szybki, że przy sensownej liczbie przeciwników na mapie można pozwolić, aby każdy z nich miał własną ścieżkę.

Poruszanie się przeciwników zostało ograniczone do czterech głównych kierunków (góra, dół, lewo, prawo). Najbardziej odpowiednią w tym przypadku miarą odległości jest miara odległości Manhattan. Inną ważną miarą odległości jest odległość Czebyszewa, która jest wykorzystywana w algorytmach opisanych w późniejszych etapach.



Rysunek 3.16: Wizualizacja odległości Manhattan na mapie 8 × 6, przy punkcie a na pozycji 1,1 oraz punkcie b na pozycji 6,4

Odległość między punktami a i b jest sumą różnic ich pozycji na wierszach oraz kolumnach. Na przykładowej mapie przedstawionej na rysunku 3.16, prezentowane są dystanse od punktu a w formie liczby na danym polu. Dodatkowo wizualizacja odległości między punktami opatrzona jest oznaczeniami kolorystycznymi. W odległości Manhattan, rozkład odległości przybiera kształt gwiazdy.

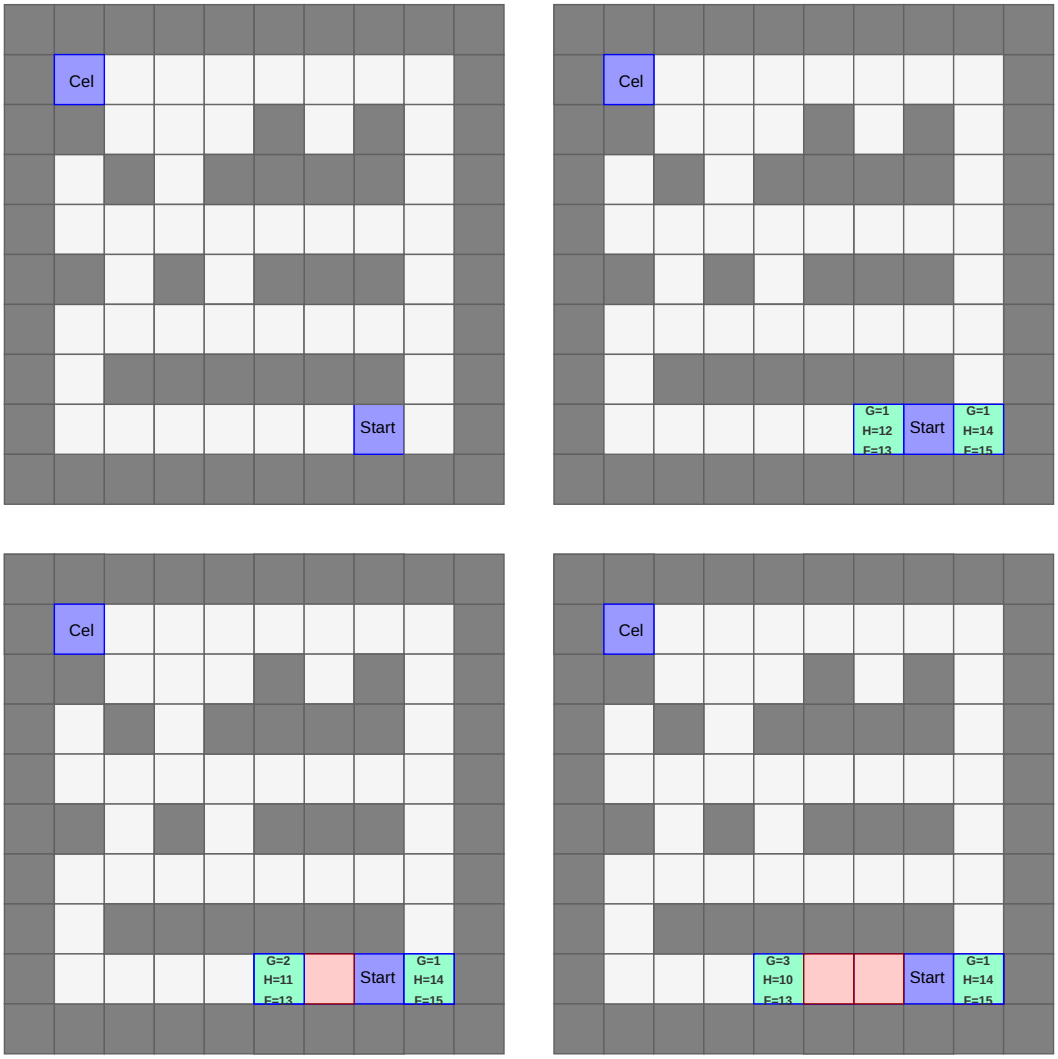


Rysunek 3.17: Wizualizacja odległości Czebyszewa na mapie 8 × 6, przy punkcie a na pozycji 1,1 oraz punkcie b na pozycji 6,4

Odległość między punktami a i b jest wartością maksymalną z odległości po kolumnach lub wierszach. Mapa na rysunku 3.17 zawiera tą samą sytuację, co rysunek 3.16, jednakże poprzez zmianę metody, odległość między punktami wynosi teraz tylko 5, a nie 8. Rozkład odległości w metodzie Czebyszewa przybiera kształt kwadratu.

Algorytm A* polega na badaniu pól o najmniejszym koszcie. Koszt (F) jest obliczany jako suma kosztu kroków już podjętych (G) oraz odległości od celu (H). W przypadku dwóch pól o tym samym koszcie należy zdecydować, które będzie zwiedzone jako pierwsze. W implementacji znajdującej się w TDPG porządkuje to chronologia, pierwsze pole dołożone do kolejki zostanie zbadane przed późniejszymi. Koszt G w poniższym przykładzie jest równy 1 dla każdego pola. Zaimplementowany algorytm szukania ścieżki można opisać jako następującą listę kroków:

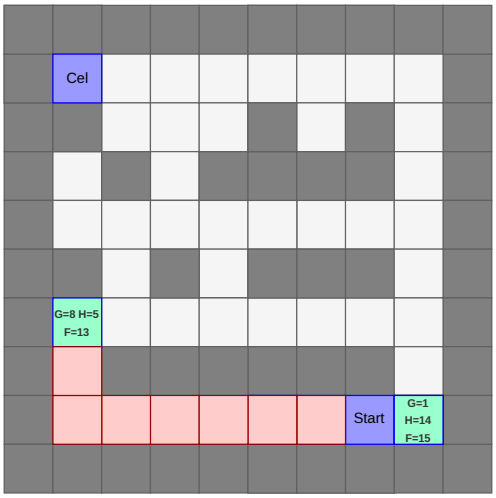
1. Inicjalizacja pustej kolejki priorytetów, dodanie do niej startu (koszt = 0, ponieważ jest to początek).
2. Wybranie z kolejki pola o najmniejszym koszcie F, zamknięcie go.
3. Jeśli aktualne pole jest celem, to przejście do punktu 5.
4. Dla każdego sąsiada aktualnego pola, którego można odwiedzić (nie ściana):
   - Obliczenie kosztów dla sąsiada (G+H).
   - Jeżeli sąsiad jest nieodwiedzony lub nowa droga do niego jest tańsza, to zapisanie aktualnego pola jako jego poprzednika i poprawka instancji sąsiada w kolejce (rysunek 3.25).
5. Powtarzanie kroków 2-4, aż algorytm nie dojdzie do celu lub kolejka jest pusta.
6. Jeśli cel został osiągnięty, odtworzenie ścieżki (cofając się po poprzednikach od celu do startu).



Rysunek 3.18: Pierwsze 4 kroki algorytmu A* (Zielony-możliwe pole, Czerwony-już odwiedzone, Niebieski-Cel/Start). Pola możliwe do trawersowania są oznaczone na biało. Przykład mapy został przygotowany do zaprezentowania pełnej funkcjonalności algorytmu. Stan początkowy (0) znajduje się w lewym górnym rogu, następnie prawy górny(1), lewy dolny(2), lewy prawy(3).

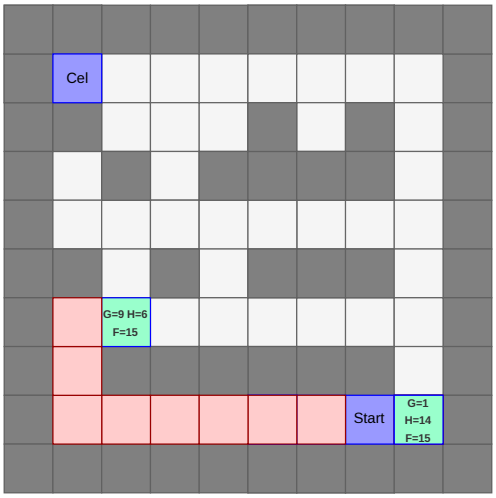
Aby wytłumaczyć działanie algorytmu A* została przygotowana mapa widoczna na rysunkach 3.18 - 3.27. Mapa zawiera punkt Start, od którego zaczyna się szukanie oraz punkt Cel, do którego szukana jest ścieżka. Poza tymi punktami na mapie znajdują się przeszkody oznaczone kolorem szarym.

Zaczynając od punktu startu w dolnym wierszu mapy, wszystkie pola sąsiednie zostają dodane do listy możliwych do zwiedzenia, każdy z nich jest jeden krok od startu. Pole na lewo od startu ma odległość do celu *H=12*, natomiast ten po prawej *H=14*. Algorytm będzie odwiedzać pola idące w lewo ponieważ ich koszt jest mniejszy. W następnym kroku do wyboru są: pole po prawej, chronologicznie starsze, ale o koszcie wynoszącym cały czas 15 lub nowe pole po lewej *G=2, H=11, F=13*. Pole po lewej ma mniejszy dystans do celu, ale dokonał jednego więcej kroku od startu w porównaniu do jego pola poprzedzającego je. Ponieważ wartość H spada o 1, natomiast wartość G rośnie o 1, to zmiany się zerują i jest to wciąż lepszy wybór.



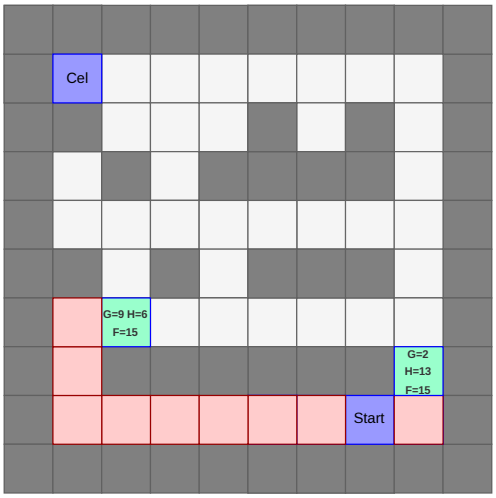
Rysunek 3.19: Stan pracy algorytmu po 5 następnych krokach. Zakończenie ciągu pól o koszcie równym 13 wychodzących na lewo ze startu.

Algorytm będzie kontynuować przeszukiwanie mapy w korytarzu wychodzącym na lewo od startu, ponieważ nie napotyka żadnej przeszkody, dystans do celu jest z każdym polem mniejszy a koszt przebytej ścieżki rośnie o 1 (ze względu na przyjęty koszt), więc co krok wartość F pozostaje równa 13. Rysunek 3.19 przedstawia ostatni krok, w którym ta ścieżka wynosi F=13.



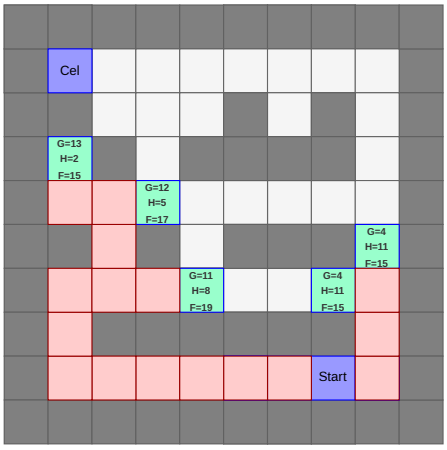
Rysunek 3.20: Krok 8: Lewa ścieżka ma teraz koszt równy 15

W następnym kroku (przedstawionym na rysunku 3.20) lewa ścieżka przestaje mieć mniejszy koszt, ponieważ musi zrobić krok w prawo, zwiększając swoją odległość do celu. Porównując odwiedzone pola na rysunkach 3.19 oraz 3.20, należy zwrócić uwagę na wartości G oraz H. Wcześniejszy stan algorytmu ma pole znajdujące się w kolejce o wartościach G=8 oraz H=5, natomiast następny krok ma G=9 ze względu na przejście po kolejnym polu, ale H=6, ponieważ krok w prawo, wymuszony brakiem innych możliwych, powoduje oddalenie się od celu. Nastąpić musi teraz decyzja pomiędzy odwiedzeniem dwóch pól o skrajnie różnych parametrach G i H (przedstawionych na rysunku 3.20), ale o tej samej sumie F. Implementacja kolejki wybiera pierwszą do niej wstawioną, czyli w wypadku tego przykładu ścieżkę na prawo od startu.

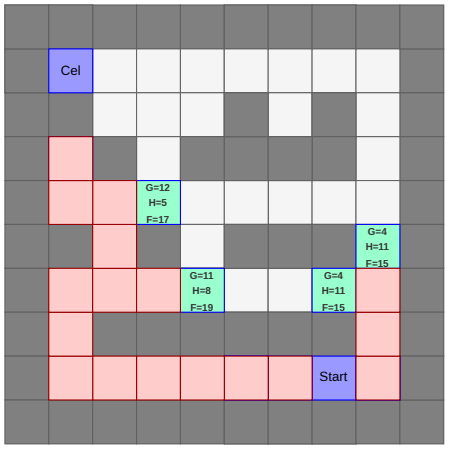


Rysunek 3.21: Następny krok w algorytmie. Ze względu na pierwszeństwo chronologii, odwiedzane jest teraz pole na prawo od startu (wiersz 3 od spodu, kolumna 2 na prawo).

Konsekwencją następnego kroku jest dalsze odwiedzanie pól na prawo od startu (rysunek 3.21). W następujących krokach będą na zmianę odwiedzane pola z jednej i drugiej ścieżki, ponieważ w obu z nich znajdą się możliwe pola o wartości F = 15. Pokazuje to działanie algorytmu w sytuacji, gdy nie ma jawnie dostępnej optymalnej ścieżki w danym momencie. Wówczas rozważane są wszystkie, które mogą prowadzić do lepszych wyników. Na skutek następnych kroków zostają też dodane do kolejki pola o wysokich wartościach F, które będą czekać w kolejce, aż nie zabraknie innych o większym priorytecie.

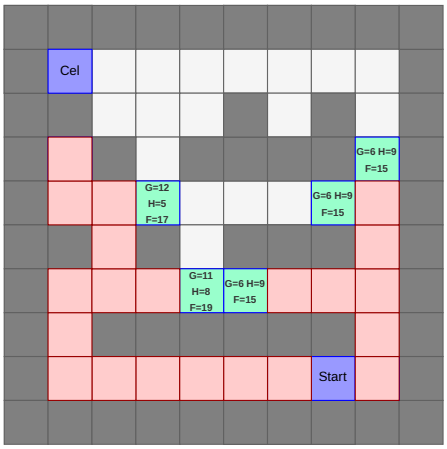


Rysunek 3.22: Krok 15, w kolejce priorytetów znajduje się wiele możliwych pól do odwiedzenia, pierwszą w kolejce jest pole o G=13, H=2, F=15 znajdujcie się w skrajnie lewej kolumnie.

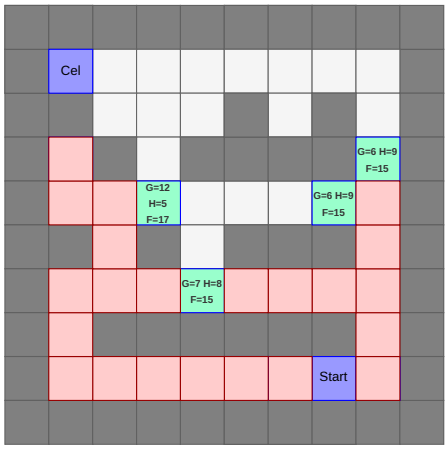


Rysunek 3.23: W następnym kroku, na skutek odwiedzenia pola w ślepej uliczce do kolejki nie jest dodawane nic, algorytm w kroku 16 jedynie zamyka badaną ścieżkę.

Następne ważne do zaprezentowania kroki znajdują się na rysunkach 3.22 oraz 3.23. Odwiedzone jest pole o sumie F=15 w lewym górnym rogu, jego jedyny sąsiad to jego poprzednik. Algorytm A* nie posiada żadnego mechanizmu wczesnego wykrycia ślepych uliczek. W przypadku wystąpienia takiej sytuacji po drodze od startu do celu, gdyby takie pole występowało w pobliżu celu, wówczas jest duża szansa na obranie tej ślepej uliczki.

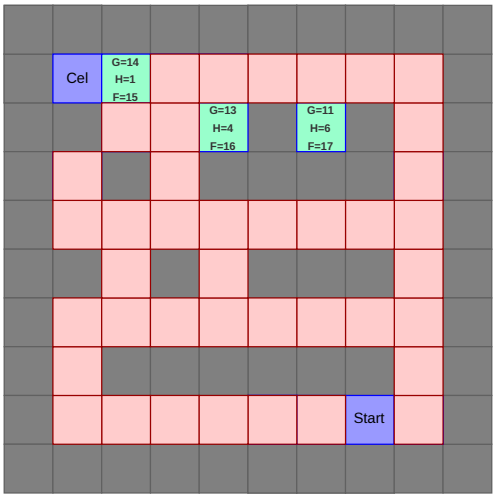


Rysunek 3.24: Krok 20: Następnym odwiedzanym polem jest to o F=15 po środku mapy (wiersz 4 od dołu, kolumna 6 od lewej).

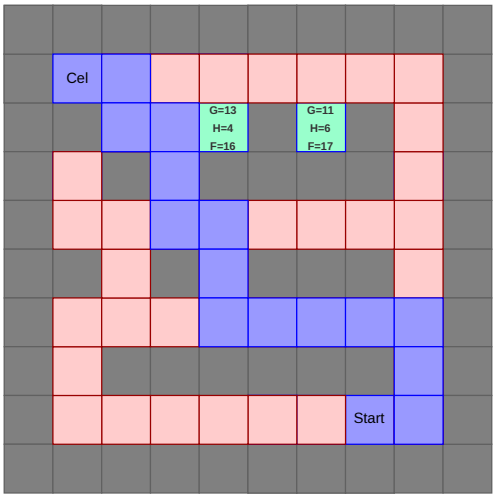


Rysunek 3.25: Na skutek poprzedniego kroku odwiedzone pole dodało do kolejki nowego sąsiada "wchodzącego" na inne, zostawiane w pamięci jest to o mniejszej sumie F, pole w wierszu 4 od spodu i kolumnie 5 od lewej nadal jest w kolejce, jednak teraz z wartościami G=7 H=8 zamiast G=11 H=8, co sprawia, że jego pozycja w kolejce priorytetów jest znacznie bliższa zeru.

W przypadku sytuacji takiej jak zaprezentowana na rysunkach 3.24 i 3.25, czyli gdy dochodzi do wejścia dwóch badanych ścieżek na to samo pole, to gorsza ścieżka jest zamykana, a lepsza zostaje w kolejce.



Rysunek 3.26: Krok 38: Przedostatni krok algorytmu. W kolejce zostały 3 możliwe pola do odwiedzenia, priorytetem jest to mające G=14, H=1, F=15 ze względu na najmniejsze F.



Rysunek 3.27: Krok 39: Znaleziona ostatecznie ścieżka, odtworzona na bazie odwiedzonych pól, aby dotrzeć do celu. Algorytm jest przerywany w tym momencie.

Algorytm kończy się odtworzeniem kroków po polach wykonanych, aby dojść do celu (rysunek 3.27). W prezentowanym przykładzie, A* zostawił dwa pola nieodwiedzone, gdyż implementowany jest wczesny stop w momencie osiągnięcia celu. Dodatkowo algorytm został rozszerzony o koszt terenu. Oznacza to, że w przeciwieństwie do przykładu powyżej, pola nie mają równego kosztu wejścia na nie. Niektóre pola mają na sobie budynki, więc powinny być trudniejsze w oczach przeciwnika poruszającego się po ścieżce. Wewnątrz środowiska testowego koszt (G) jest równy 1 dla wszystkich pól + aktualne zdrowie budynku znajdującego się na nim.

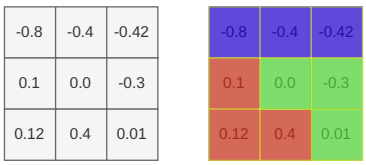
Generator Mapy

Biblioteka zapewnia generator mapy w celu zapewnienia rożnych przestrzeni i problemów dla gracza w każdej rozgrywce. Generacja map znajduje się w wielu grach implementujących PCG (Pokazane w tabeli 2.3), więc ze względu na popularność rozwiązania TDPG też ją oferuje. System może być użyty do stworzenia map o różnym rozmiarze, stopniu trudności dla gracza oraz budowie.

Algorytm generowania mapy w bibliotece TDPG jest oparty na szumie Perlina (dokładniej na poprawionej wersji, którą generuje OpenSimplex2, jak wytłumaczono w tabeli 2.3), siatce TDPG.Grid opisanej w sekcji 3.3.2 oraz implementuje szukanie ścieżek dla weryfikacji użyteczności mapy. Mapa ma określony rozmiar szerokość $\times$ wysokość, który oznacza część mapy, która będzie elementem gry i wymaga generacji. Prawdziwa mapa w grze jest lekko większa od tego rozmiaru, ponieważ zawiera też barierę grywalnej części i pola typu "Null".

Jako parametr generacji podawana jest też maksymalna liczba spawnerów na mapie, oraz minimalny dystans między obiektami. Liczba spawnerów po rzeczywistej generacji będzie wynosić minimalnie 1 (chyba, że została przerwana wcześniej), natomiast maksymalnie będzie zgodna z parametrem i możliwym rozmieszczeniem na mapie. Minimalny dystans sprawia, że spawner i cel nie będą za blisko siebie, ale też, że dwa spawnery nie są sąsiadami. Parametr ten może natomiast doprowadzić do niemożliwej do wygenerowania mapy jeśli jest za duży (np. minimalny dystans równy 100 na mapie która jest 5×5), więc jest on ograniczany w kodzie.

Generator mapy opracowany w bibliotece wymaga "GridManagera" do współpracy, generator wypełnia struktury danych, ale siatka oraz mapy pól są innymi obiektami, o których wie tylko poprzez zarządcę. W bibliotece zawarta jest też mapa statyczna (wcześniej przygotowana), która była używana przy testowaniu systemu i może się przydać przy implementowaniu systemu z biblioteki w projekcie dewelopera. Proces generacji mapy zaczyna się od wygenerowania szumu, korzystając z biblioteki FastNoiseLite. Daje nam to macierz liczb zmiennoprzecinkowych, które następnie są tłumaczone na rodzaje pól zależnie od wartości. Mapa w czasie generacji składa się z trzech poziomów: zalanego wodą, ziemi oraz wysokich gór, emulując w ten sposób mapę wysokościową. Generator ma ustawione wartości od których następuje odcięcie danego poziomu, dla wody jest to <-1,0), a dla gór (0,1>.



Rysunek 3.28: Normalizacja z wartości liczbowych na poziomy mapy. Odcięcie wody = -0.4, Odcęcie gór = 0.1 (Zielony - ziemia, Czerwony - Góry, Niebieski - Woda)

Na rysunku 3.28 przedstawione jest przejście z macierzy liczb zmiennoprzecinkowych otrzymanych z szumu, na macierz TileTypeów. W przykładzie wartości równe i mniejsze od -0.4 są tłumaczone na wodę, a te powyżej 0.1 na góry, zgodnie z założonymi wartościami odcięcia. Takie rozwiązanie nie jest autorskie. Opis podobnego systemu znajduje się między innymi w książce "Procedural Content Generation in Games" [21]. Możliwe było jednak rozbudować ten pomysł o systemy naprawy mapy podczas jej generacji (Rysunki 3.37 - 3.39).

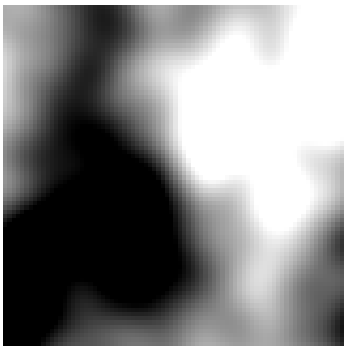
Na takiej mapie po normalizacji następuje próba znalezienia pozycji dla celu wrogów oraz spawnerów. Taka mapa jest następnie poddawana testowi użyteczności, który sprawdza, czy wystarczający procent pól jest na niej wolnych, istnieje miejsce na przynajmniej jeden spawner i czy

istnieje jakakolwiek ścieżka od spawnera do celu. Jeśli w tym momencie nie przejdzie testu, to dojdzie do ponownej próby maksymalnie jeszcze 5 razy, ale z odcięciami dającymi więcej pól wolnych. Po 6 nieudanych próbach odcięcia zostały przesunięte o 0.5 od swoich oryginalnych wartości, więc jeśli test użyteczności wciąż by nie przechodził, wówczas jest zgłaszany błąd i generacja dobiega końca. W przypadku kiedy test użyteczności przejdzie, można założyć, że generacja mapy logicznej jest skończona i przekazywana jest ona do zarządcy, aby była zaimplementowana w świecie. Zarządca dokonuje kolejnego testu, teraz bazując na pozycjach w świecie. Jeśli test poprzedni się udał, a ten nie, to można przypuszczać, że deweloper popełnił błąd w implementacji komunikacji między podsystemami. Test wykona kilka podejść, co każde manualnie zmieni pola wokół celu na puste, tworząc w ten sposób pusty krąg o coraz większym promieniu.

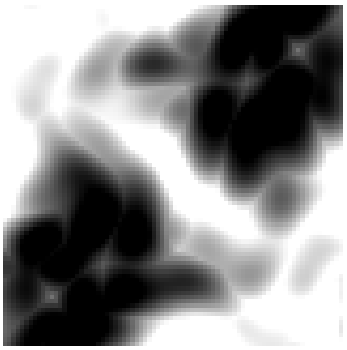
Końcowa mapa przy udanym drugim teście albo jest gotowa, albo zgłaszany jest błąd i dochodzi do próby wygenerowania mapy z innym ziarnem inicjalizującym.

Generator poza mapą statyczną ma trzy rodzaje generowanych typów map:

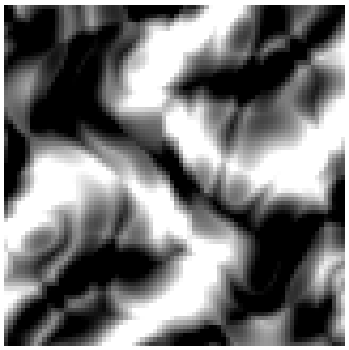
- Smooth - mapa "Gładka", szum jest bliski wartości 0, przez co mapa ma dużo pól pustych w porównaniu z innymi. Dużo szarości, pojedyncze "plamy" innych kolorów. Korzysta z rodzaju fraktali *FBm*.
- Mountainous - mapa "Górzysta", tworzy więcej wartości wysokich, przez co jest skłonna do większej ilości gór. Plamy są bardziej ustruktyryzowane. Użyty jest fraktal *Ridged*.
- Chaotic - mapa "Chaotyczna", szum jest mniej uporządkowany, więc powstają mapy mające pola o wartościach mogących znacznie odbiegać od wartości sąsiadów. Używa rodzaju fraktali *PingPong*.



Rysunek 3.29: Przykładowa mapa szumu Perlina 80x80 dla mapy rodzaju gładkiego



Rysunek 3.30: Przykładowa mapa szumu Perlina 80x80 dla mapy rodzaju górzystego

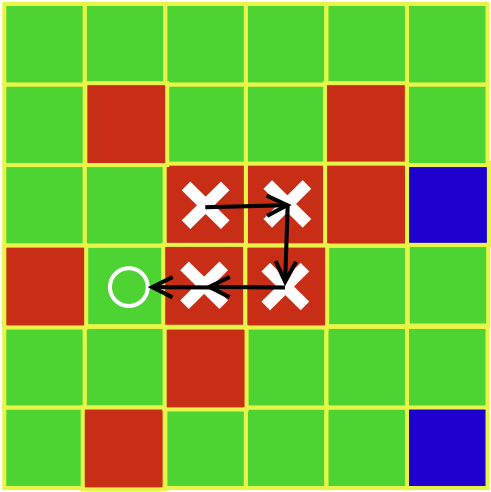


Rysunek 3.31: Przykładowa mapa szumu Perlina 80x80 dla mapy rodzaju chaotycznego

Szumy prezentowane na rysunkach 3.29 - 3.31 zostały wygenerowane poprzez demo biblioteki FastNoiseLite [26], wartości numeryczne od 0 do 1 są przypisane kolorom od czarnego do białego. Każdy z rodzajów map dodatkowo ogranicza przedział liczb, w których mogą być odcięcia poziomów. Parametry te zostały obrane poprzez test jakości map, więcej o teście zostanie opisane w rozdziale 5. Rysunki przedstawiają różnice między generowanymi typami map. Rodzaj gładki (rysunek 3.29) ma dwa duże skupiska, jedno czarne, drugie białe i szare szumy między nimi, skutkuje to mapą o zgrupowanych polach gór i wody, oraz polami wolnymi pomiędzy. Na rysunku 3.30 przedstawiony jest szum odpowiadający mapom rodzaju górzystego, w przeciwieństwie do map gładkich, te charakteryzują się zbiorem wartości niskich tworzących pewne wzory. Skutkiem tego będzie utworzenie pasm górskich na mapie. Rodzaj map chaotycznych jest pokazany na rysunku 3.31, szum układa się w kształty o większych skokach między pikselami, niż w pozostałych dwóch oraz o większej ilości pasm o skrajnych wartościach.

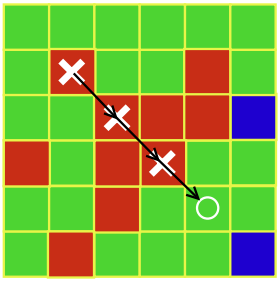
Generator mapy, po pierwszej fazie generacji spróbuje wyznaczyć kluczowe pozycję, cel i spawnery. Na start determinowana jest pozycja celu dla przeciwników, jest na to pięć możliwych opcji, wybór których następuje deterministycznie poprzez ziarno. Miejsce dla celu, musi być puste. W wypadku nie znalezienia takiego, generator zaproponuje i tak jakąś pozycję, ponieważ przy układaniu celu, jego pozycja jak i okoliczne są usilnie ustawiane jako puste.

Jeśli wartość wyrwana z ziarna wynosi 0 lub 9 to miejsce do celu będzie wyszukiwane od środka mapy, w kształcie spirali. Środek jest wyznaczany jako połowa szerokości i połowa wysokości mapy, więc w przykładowej mapie 6×6 będzie to pozycja 3×3. Następnie szukane są pozycje w spirali, zaczynając od kroku w prawo i dół (zaprezentowane w rysunku 3.32).

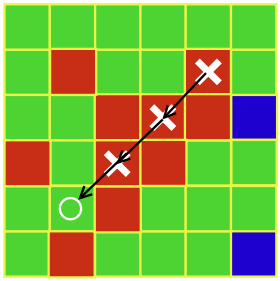


Rysunek 3.32: Przykład ustalania pozycji poprzez spiralę od środka. Początkowo badane jest pole w 3 wierszu z góry i 3 kolumnie od lewej, ponieważ są to góry to poszukiwanie trwa dalej. W następnych krokach badane są pola podążając za spiralą, aż algorytm nie dojdzie do pola w 4 wierszu od góry i 2 kolumnie od lewej, które jest dozwolone.

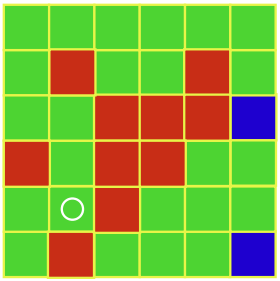
Przy pozostałych wartościach szukanie miejsca dla celu zaczyna się od rogów mapy (z odsunięciem o 1 w poprzek, aby cel nigdy nie był idealnie na polu graniczącym z barierami mapy). Dla wartości 1 oraz 2 jest to prawy górny róg (rysunek 3.33), 3 i 4 szukają od górnego lewego (rysunek 3.34). Dolny lewy róg jest sprawdzany jeśli wartość pseudolosowa jest 5 lub 6 (rysunek 3.35), co zostawia 7 i 8 do sprawdzenia rogu dolnego prawego (rysunek 3.36).



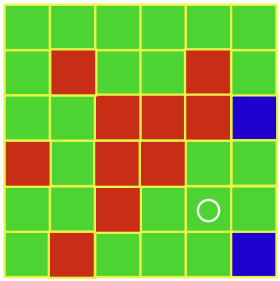
Rysunek 3.33: Przykład ustalania pozycji od lewego górnego rogu. Przeszukanie zaczyna się nie bezpośrednio na rogu zgodnie z założoną procedurą. Idąc po skosie, algorytm ostatecznie znajduje miejsce potencjalnego celu w 2 wierszu od dołu i 2 kolumnie od prawej.



Rysunek 3.34: Przykład ustalania pozycji od prawego górnego rogu. Przeszukanie jest podobne do tego na rysunku 3.33.



Rysunek 3.35: Przykład ustalania pozycji od lewego dolnego rogu. Do znalezienia potencjalnego pola następuje natychmiastowo ze względu na pole puste w pierwszym kroku.



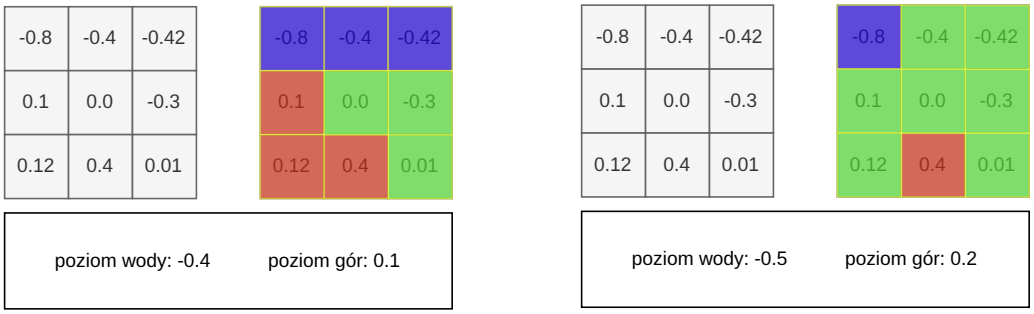
Rysunek 3.36: Przykład ustalania pozycji od prawego dolnego rogu. Pierwsze badane pole jest puste, więc jest przyjmowane jako potencjalne miejsce na cel.

Jak można zauważyć na przykładach (rysunki 3.33 - 3.36), mimo użycia innych punktów startowych, na cztery podejścia, zostały znalezione tylko 2 miejsca. Jest to kwestia małych map przy których pojedyncze pole ściany ma bardzo duży wpływ na proponowaną pozycję dla celu. Z tego właśnie powodu, generator map ma ustaloną minimalną mapę na rozmiar 20×20, aby zapobiec takiemu problemowi. Mając teraz pozycję dla celu, generator będzie szukał pozycji dla spawnerów, poprzez szukanie pól pustych, jak najdalej możliwych od celu (pokazane wraz z pełną generacją na rysunku 3.44).

Dodatkowo biblioteka jest wyposażona w mechanizmy próbujące dostosować mapę do wymagań podczas generacji. Zostały one zaimplementowane przez autorów w celu zwracania map, które są lepsze w rozgrywce. Proces oceny grywalności mapy jest opisany w rozdziale 5. Mechaniki poprawy są uruchamiane jeśli mapa nie zdaje testów (są to tzw. plany awaryjne/fallback). Ich wizualizacje znajdują się na rysunkach 3.37 - 3.39. Są to kolejno:

Fallback 1. Zwiększenie odległości parametrów odcięć od 0.

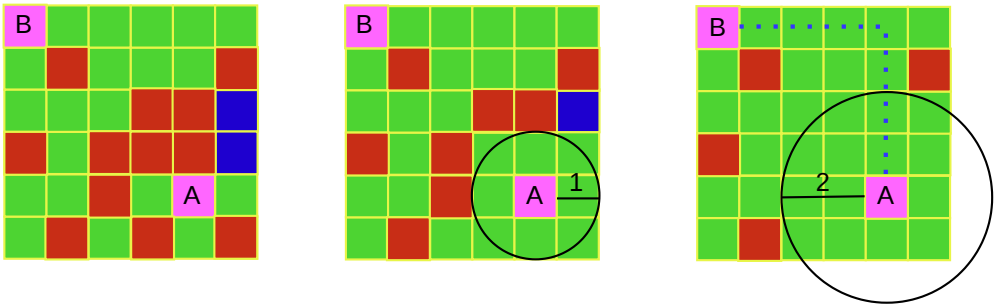
Celem pierwszego mechanizmu, prezentowanego na rysunku 3.37 jest poprawienie dużej części mapy, przy małej zmianie w parametrach generacji. Fallback 1 zmienia podane poziomy odcięć o +0.1 na górach oraz -0.1 na wodzie, od ich wartości aktualnej.



Rysunek 3.37: Działanie metody planu awaryjnego 1. Po lewej przed zastosowaniem, po prawej po zastosowaniu. Na skutek użycia tej metody poziomy odcięć są oddalone od 0, skutkując większą liczbą pól pustych na mapie.

Fallback 2. Wyczyszczanie pól dookoła celu.

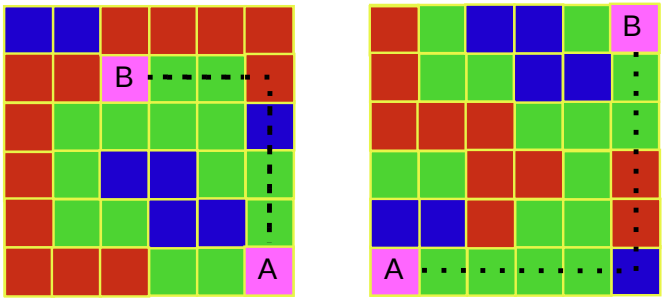
Metoda 2 jest bardziej lokalna pod względem zmian na mapie, ale bardziej wymuszająca. Działa na zasadzie "wyczyszczania" pól, czyli zmiany ich TileType na pusty niezależnie od wartości szumu. Przykład znajduje się na rysunku 3.38. Na początku nie ma dostępnej ścieżki między A i B. W pierwszej iteracji metody pola położone najbliżej celu są zamieniane na puste, lecz nadal nie ma ścieżki. W drugiej iteracji promień wynosi 2, po wyczyszczeniu mapy powstaje ścieżka. W tej metodzie zamiast odległości Manhattan używana jest odległość Czebyszewa (zaprezentowana na rysunku 3.17).



Rysunek 3.38: Działanie metody planu awaryjnego 2 na przykładowej małej mapie z dużą liczbą przeszkód między punktami A i B. Metoda planu awaryjnego 2 wykona 2 iteracje, w pierwszej wyczyści pola w odległości 1 od celu, następnie w odległości 2. Po dwóch iteracjach odnajdowana jest ścieżka.

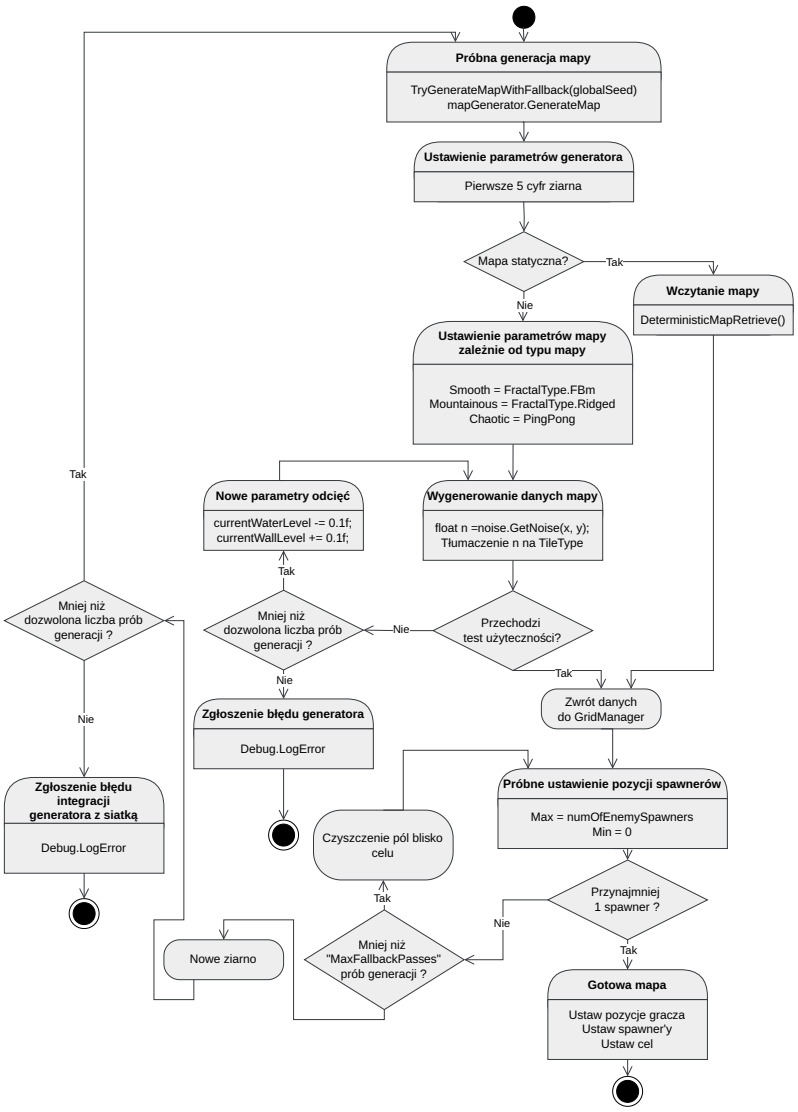
Fallback 3. Zmiana ziarna

Ta metoda powoduje największe zmiany między mapami, ma na celu weryfikację czy błąd występował na pojedynczej mapie, czy integracji generatora. Zmiana ziarna może też wyprowadzić z problematycznej sytuacji gdy generacja ma bardzo ograniczający parametr minimalnego dystansu.



Rysunek 3.39: Działanie metody planu awaryjnego 3. Po lewej znajduje się mapa przed wykonaniem fallbacku, a na prawo po. W przykładowej sytuacji parametr minimalnego dystansu wynosi 8 pól. Mapa oryginalna nie spełnia tego wymogu, maksymalnie oddalony punkt od A został zaznaczony jako B i ma odległość Manhattan równą 7. Po regenerowaniu nowa maksymalna odległość wynosi 10.

Mapy przez samą zmianę ziarna mogą wyglądać zupełnie inaczej. Na mapie lewej w rysunku 3.39 maksymalny dystans między punktami wynosi 7, a na prawej 10. Gdyby minimalny dystans był ustawiony przykładowo na 8 pól (licząc odległością Manhattan), to mapa po lewej nie miałaby żadnego miejsca na ustawienie nawet jednego spawnera. W takiej sytuacji, mimo bycia poprawną mapą pod każdym innym względem, mapa po lewej nie przechodzi testów generacji, co może zostać naprawione poprzez zmianę ziarna.



Rysunek 3.40: Algorytm generacji mapy, pokazujący przejścia między wszystkimi wytłumaczonymi elementami generacji mapy.

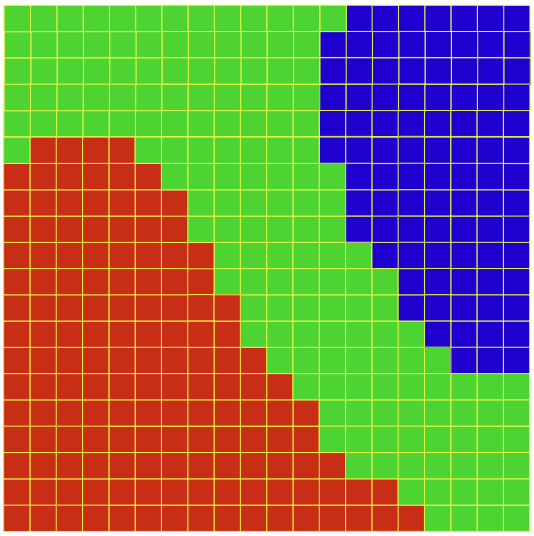
Całokształt procesu generacji mapy znajduje się na rysunku 3.40. Generacja zaczyna się od inicjalizacji parametrów poprzez ziarno. Następnie sprawdzany jest rodzaj generowanej mapy i w przypadku mapy statycznej pomijana jest główna pętla generacji. Po ustaleniu parametrów wygenerowane są dane mapy za pomocą szumu, po czym rozpoczyna się pierwszy test użyteczności. W wypadku niezaliczenia testu dojdzie do próby poprawy nr. 1 (rysunek 3.37), ten mechanizm może być powtórzony maksymalnie do 5 razy, przy szóstej mapie niezaliczającej testu zgłaszany jest błąd oraz dochodzi do końca generacji. Jeśli natomiast mapa przejdzie test, wówczas dane są zwracane do GridManagera. Następnie rozpoczyna się próba wyznaczenia ważnych punktów (rysunki 3.32 - 3.36 oraz 3.44). Kolejny test mapy sprawdza, czy istnieje przynajmniej 1 spawner przeciwników, w wypadku niezaliczenia tego testu używana jest druga poprawa mapy (rysunek 3.38). Przy braku polepszenia rezultatów dochodzi do poprawy 3. (rysunek 3.39). Jeśli nawet ostatnia z metod poprawy nie poskutkuje poprawną mapą, zwracany jest błąd i przerywa się generację. Natomiast w wypadku poprawnej mapy przygotowane są już na niej pozycje celowe oraz jest przygotowana na scenie.

Teraz znając wszystkie części generacji mapy, można zaprezentować pełny potok przykładowej generacji. Zakładamy początkowe parametry:
rozmiar 8×8, rodzaj mapy ustawiony na "Smooth", poziom wody na -0.35, poziom gór na 0.3, liczba pól pustych dookoła celu na 2, liczbę spawnerów na 5 oraz minimalny dystans na 3. Ponieważ jest to zwykła mapa to flagę zakładającą, że każdy przeciwnik może pływać (flaga istnieje w celu poprawienia skrajnych map) oraz zostawiamy pole debugowanego ziarna jako puste.

1. Wykonana jest funkcja "Precalc". Rozmiar mapy jest ustawiany na dozwolone minimum 20×20, oraz rozszerzane jest o 5 pól z każdej strony, które służą rolę mgły stanowiącą granicy grywalnej mapy. Prawdziwy rozmiar mapy wynosi 0-30, grywalna mapa jest 5-25.

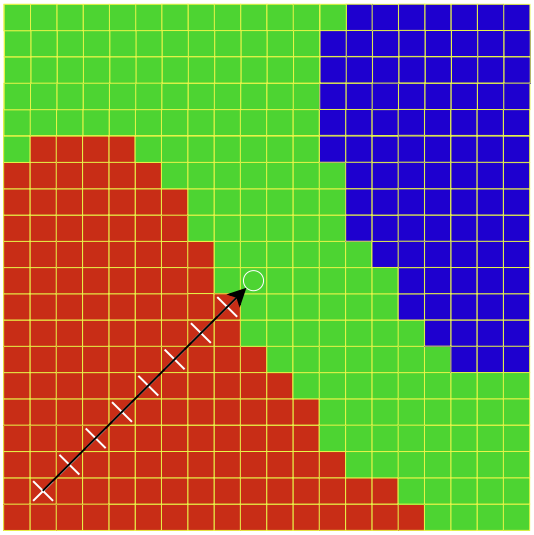
2. Początek generacji mapy po walidacji parametrów. Pobierane jest ziarno z zewnątrz generatora (lub awaryjne w kodzie generatora jeśli nie otrzymano żadnego). Następnie z ziarna wydzielane są pierwsze 6 cyfr. Przykładowo otrzymujemy ziarno 6030566406311752863, to zostaną odczytane wartości d0=6, d1=0, d2=3, d3=0, d4=5, d5=6. Na bazie zmiennych d0-d4 ustawiane są parametry generacji szumu (częstotliwość, oktawy, lakunarność, wzmocnienie, ważona siła).

3. Zaczyna się pętla generacyjna. Gdyby wybrać mapę statyczną to pętla jest pominięta. Wygenerowana jest mapa logiczna (macierz pól)



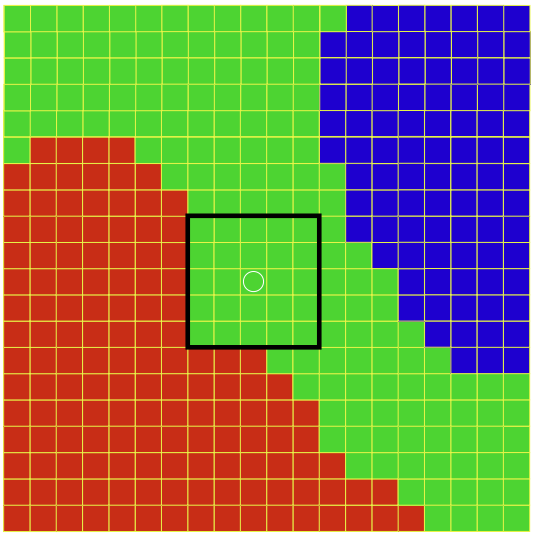
Rysunek 3.41: Pierwsza postać mapy (z pominięciem pól mgły) wygenerowanej na bazie badanego przykładu. Składa się ze zbioru pól wody w prawym rogu, serii gór po lewej stronie oraz pól pustych między nimi.

4. Po wygenerowaniu mapy, próbuje się ustalić pozycje specjalne. Wartość d5=6 jest wykorzystywana do wybrania pozycji, od której zaczęte jest poszukiwanie. Ponieważ wynosi 6, to zaczyna od lewego dolnego rogu.



Rysunek 3.42: Przeszukiwanie pola na cel analogicznie do rysunku 3.35. Ostatecznie odnaleziony możliwy cel znajduje się na środku mapy

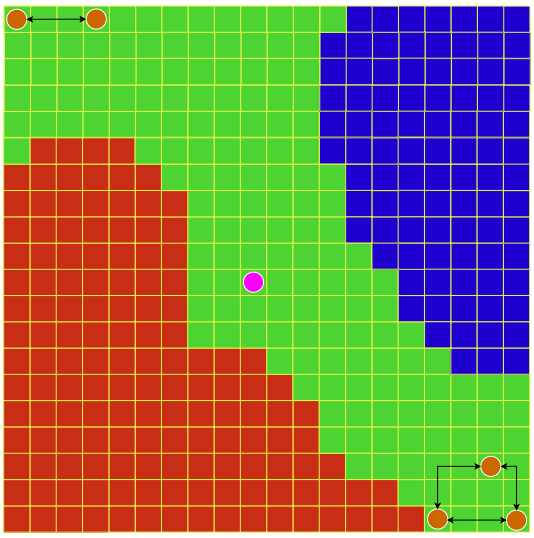
5. Po znalezieniu pozycji odpowiedniej dla celu będzie przygotowywane jego najbliższe otoczenie. Parametr "liczba pól pustych dookoła celu" jest ustawiony na 2, więc wszystkie pola w odległości Czebyszewa równej 2 są zamieniane na puste.



Rysunek 3.43: Przygotowywanie okolicy celu, analogicznie do rysunku 3.38. Parametr liczby pustych pól=2, więc wyczyszczane są wszystkie pola odległe o 2 w odległości Czebyszewa od celu.

6. Mając przygotowany cel, obliczany jest procent mapy "użytecznej", czyli pól pustych z których można dojść do celu. Ta mapa ma procent równy 42%, który jest mały, ale akceptowalny. Mapa jest teraz przesyłana do zarządcy mapy, aby ją zaimplementował.

7. Ostatnim elementem przygotowywanym poprzez generator jest ustalenie możliwych pozycji dla spawnerów. Parametry tutaj ważne to oczekiwana liczba spawnerów równa 5 oraz minimalny dystans równy 3, oznacza to, że dystans między dwoma punktami kluczowymi musi wynosić minimalnie 2 kratki.



Rysunek 3.44: Zaznaczone przygotowane pozycje dla spawnerów. Różowe koło reprezentuje umieszczany cel, a pomarańczowe spawnery. Cel został umieszony na przygotowanej dla niego pozycji, natomiast spawnery możliwie jak najdalej od niego. Dodatkowo wszystkie spawnery utrzymują odległość "minimalny dystans"=3 między sobą.

Pozycje dla przeciwników są rozmieszczone możliwie daleko od celu z zachowaniem minimalnego dystansu między sobą. Pozycje te są przesyłane do zarządcy, aby je zaimplementował. Odbywa się to w ten sposób ponieważ algorytm szukania ścieżek wykorzystuje pozycjonowanie w świecie, więc przed wyznaczeniem pozycji dla przeciwników, mapa musi już być na scenie. Ważne jest by zwrócić uwagę na to, że generator jedynie przygotowuje pozycje specjalne, rzeczywiste obiekty są na nich układane w późniejszych etapach.

3.3.3 System efektów gry (moduł stanów)

W trakcie prac nad biblioteką stało się dla autorów jasne, że podstawowe formy interakcji obiektów w grze w postaci np. ataków zadających obrażenia, których wartość byłaby wygenerowana w sposób proceduralny nie wyczerpują możliwości, jakie otwiera PCG. Z tego powodu zdecydowano się opracować system, opierający się na modyfikatorach wartości statystyk, który deweloperzy są w stanie w dowolny sposób rozszerzać. Pozwala to urozmaicić rozgrywkę na dowolny sposób, który opracuje deweloper. Modyfikatory te określa się w bibliotece mianem Efektów, natomiast ich złożenia, które działają na zasadzie nakładania wielu modyfikatorów jednocześnie, zyskały miano Element (ang. żywioł). Klasy odpowiadające za definicję tych obiektów oraz struktur wspomagających ich obsługę zawarto w bibliotece we współpracujących ze sobą modułach.

Logika

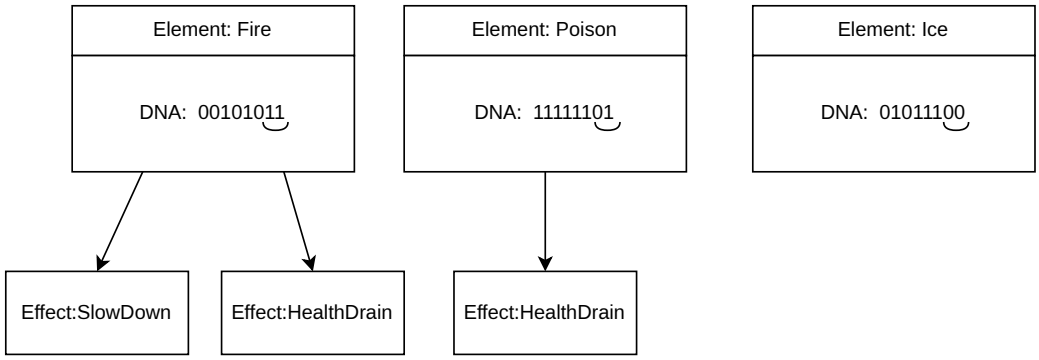
Wewnątrz TDPG.EffectSystem.ElementLogic znajdują się podstawowe elementy tego systemu. Klasa abstrakcyjna "Effect" określa strukturę i parametry danego efektu wewnątrz środowiska np. "SlowDown", mający na celu zwolnić prędkość postaci o pojedynczą wartość. Następnie "Element" jest ustrukturyzowanym zbiorem efektów o własnej nazwie i id, które docelowo mają być wywołane w ramach systemu żywiołów wewnątrz środowiska. Efekty same w sobie nie są wywoływane wewnątrz rozgrywki. Każdy Element ma przypisane DNA w formie ziarna (Seed). DNA elementu określa, jakie rodzaje efektów są w nim zawarte. Ponieważ ulong ma rozmiar 64 bitów, ogranicza to do maksymalnie 64 liczbę efektów podpiętych do dna. Każdy bit jest powiązany z jedną, niezmienną implementacją efektu, mogącą występować z różnymi parametrami (np. najstarszy bit zawsze odpowiada "SlowDown", ale Elementy między sobą mają inne instancje spowolnienia).

Rejestr

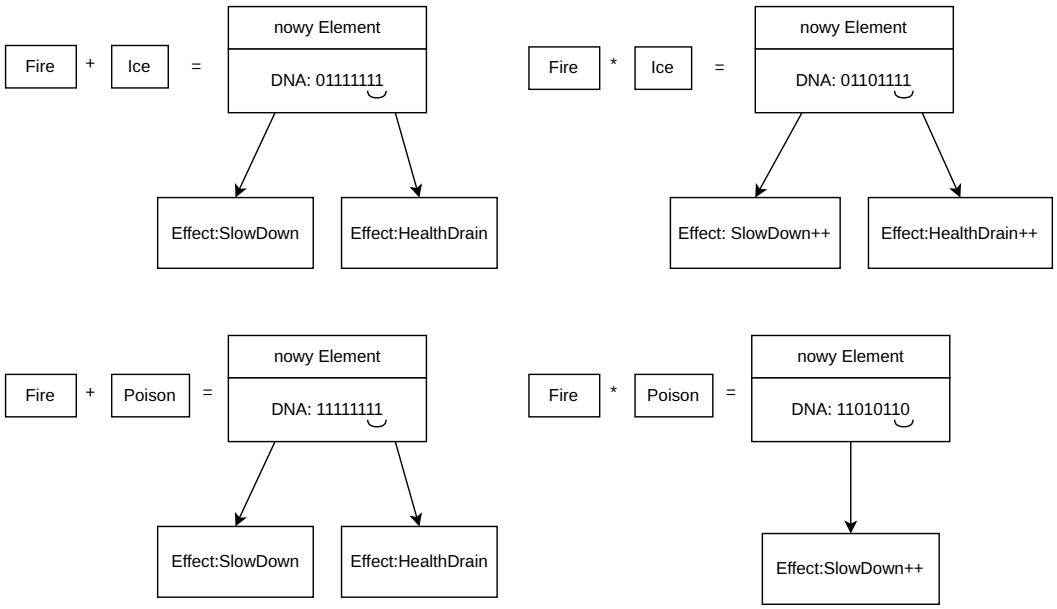
Rejestr jest strukturą elementów, której zadaniem jest utrzymanie ich w formie, z której inne systemy mają do nich łatwy dostęp. Powstał on w celu zebrania w jednym obiekcie wszystkich elementów, a także przechowywania w intuicyjnej postaci możliwych zależności między nimi. Możliwe są między elementami zależności działające na zasadzie relacji pokrewieństwa - dany element może mieć nieograniczoną liczbę "rodziców", na podstawie których powstał i po których dziedziczy on własności zapisane w swoich efektach. Takie własności skutkują wprowadzeniem do przestrzeni elementów zasad przekazywania swoich efektów na zasadzie analogicznej do propagacji genów.

Rejestr ma postać grafu skierowanego dwustronnego, dając możliwość szybkiego poruszania się po nim do szukania potomków, ale też rodziców danego wierzchołka. Do zadań rejestru należy tworzenie, dodawanie i zwracanie elementów. Rejestr znajduje się w wielu plikach ze względu na jego rozmiar, do tego użyto mechaniki "partial" języka C#. Implementacja tworzenia nowych elementów na bazie już istniejących jest zawarta wewnątrz rejestru.

Dla wytłumaczenia zakładany jest następujący przykład. W systemie istnieją tylko dwa efekty: "SlowDown" przypisany do bitu o indeksie 0 oraz "HealthDrain" przypisany do indeksu 1. W rejestrze znajdują się trzy istniejące jak dotąd elementy: "Fire" (powodujące u zaatakowanego spowolnienie i obrażenia), "Poison" (skutkujące zatruciem) oraz "Ice" (bez żadnego efektu).



Rysunek 3.45: Przykładowy stan startowy modelujący sytuację, gdzie w rejestrze znajdują się 3 istniejące elementy



Rysunek 3.46: Możliwe sposoby generacji nowego elementu, w zależności od dobranych rodziców oraz trybu generacji, z uwzględnieniem bitów DNA branych pod uwagę w procesie generacji.

Jak zilustrowano na rysunku 3.46, istnieją dwie metody na generowanie nowego elementu: poprzez dodawanie efektów lub poprzez ponowne złączenie DNA. Wewnętrznie korzystające z operacji arytmetycznych na ziarnach.

Dodawanie skutkować będzie nowym elementem dziedziczącym efekty po przodkach. Jeżeli któryś z efektów występować będzie wielokrotnie pośród przodków, wówczas dopisana do niego wartość będzie zależna od ustawionego trybu. Może dziedziczyć od pierwszego rodzica, drugiego lub utworzyć nowy efekt o uśrednionych wartościach. (Ilość ponad jakość)

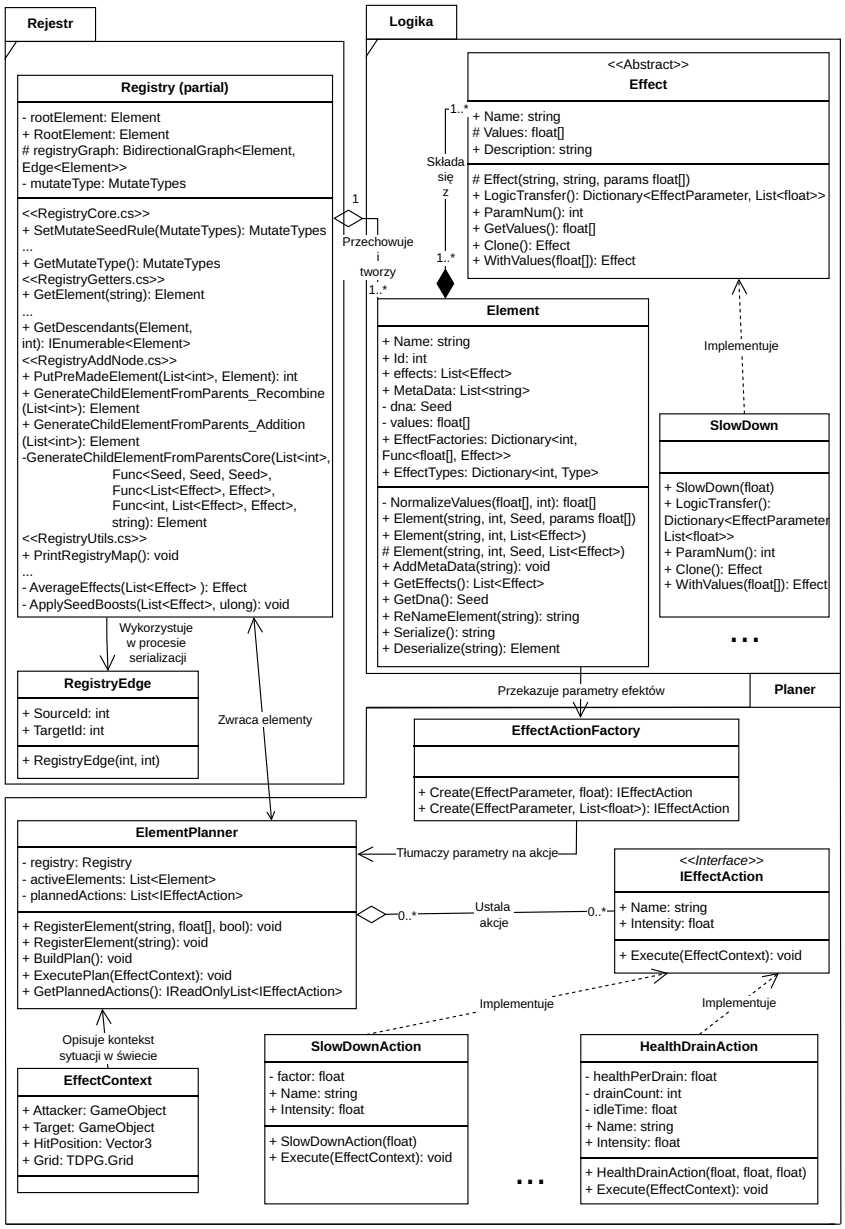
Złączenie wykonuje odpowiednik operacji XOR na DNA. W wyniku tego dziedziczona jest mniejsza lista efektów, co jest rekompensowane przez ulepszenie wartości parametrów odziedziczonych efektów. (Jakość ponad ilość)

Tak zbudowany system sprawia, że gracz zachęcany jest do korzystania z obu rodzajów kombinacji. Nie powinno dojść do sytuacji, gdy uzyskuje się DNA zawierające same bity ustawione na 1 w późnych etapach gry. Dokonując rekombinacji na dwóch elementach duża ilość genów jest wyłączana, a nawet gdyby stworzony został element zawierający wszystkie efekty, to składałby się on ze słabszych wariantów, niż element wyspecjalizowany. Sposób tworzenia nowych elementów bazuje na pseudo-genetyce, zaprezentowanej w rozdziale 3.3.1.

Planer

Wewnątrz biblioteki napisany został również moduł odpowiadający za wykonanie w poprawnej logicznie kolejności wszystkich efektów powiązanych z elementami, jakie mają do czynienia z danym wydarzeniem, określany jako *ElementPlanner*. Potrzeba uwzględnienia go wynikła z faktu, iż nakładanie wszystkich efektów ma szansę zwrócić różne rezultaty w zależności od kolejności wywołania poszczególnych efektów (za przykład może posłużyć hipotetyczna sytuacja wywołania efektów zmniejszenia HP przeciwnika o określony procent oraz pomniejszenia go o stałą wartość, wówczas w zależności od tego, który z efektów wykona się jako pierwszy, rezultat będzie inny). Zastosowanie planera rozwiązuje możliwy problem braku jednoznacznej kombinacji efektów. Przechowuje on listę elementów istotnych w danym kontekście, referencję do rejestru (aby umożliwić pobieranie elementów) oraz listę zaplanowanych akcji wynikowych, mających postać obiektów implementujących interfejs *IEffectAction*.

W momencie, kiedy powinny zostać wykonane akcje, następuje tłumaczenie parametrów efektów na korespondujące z nimi akcje (np. Effect: SlowDown na IEffectAction: SlowDownAction). Kontekst określonej akcji (czyli zestaw obiektów, między którymi następuje dana akcja, to znaczy przykładowo atakujący, atakowany oraz ich lokalizacja) przekazywany jest planerowi pod postacią instancji klasy EffectContext.



Rysunek 3.47: Diagram UML TDPG.EffectSystem. Przestrzeń ta jest podzielona na trzy podprzestrzenie, zawierające odpowiednio: logikę, która kontroluje efekty i elementy, rejestr w formie grafu służącego do przechowywania wszelkich elementów i utrzymywania sieci powiązań między nimi, a także planer, czyli strukturę pomocniczą automatyzującą wywołanie wszystkich efektów, jakie zawierał dany element, czy też zbiór elementów.

3.3.4 Moduły generacji i modyfikacji treści

W tej sekcji znajdują się opisy trzech podprzestrzeni, których zadaniem jest generowanie obiektów, które odgrywać będą rolę w warstwie audiowizualnej. Generacja audio, generacja wideo oraz tekst zostały zebrane do jednej sekcji tej pracy, ponieważ wszystkie mają ten sam cel, jakim jest zwiększenie różnorodności i charakteru danego generowanego obiektu. Z punktu widzenia gracza wszystkie trzy dają ten sam efekt: zróżnicowanie na warstwie audiowizualnej, poprzez dodanie danej instancji wygenerowanego obiektu personalizacji widocznej bez spojrzenia w wewnętrzne statystyki.

Generator tekstu oparty na łańcuchach Markowa

Generator tekstu został zaimplementowany, aby dodać unikalności nowo wygenerowanym elementom, np. elementy tworzone w sekcji 3.3.3 przybiorą nazwę, która zostanie im wygenerowana. Tekst może być wykorzystany do nadania proceduralnie wygenerowanej nazwy przeciwnikowi lub jakiemukolwiek obiektowi, który może być nazwany. Jest to ostatni z wyjątków, gdzie potrzebna była entropia niedeterministyczna, co wymagało użycia losowości systemowej zamiast tej zapewnianej przez autorów.

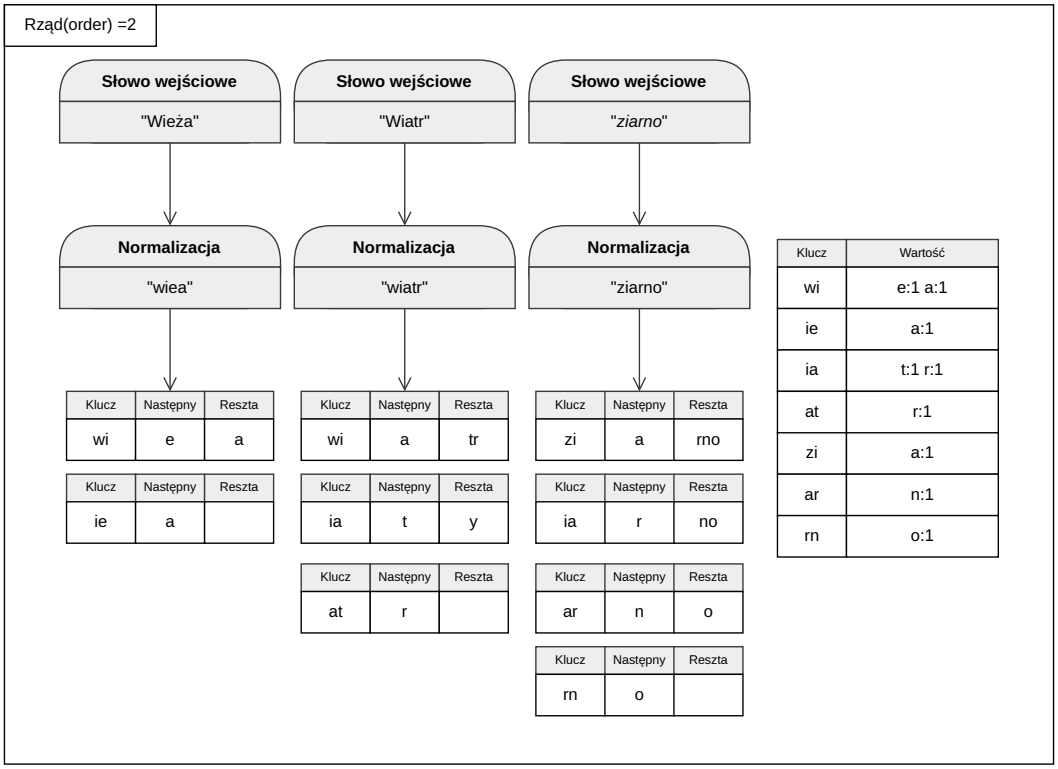
Implementacja generacji tekstu mieści się całkowicie w jednym pliku, MarkovChain. Działanie jest oparte na łańcuchach Markowa. Zaimplementowano gotowy algorytm i rozszerzono na jego bazie generator. Kluczowym pojęciem w takim generatorze jest *rząd*. Oznacza on ile liter tworzy "klucz", na bazie którego zostanie wygenerowana kolejna litera. Małe rzędy (tylko jedna litera na klucz) sprawiają, że trening generatora nie nauczy się zasad gramatycznych z danych treningowych. Duże rzędy (4-5) sprawiają za to, że zamiast tworzyć nowe wyrazy, generator będzie zwracał dane treningowe (Zweryfikowane w sekcji 5).

Najważniejsze są dwie operacje: trenowanie obiektu klasy i wykonywana przez niego generacja.

Trenowanie: Podczas trenowania podajemy generatorowi zbiór treningowy, każdy wyraz jest normalizowany do małych liter i wyklucza znaki specjalne (np. !, ń, ą, #). Po normalizacji dochodzi do głównego etapu trenowania.

Wyraz jest dzielony na części, klucz składa się ze znaków (liczba znaków jest określana przez rząd), następny znak w wyrazie jest wartością, którą generator teraz zapamiętuje jako możliwy następnik. Jako kolejny krok wyraz jest badany o jeden znak dalej, aż nie zabraknie znaków (pozostanie ich mniej niż rząd+1). Ostatecznie po treningu generator ma listę kluczy i przypisane do nich ich możliwe następniki oraz ile razy dany następnik wystąpił. Ważne w treningu jest, aby podać duży zbiór treningowy, najlepiej trzymający się tego samego języka/gramatyki. Wraz z biblioteką TDPG dostarczone są dwa zbiory danych treningowych:

- Elements_Clean.txt - Lista składa się z 2857 wyrazów wywodzących się prawie wyłącznie z łaciny (ok. 20 jest z greki. Wynika to z faktu, iż łacina nie zawiera wyrazów zaczynających się na K).
- Names_Fantasy_Clean.txt - Lista składa się z rzymskich cesarzy, imion autorów, demonów i aniołów z biblii, egipskich bogów, słowiańskich potworów oraz dynastii europejskich od lat 300 do 2025. Wyrazy pochodzące z różnych źródeł, ale trzymające się języków indoeuropejskich celem generowania wyrazów brzmiących jak słowa pochodzące z fantastyki.



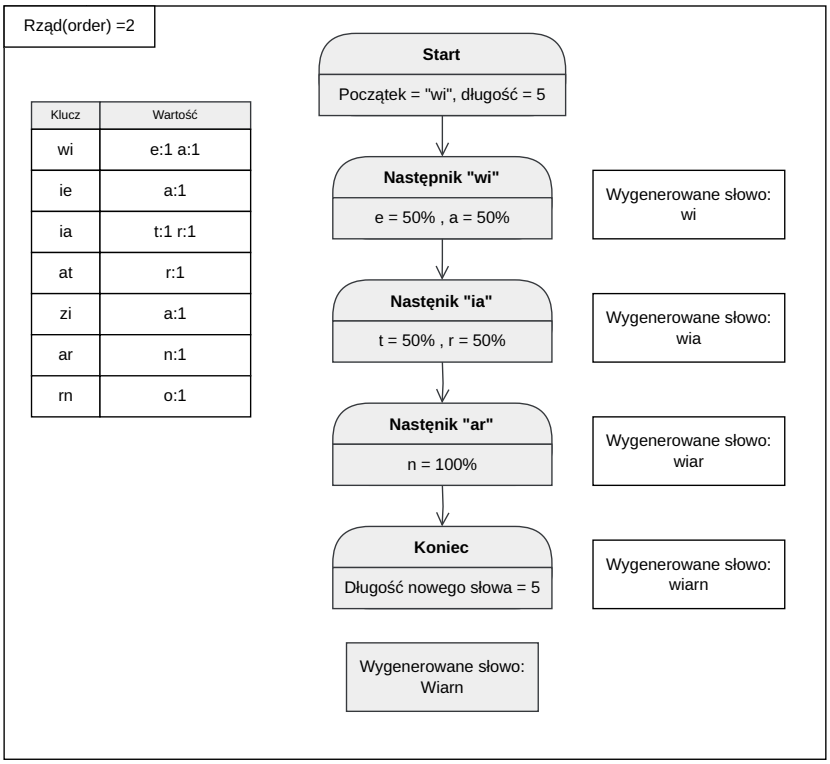
Rysunek 3.48: Przykładowe trenowanie generatora o rzędzie=2 za pomocą trzech wyrazów "Wieża", "Wiatr" oraz "ziarno".

W przykładowej generacji (rysunek 3.48) podawane są trzy wyrazy. Każdy z nich przechodzi przez normalizację, a następnie jest badany krok po kroku w celu wyznaczenia zasad gramatycznych. Z wyrazu "Wieża" po normalizacji zostaje "wiea", ponieważ usuwane jest "ż". Wyraz następnie zostaje pocięty na 2 części oraz resztę. W przypadku aktualnego wyrazu "wiea" klucz, który składa się z 2 znaków (bo rząd=2) "wi" oraz ich następnik "e". Taka kombinacja jest zapisywana w wewnętrznym słowniku generatora. Ponieważ jest to pierwsze odnalezienie takiej zasady, to przypisywana jest mu wartość 1. Takie badanie jest przeprowadzone na każdym z podanych wyrazów, dzięki czemu otrzymuje się słownik z wieloma kluczami i ich możliwymi następnikami.

Generowanie: Generator na start przyjmuje początek nowego słowa oraz jego długość (lub ziarno, za pomocą którego te parametry będą ustawione). Następnie wyszukiwany jest następnik dla aktualnie badanych znaków, wewnątrz wytrenowanego łańcucha. Następnik dla danego klucza jest wybierany pod względem jego zaobserwowanego prawdopodobieństwa w treningu. Np. dla "wi" jest e oraz a. Łącznie są dwie wartości, każda po jednym wystąpieniu, co daje szanse równe 1/2 dla nich obu. W przypadku braku następnika dla danego klucza praca generatora zostaje przerwana, chyba, że zdarza się to już na etapie inicjalizacji. W takim przypadku generator będzie szukał najbliższego podobnego wyrazu (za pomocą funkcji *FindClosestPrefix*), np. gdyby podać "la" jako początek w generatorze powyżej, użyte zostałoby "le" ze względu na wspólny początek.

Dodatkowo generator został wyposażony w metody rozszerzające działania na łańcuchach Markowa. Posiada listę zakazanych wyrazów, jeżeli taki wyraz zostałby wygenerowany, wówczas zamiast go zwrócić, dojdzie do ponownej generacji. Ma to zapobiec tworzeniu niechcianych wyrazów już na poziomie generowania. Takimi wyrazami mogłyby być wulgaryzmy, bądź nazwy objęte prawami autorskimi.

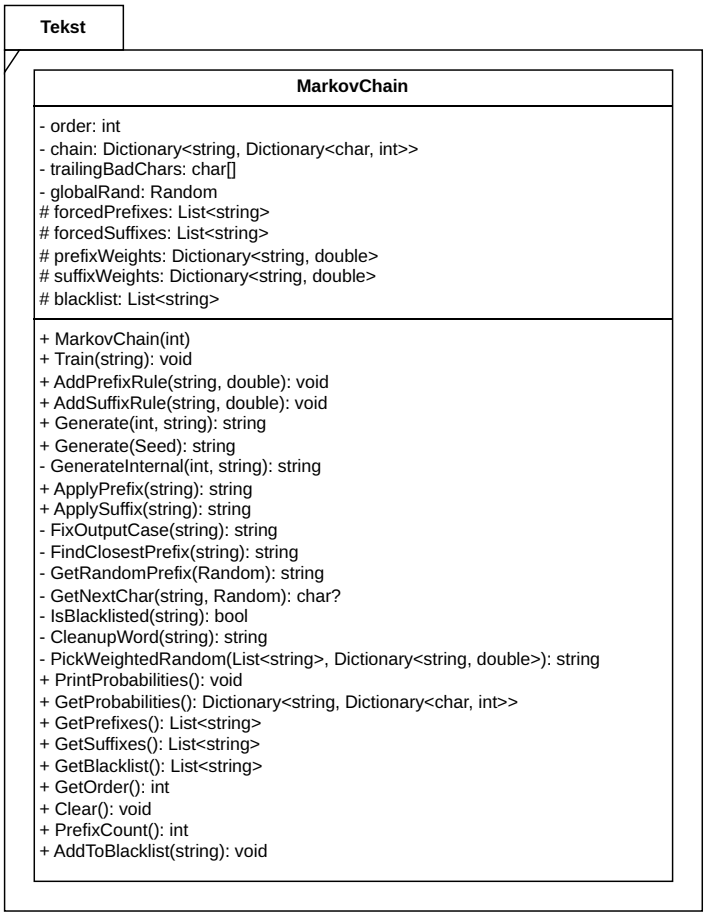
Generator posiada też listy wyrazów do konkatenacji na początek lub koniec wygenerowanego wyrazu. Dzięki temu można np. wygenerować nazwę wroga i po fakcie doklejać do niej rangę (wyraz: "Gnary" warianty: "Super-Gnary", "Zwykły-Gnary", "Gnary Wielki" itd.).



Rysunek 3.49: Przykład generacji wyrazu, generator wytrenowany z rysunku 3.48.

Przykład generacji jest zaprezentowany na rysunku 3.49. Bazując na wytrenowanym generatorze (Rysunek 3.48) oraz parametrach: początkowy wyraz = "wi" i długość wyrazu = 5, algorytm wykorzystuje klasyczne rozwiązanie łańcuchów Markowa. Odszukiwany jest następnik dla "wi", są dwie możliwości o równym prawdopodobieństwie, w przykładzie wygrywa "a". Następnie szukany jest następnik dla "ia" (ponieważ aktualnie wyraz generowany ma postać "wia", a rząd 2 sprawia, że patrzymy na ostatnie dwa znaki). Algorytm po kolei znajduje możliwe następniki aż

do utworzenia pięcioznakowego wyrazu, w tym przykładzie jest to "wiarn". Pod koniec generacji nowe słowo jest kapitalizowane i ewentualnie szukane w liście słów zabronionych.



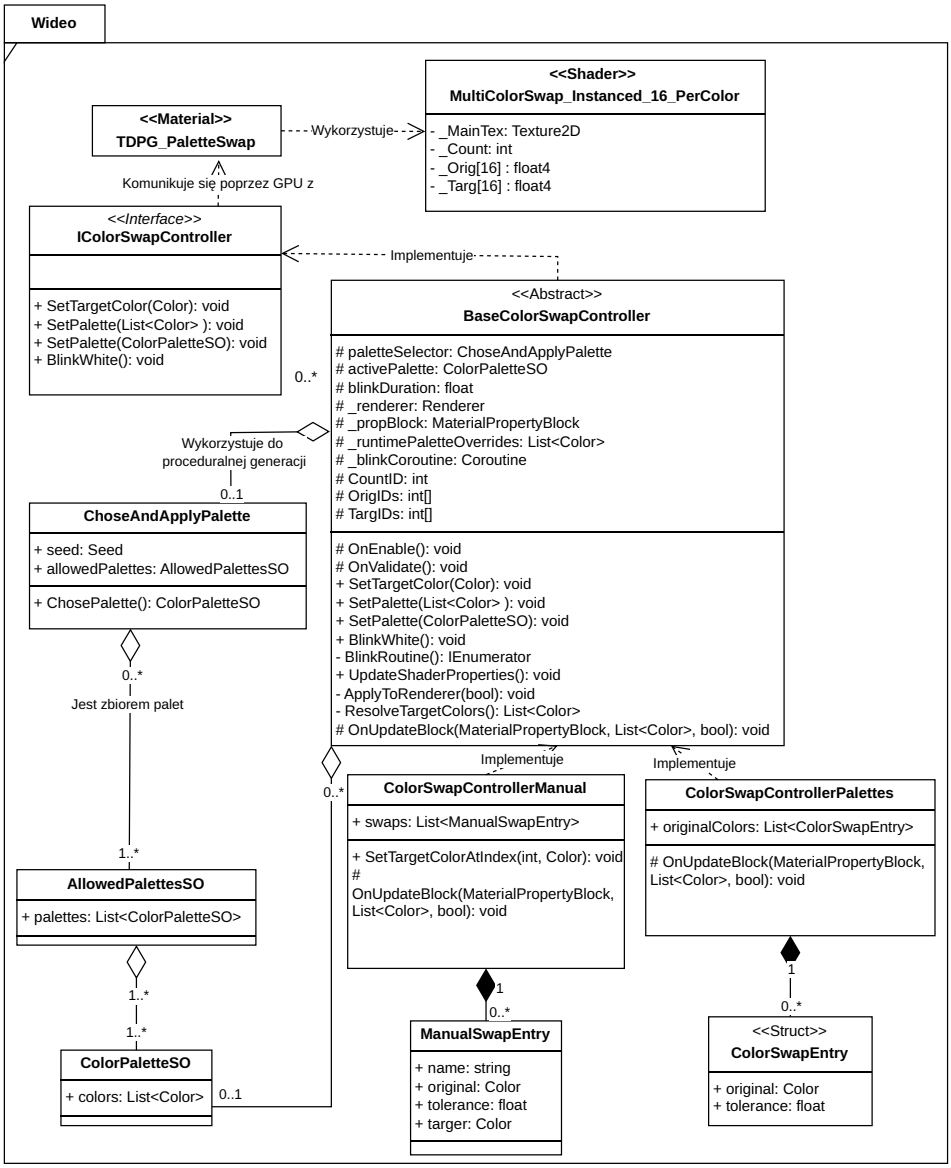
Rysunek 3.50: Diagram UML TDPG.TextGeneration, prezentujący implementację generatora tekstu. Klasa bazuje na rozwiązaniu łańcuchów Markowa i jest rozszerzona poprzez autorów.

Rysunek 3.50 jest diagramem UML pliku MarkovChain.cs. Klasa składa się z funkcji i parametrów potrzebnych do implementacji łańcuchów Markowa oraz rozszerzeń autorskich w formie listy zabronionych słów oraz list przedrostków i przyrostków.

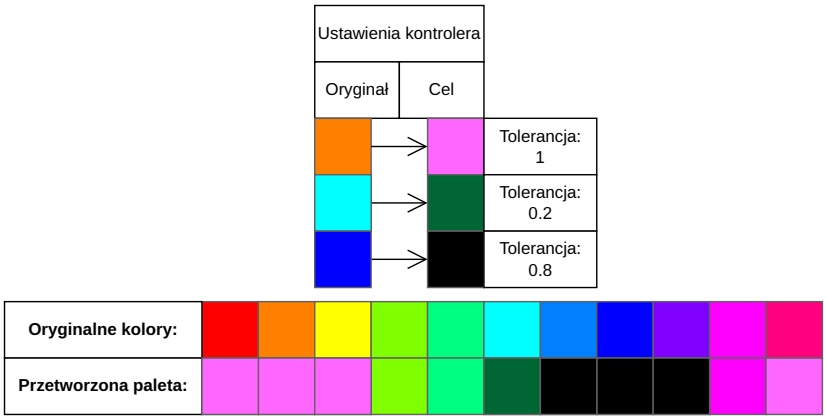
Zmiany palet barw obiektów wizualnych (Palette Swap)

Do generacji elementów wizualnych użyta jest technika zmiany palet barw (Palette Swap), używana w wielu grach (np. wspomniane w sekcji 2.3, No Man's Sky). Oznacza to, że przy przygotowanym jednym rysunku biblioteka oferuje blisko nieskończoną liczbę jego wariantów kolorystycznych (rysunek 3.55). Użyto tej mechaniki ponieważ można ją wykonać w całości używając shaderów na karcie graficznej, odciągając obliczenia odpowiadające za ten mechanizm od CPU, wykonującego resztę elementów gry. Dzięki wykorzystaniu zmienników, można wpłynąć na warstwę wizualną danego generowanego obiektu bez dużego nakładu dewelopera.

Kluczowym elementem w tym systemie jest Shader, program cieniujący, którego zadaniem jest zapewniać funkcjonalność zmiany kolorów przy pomocy karty graficznej. Shader przyjmuje listę 16 par kolorów (w każdej parze kolor źródłowy i docelowy). Ograniczenie do 16 kolorów wynika ze struktury języka HLSL oraz architektury GPU, które nie wspomagają list o nieustawionym rozmiarze, czego przyczyną jest statyczna alokacja pamięci w buforach. Limit par zmian został ustawiony na 16 kolorów, ponieważ technika palette swap z założenia powinna być wykorzystywana tylko do zmiany kluczowych barw (np. kolor skóry, odzieży, wiodący kolor). Shader przyjmuje 16 par zamienników, nie ogranicza palety do 16 kolorów. Paleta barw jest cały czas w szerokiej gamie. W przypadku potrzeby większej ilości wystarczy podzielić rysunek na części i nałożyć zmianę na każdą z nich osobno. Program do każdej pary przyjmuje też tolerancję, która wskazuje na przestrzeń kolorów zbliżonych do oryginalnego, które też mają być przemalowane. Dokonywane jest to, aby zapobiec artefaktom. Do komunikacji z shaderem przygotowany jest interfejs *IColorSwapController* oraz klasa abstrakcyjna *BaseColorSwapController*. Dzięki nim można ustawić parametry w inspektorze silnika lub aktualizować je poprzez kod. Kontrolery pozwalają też na szybką tymczasową podmianę kolorów docelowych na biały kolor.



Rysunek 3.51: Diagram UML TDPG.VideoGeneration. Obiektem centralnym jest kontroler zmian kolorów, który komunikuje się z shaderem aby przekazać wybrane parametry zmiany. Istnieją dwie implementacje kontrolera, jedna dla zmian manualnych, a druga dla zmian bazujących na paletach. Proceduralność jest zapewniona poprzez klasę pomocniczą "ChoseAndApplyPalette", która posiada listy przygotowanych palet barw oraz ziarno, za pomocą którego wybierana jest paleta dla danego obiektu.



Rysunek 3.52: Uproszczona funkcjonalność Shaderu. Wprowadzono trzy przykładowe pary, z pomarańczowego na róż o skrajnie wysokiej tolerancji, z niebieskiego na zielony o niskiej tolerancji oraz z lazurowego na czarny o rozsądnie wysokiej tolerancji. Poniżej par ze zmianami zaprezentowane jest porównanie gamy barw przed i po przemalowaniu.

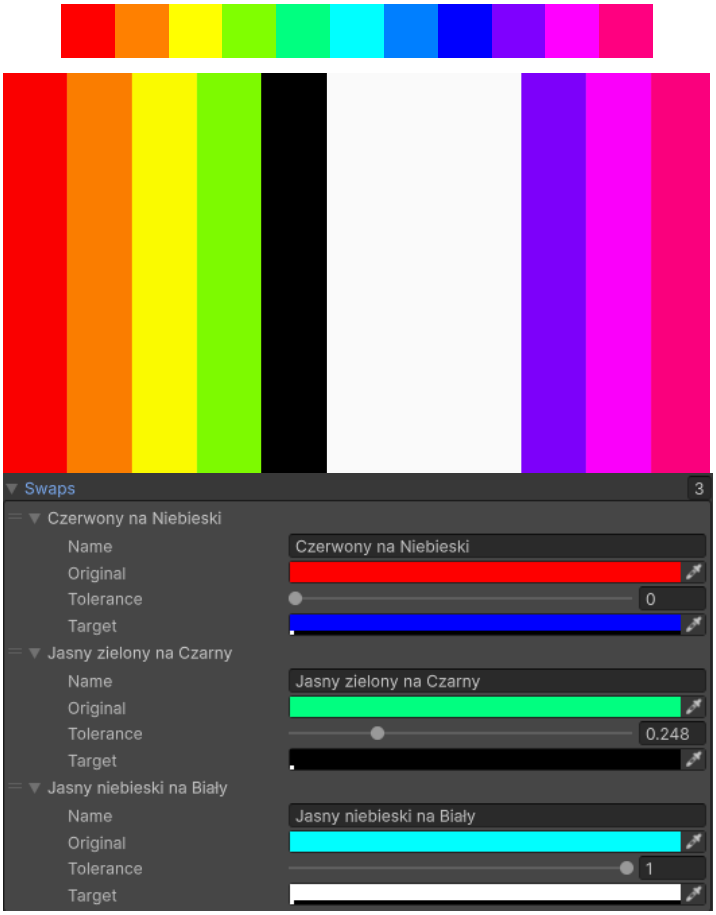
Na rysunku 3.52 pokazano przejście na uproszczonej gamie oryginalnych kolorów, w rzeczywistości kolor jest reprezentowany strukturą RGBA. Tolerancja nie oznacza kolorów o bliskich indeksach, a różnicę między kolorami. Różnica jest obliczana na wartościach RGB jako odległość między pozycją koloru w trójwymiarowej przestrzeni kolorów za pomocą odległości euklidesowej:

$$odl = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

Przykład z rysunku 3.52 pokazuje jak niska tolerancja sprawia, że zamieniany jest wyłącznie kolor podany jako celowy, natomiast przy większej wartości zamianie podlega coraz szerszy przedział kolorów. Przykład zmiany o tolerancji równej 1 sprawia, że wszystkie kolory o wysokiej wartości R zostają uwzględnione w przemalowaniu, którego oryginalnym kolorem był pomarańczowy. Dochodzi do tego ponieważ nawet duże odstępy na zieleni lub błękicie są uwzględniane.

Shader działa na wartościach nasycenia kolorów 0-1 zamiast tradycyjnego dla przestrzeni RGB 0-255, więc przy stosowaniu tolerancji należy stosownie używać równej przestrzeni. Aby zaoszczędzić na przesyłanych danych do GPU, tolerancja zostaje przesłana jako wartość przezroczystości (A) w kolorze celowym. Rzeczywiste przemalowanie pozostawi oryginalną przezroczystość. Należy też zwrócić uwagę na to, że shader jest przeznaczony do grafik dwuwymiarowych i działa najlepiej z rysunkami z ostrymi krawędziami, gdzie nie ma artefaktów lub gradientów. Palety barw są przechowywane w strukturze SO i są listą barw RGBA.

Przygotowane zostały dwa rodzaje kontrolera zmian: manualny i bazujący na predefiniowanych paletach. W manualnym należy podać kolory docelowe i oryginalne, natomiast w drugim podawane są kolory oryginalne i paleta barw, która jest już gotową listą docelowych kolorów. Oba kontrolery obsługują też proceduralną zmianę kolorów, ponieważ została ona zaimplementowana w klasie bazowej. Proceduralność polega na podaniu listy palet kolorów oraz ziarna, pozostawiając kontrolerowi do wyboru jedną paletę barw za pomocą entropii z ziarna.



Rysunek 3.53: Przykład rzeczywistej implementacji zmiany kolorów. Obiekt jest 11-kolorową tęczą, a kontroler zawiera 3 manualne zmiany, z czerwonego na niebieski o tolerancji równej 0, z zielonego na czarny o tolerancji równej 0.248, oraz z niebieskiego na biały o tolerancji równej 1.

Rzeczywiste działanie shadera jest zaprezentowane na rysunku 3.53. Pokazane jest rzeczywiste działanie kontrolera manualnego. Ustawione są trzy zmiany:

- Z czerwonego na niebieski, tolerancja = 0 sprawia, że do zmiany nie dochodzi, ponieważ zerowa tolerancja wyłącza przemalowanie
- Z jasnozielonego (5 kolor) na czarny, tolerancja = 0.248 sprawia, że tylko ten jeden kolor został zmieniony.
- Z jasnoniebieskiego na biały, tolerancja = 1 sprawia, że wszystkie odcienie niebieskiego zostały uznane za kolor do zmiany na biały.

W bibliotece znajdują się przykładowe palety barw, zostały one przygotowane z myślą o obiektach w grze, lecz można ich użyć do czegokolwiek. Korzystając z palety należy upewnić się, że ustawiony kolor oryginalny jest ustawiany na zmianie o dobrym indeksie, aby został dobrany do pary z oczekiwanym kolorem palety.

Palety\ Kolory	Bez shadera	Dayo	Windy	Winter
Kolor 0 Szata maga				
Kolor 1 Twarz maga				
Kolor 2 Cień na twarzy				
Kolor 3 Różdżka				
Kolor 4 Detale szaty				
Kolor 5 Trawa podstawa				
Kolor 6 Trawa detale				
Kolor 7 Woda podstawa				
Kolor 8 Woda detale				
Kolor 9 Góry podstawa				
Kolor 10 Góry środek				
Kolor 11 Góra szczyt				
Kolor 12 Przeciwnik środek				
Kolor 13 Przeciwnik zarys				
Kolor 14 Wolne				
Kolor 15 Wolne				

Rysunek 3.54: Porównanie kolorów oryginalnych z zamiennikami w wybranych paletach barw. Pierwsza kolumna pokazuje oryginalne kolory bez zmiany, następnie pozostałe 3 pokazują wybrane palety barw. W wierszach znajdują się pojedyncze zmiany z oryginału na wartość docelową.

Palety\ Kolory	Bez shadera	Dayo	Windy	Winter
Mag				
Trawa				
Woda				
Góry				
Przeciwnik				

Rysunek 3.55: Przykładowe przemalowane elementy gry przy wykorzystaniu rysunku postaci gracza (mag), pola trawy, gór, wody oraz rysunku przeciwnika. Ukazane palety barw są komplementarne do tych w rysunku 3.54.

Rysunki 3.54 i 3.55 pokazują przykładową implementację palet barw. W przeciwieństwie do zmian manualnych pokazanych na rysunku 3.53, zmiana poprzez paletę wymaga utrzymania zgodnej struktury kolorów oryginalnych, w przeciwnym wypadku kolor docelowy, który miał być nakładany na np. skórę, może zostać nałożony na np. szatę. W wypadku kontrolera manualnego taki błąd byłby trudniejszy do uzyskania, ponieważ wymagałby dużego błędu dewelopera.

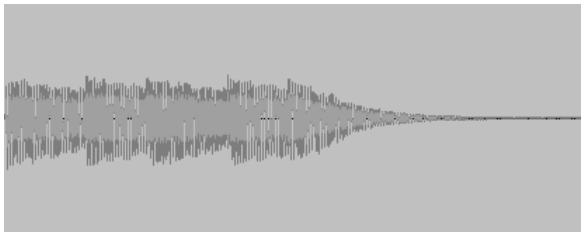
Zmiany na obiektach dźwiękowych

Struktura modyfikacji audio jest dosyć podobna do modyfikacji wideo. Istnieje centralny kontroler przyjmujący obiekty zmian i komunikujący się z elementem w grze, gdzie zamiast shadera jest to źródło audio. Kontroler komunikuje się z "AudioSource", aby ustawić w nim dźwięk i jego modyfikacje. Można wypełnić modyfikatory w inspektorze manualnie, co spowoduje stałą modyfikację. Celem podsystemu jest zapewnienie kolejnej warstwy widocznie zmiennej dla gracza w generowanych obiektach bez potrzeby pokazania wewnętrznych funkcji gry.

Zmiana dźwięku polega na poddawaniu go modyfikacjom, system został zaimplementowany tak, że przygotowano typy modyfikacji w postaci plików skryptowych, deweloper może utworzyć nieskończoną liczbę takich modyfikatorów, ale też dopisać własny typ modyfikatora. Kontroler komunikuje się z "AudioSource", aby ustawić w nim dźwięk i jego modyfikacje. Można wypełnić modyfikatory w inspektorze manualnie, co spowoduje stałą modyfikację. Modyfikacje przekazywane w bibliotece to:

- SimpleRandomizer - deterministycznie zmienia ton i głośność dźwięku. Nowe wartości będą wybrane między minimum i maksimum ustalonymi w instancji modyfikatora. Nowe wartości są niezależne od siebie.
- SpectralMuffler - nakłada filtr dolnoprzepustowy (Low Pass Filter). Powoduje efekt "przytłumienia", daje efekt dźwięku np. pod wodą lub za ścianą.
- ReverbRoulette - deterministycznie wybiera i nakłada wybrany *AudioReverbFilter* Unity na źródło dźwięku. Posiada też flagę sprawdzenia bitu, modyfikacja nastąpi tylko, jeśli ziarno generacji ma ustawiony wybrany bit.
- PerlinDrift - Nakłada ciągłą zmianę tonu na źródło dźwięku używając szumu Perlina. Sprawia, że tonacja płynnie przechodzi od wysokich do niskich wartości.
- LFOWobble - nakłada na tonację fale sinusoidalną LFO (Low-Frequency Oscillator). Tworzy przez to efekt wibracji (vibrato).
- ClipSegmenter - deterministycznie wybiera fragment klipu audio. Można ustalić minimalną i maksymalną długość podklipu.
- BitwiseGlitch - gwałtownie nakłada duży odstęp na ton. Sprawia efekt podobny do zakłócenia sygnału (digital stuttering).
- AudioCrusher - nakłada stałe zniekształcenie (AudioDistortionFilter) na źródło.

Rysunek 3.56: Kształt fali przykładowego pliku audio (5.wav dostarczonego z projektem) przed modyfikacją.

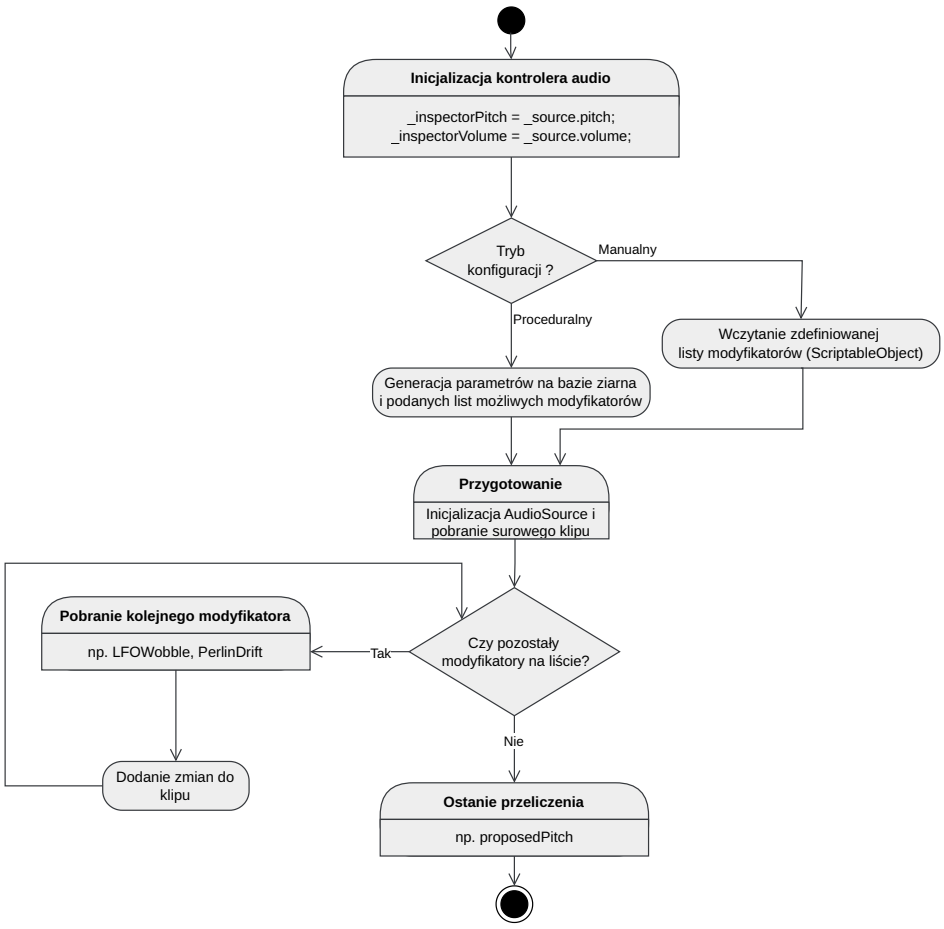


Rysunek 3.57: Kształt fali pliku audio 5.wav po nałożeniu "BitwiseGlitch" i "LFOWobble"

Dźwięk prezentowany na rysunku 3.56 w wyniku modulacji został wydłużony oraz ma bardziej "poszarpaną" strukturę. Stan końcowy jest widoczny na rysunku 3.57.

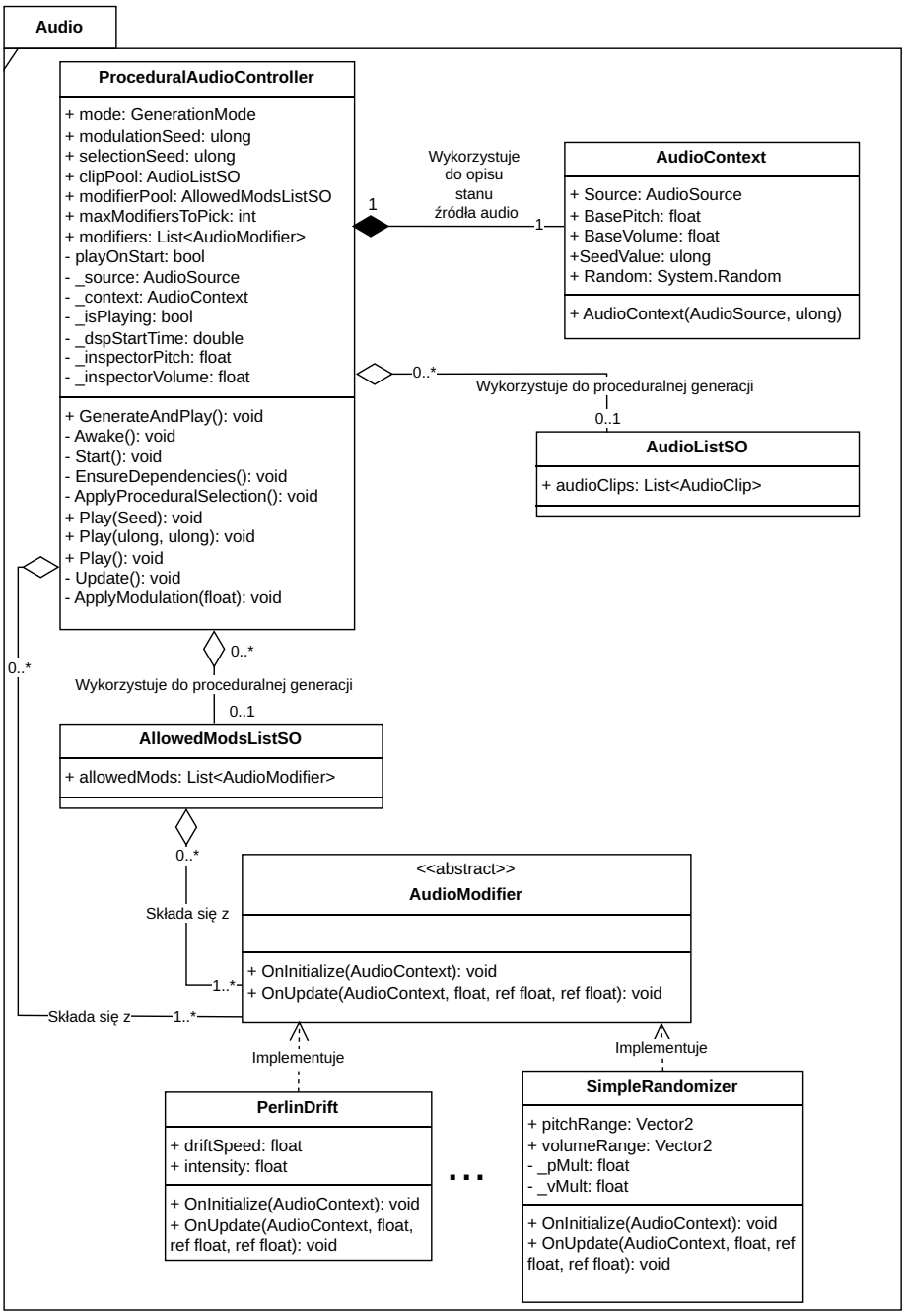
Kroki algorytmu odpowiadającego za modyfikację dźwięku są zaprezentowane na rysunku 3.58. Metoda manualna polega na wczytaniu parametrów ustalonych w kontrolerze. Aby użyć metody proceduralnej należy podać dwa źródła entropii oraz dwie listy, w pierwszej liście znajdują się klipy audio, w drugiej możliwe modyfikatory. Obie listy są obiektami SO, "AllowedModsListSO" zawiera listę modyfikatorów, których można użyć do tego źródła, a "AudioListSO" zawiera klipy audio.

Niezależnie od rodzaju dostarczenia modyfikatorów, kontroler następnie zacznie aplikować je na podane źródło audio aż do wyczerpania listy, ostatecznie dostarczając nowy klip audio.



Rysunek 3.58: Algorytm modyfikacji dźwięku wykonywanej poprzez generację audio w bibliotece.

Zarówno generacja audio, jak i wideo opiera się o kontroler przechowujący planowane zmiany na oryginalnym komponencie, który dalej komunikuje się ze składowymi w silniku Unity. Diagram UML do audio (Rysunek 3.59) jest zatem podobny do diagramu generatora wideo (Rysunek 3.51). Audio podobnie ma centralny kontroler, który zawiera listy SO zawierających zmiany (AllowedModsListSO) oraz obiekty oryginalne (AudioListSO). W systemie wideo, kontroler komunikował się z shaderem, w audio natomiast do komunikacji następuje między kontrolerem, a kontekstem audio (AudioContext), który jest stanem audio danego klipu i wpływa bezpośrednio na implementację dźwięku od strony silnika. AudioModifier jest klasą abstrakcyjną na bazie której utworzone są wszystkie modyfikatory audio, takie jak np. perlinDriftt.



Rysunek 3.59: Diagram pakietów obrazujący strukturę i zależności wewnątrz TDPG.AudioModulation.

3.3.5 Szablony kluczowych elementów gry typu "Tower Defense"

Przestrzeń szablonów jest silnie połączona z silnikiem Unity.

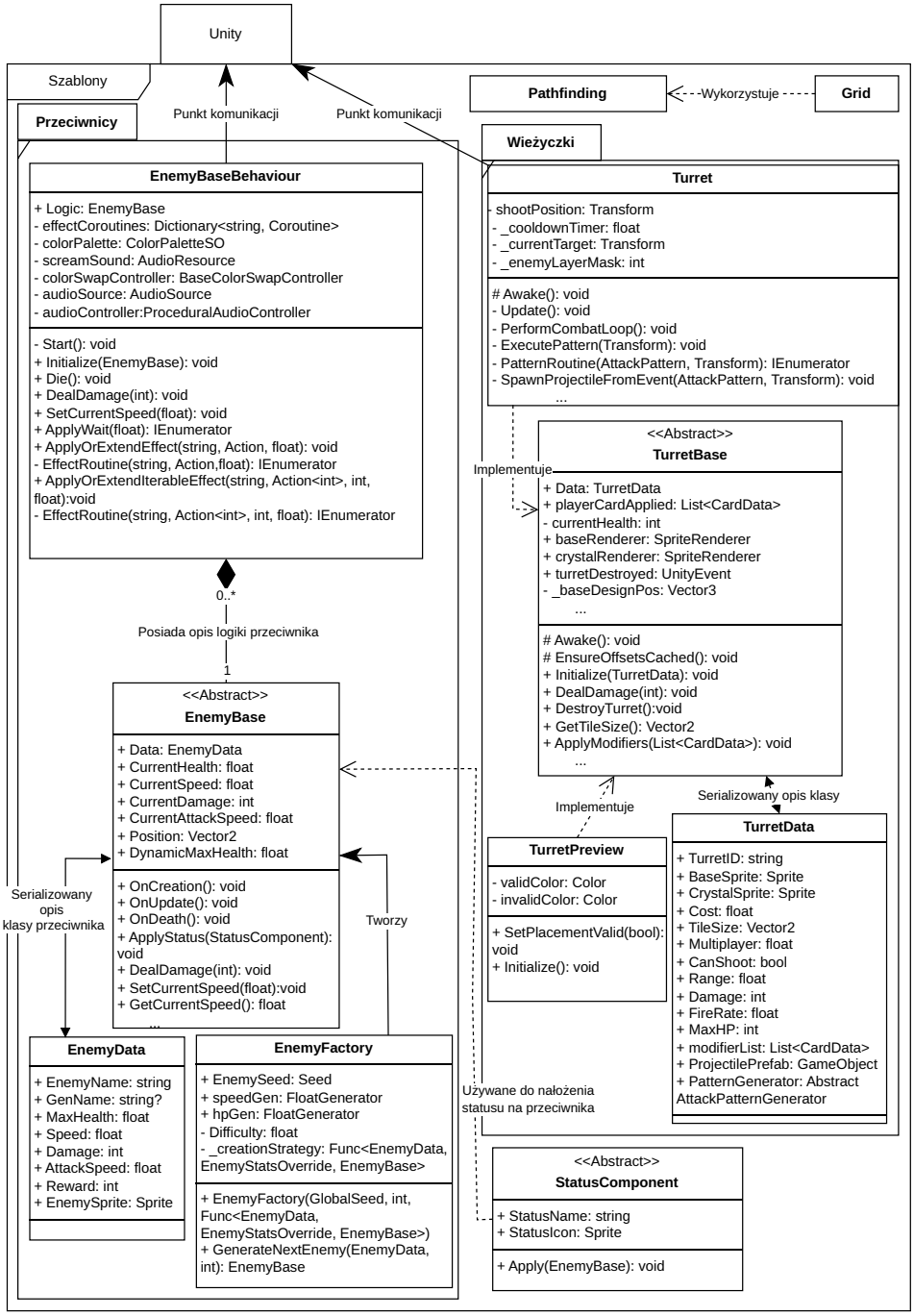
Przestrzeń składa się z przygotowanych przez autorów podstaw implementacji klas przeciwników, wieżyczek oraz elementów mapy. Algorytmy związane z siatką zostały wytłumaczone we wcześniejszym rozdziale (rozdział: 3.3.2). Podprzestrzenie związane z przeciwnikami i wieżyczkami są zbiorem klas abstrakcyjnych i klas bazowych, na których deweloper gry powinien stworzyć prawdziwe implementacje tych obiektów do scen. Zostały one przygotowane w celu ułatwienia implementacji biblioteki w grze oraz jako przykład ich wykorzystania.

Przeciwnik w bibliotece zawiera wszystkie parametry dla klasycznego wroga w grze typu obrona wież: prędkość, moc ataku oraz punkty życia. Ponieważ przeciwnik porusza się po siatce, do opisania jego pozycji wystarczy Vector2 mający pozycje x,y.

Klasa "EnemyData" jest obiektem skryptowym pozwalającym szybko podmieniać bazowe parametry przeciwnika, pojedyncza taka instancja pełni rolę rodzaju wroga (np. jedno SO dla "Goblina", "Gluta", "Orka" itd.). W bibliotece znajduje się też podstawowa forma fabryki wrogów. Klasa "EnemyBaseBehaviour" została zaprojektowana z myślą o przygotowaniu dzięki niej rodzaju przeciwnika w Unity i zawiera integrację z generacją audio oraz wideo poprzez ich kontrolery opisane wcześniej (rozdział 3.3.4).

"Status Component" jest abstrakcyjną klasą używaną do wizualizacji nałożonego efektu na przeciwniku.

Wieżyczka ma podobny schemat, składają się na nią: klasa abstrakcyjna "TurretBase", implementująca tę klasę abstrakcyjną klasa bazowa (tutaj "Turret") oraz obiekt skryptowy zawierający wypełnione statystyki obiektu. Oprócz tego wykorzystywana jest również specjalna klasa "TurretPreview", która jest lekką implementacją. Celem tej klasy jest stworzenie małego obiektu wieży przed umieszczeniem jej na polu, ma ona wizualizować czy dane pole na siatce jest dozwolone do zabudowy.



Rysunek 3.60: Diagram klas przedstawiający architekturę logiczną i kluczowe zależności w przestrzeni nazw TDPG.Templates, wyjaśnione w rozdziale 3.3.5.

Działania faktycznych implementacji przeciwników i wieżyczek zostaną opisane w następującym rozdziale 4.

Rozdział 4

Środowisko Testowe (Gra)

4.1 Integracja biblioteki i architektura gry

W celu zaprezentowania jak największej części możliwości udostępnianych przez bibliotekę, w trakcie jej tworzenia równolegle powstawało środowisko testowe. Z racji specyfiki projektu przybrało ono formę dema technologicznego gry, dlatego też od tego momentu będzie ono w ten sposób nazywane. Strukturę gry można podzielić według następujących kategorii:

- bezpośrednie użycie skryptów zawartych w bibliotece (1)
- implementacja klas i systemów biblioteki korzystająca z zestawów schematów (TDPG.Templates) (2)
- elementy niepowiązane z biblioteką (3)

Wewnętrzna struktura skryptów utworzonych na potrzeby gry przyjmuje następującą formę:

- Camera (3)
- Enemies (2)
  - Enemy.cs - przetrzymywanie statystyk i zarządzanie nimi; dziedziczy po schemacie EnemyBase
  - EnemyBehaviour.cs - zarządzanie interakcją przeciwników z szerszym światem i systemami gry; dziedziczy po schemacie EnemyBehaviourBase
  - EnemyRegistry.cs - ładuje i zarządza danymi przeciwników z plików
  - EnemySpawner.cs - tworzy nowych przeciwników według rozkazów WaveManagera; wewnętrznie korzysta ze schematu EnemyFactory
- Minimap (3)
- Player (3)
- Projectiles (2)
  - BasicProjectile.cs - implementacja dziedzicząca po schemacie Projectile
  - EffectProjectile.cs - rozszerzenie BasicProjectile.cs korzystające z systemu żywiołów zaimplementowanych w bibliotece
  - ProjectileSpawner.cs - skrypt do tworzenia pocisków

- Systems (2 i 3)
  - GameManager.cs - zapewnia zapis oraz wczytanie rozgrywki, zawiera funkcje wspomagające integracje między systemami oraz zapewnia płynne i prawidłowe przejście między scenami Unity; Trzyma obiekt GlobalSeed
  - RegistryManager.cs - zarządza systemem efektów udostępnionym przez bibliotekę (TDPG.EffectSystem)
  - ResourceSystem.cs - zbiór funkcji udostępniających proste zasoby wykorzystywane m.in. przy budowie wież
  - SaveData.cs - zbiór definicji obiektów transferu danych (DTO) oraz struktury pliku zapisu
  - WaveManager.cs - system fal ataku przeciwników; dyktuje ilość, rodzaj oraz przyrost statystyk przeciwników
  - skrypty typu Compendium - kontenery zbierające instancje obiektów wraz z funkcjami pomocniczymi umożliwiającymi m.in. iterację i serializację
- Turrets (2)
  - TurretHPBarManager.cs - odpowiada za wizualizację obrażeń otrzymanych przez wieżyczkę
  - TurretList.cs - kontener zbierający instancje wieżyczek znajdujących się obecnie na mapie
  - TurretRegistry.cs - ładuje i zarządza danymi wieżyczek z plików
  - TurretSelection.cs - skrypt wyboru wieżyczki wspomagający system wizualizacji i budowy
  - TurretSpawner.cs - skrypt odpowiadający za wizualizację pozycji wieżyczki na mapie przed jej postawieniem oraz prawidłowe jej utworzenie we współpracy z systemem mapy; pośrednio korzysta ze schematu Turret
- UI (3)

W powyższym zestawieniu nie pojawiają się elementy kategorii (1), ponieważ są one przypisywane bezpośrednio w inspektorze Unity i nie występują w kodzie.

4.2 Systemy tylko w grze (UI, Save/Load, Input system, Kamera)

W celu odtworzenia realistycznego przebiegu pracy dewelopera oraz przetestowania integracji biblioteki z silnikiem Unity, autorzy gry podjęli się również implementacji elementów gry niebędących bezpośrednio w zakresie biblioteki. Pomogło to w ujednoliceniu końcowych form interfejsów biblioteki oraz identyfikacji obiektów nadających się na szablony biblioteki.

Interfejs użytkownika został zaimplementowany z użyciem systemu uGUI dostarczonego przez silnik. Umożliwia on tworzenie interfejsu poprzez prosty system hierarchiczny, zintegrowany z drzewem obiektów gry oraz składający się z komponentów takich jak panele, przyciski i suwaki. Istotną funkcjonalnością jest możliwość edycji domyślnego działania tych części z poziomu kodu, co znacząco ułatwiło integrację z niestandardowymi elementami biblioteki.

System zapisu i ładowania stanu rozgrywki nie potrzebował zewnętrznych zależności do osiągnięcia pełni funkcjonalności. Jego implementacja w znaczącym stopniu opierała się na możliwości

serializacji obiektów bibliotecznych. W przypadku elementów, których nie można było zserializować w sposób bezpośredni, utworzono obiekty transferu danych. Umożliwiło to ręczne przekazanie parametrów potrzebnych do odtworzenia aktualnego stanu gry. W ten sposób wykorzystano prostotę serializacji czystych klas języka C# przy zachowaniu kompatybilności z wewnętrznymi strukturami Unity. Po zebraniu wszystkich danych zostały one zapisane do pliku w formacie JSON, w trwałej lokalizacji, dając potencjalnemu graczowi możliwość wznowienia rozgrywki po zamknięciu programu.

Unity oferuje dwa wbudowane systemy wykrywające dane wejściowe z klawiatury, myszki oraz innych urządzeń. Starszy system (Input Manager) oferuje zestaw prostych funkcji, które sprawdzają, czy dany przycisk został wciśnięty. Drugi system (Input System Package) oferuje zestaw wydarzeń, które udostępniają podobne funkcjonalności do wykrywania interakcji z urządzeniami wejściowymi. Różnica w zastosowaniu tych systemów polega na tym, że Input Manager wymaga sprawdzania stanu klawiszy wejściowych co klatkę gry, powodując aktywne czekanie, a Input System Package pozwala na wywołanie kodu idealnie wtedy, kiedy jest to wymagane. Drugi system jest przez to wydajniejszy. Oprócz tego Input System Package pozwala na mapowanie konkretnych akcji pod kilka przycisków. Dzięki temu dla jednej akcji strzelania można ustawić np. lewy przycisk myszy, spację oraz klawisz "F". Takie rozwiązanie sprawia, że kod gry jest łatwiejszy w zarządzaniu i można łatwiej dostosować go pod różne urządzenia takie jak klawiatura, myszka, kontroler, itd.

Z powodu łatwiejszego zarządzania kodem oraz wydajnością autorzy zdecydowali się na wybranie drugiego systemu. W samej grze wykorzystano go do obsługi poruszania się postacią gracza. Gracz jest wstanie poruszać się nią w osiach X i Y oraz tworzyć pociski, które pojawiają się i lecą w kierunku punktu wskazanego kursorem myszki. Oprócz tego wykorzystując ten system zrobiono także otwieranie okienek interfejsu użytkownika, takich, jak menu budowania pozwalające na wybranie wieżyczki, a następnie postawienie jej w miejscu wskazanym przez kursor myszy. Drugim okienkiem wyświetlanym za pomocą systemu jest okienko menu gry, w którym można zapisywać i wczytywać stan gry lub zmienić przypisanie przycisków pod preferencje gracza.

Do wyświetlania obiektów na ekranie wykorzystywany jest wbudowany w silnik obiekt kamery, jednak aby nią poruszać należy stworzyć własny skrypt lub zastosować paczkę Unity "Cinemamachine". Paczka ta pozwala na ruch kamery i dostosowanie jej parametrów na poziomie silnika gry. Autorzy potrzebowali prostego systemu kamery, dlatego zrezygnowano z użycia tej paczki, z której nie użyto by większości funkcjonalności, a mogłaby wpłynąć na wydajność gry. Najprostszym sposobem poruszania kamery jest przypisanie kamery jako dziecko obiektu np. postaci gracza, który posiada skrypt poruszania się. W efekcie kiedy postać się rusza, kamera rusza się razem z nim. Jednak takie rozwiązanie często wpływa negatywnie na odczucia gracza. Dlatego zdecydowano się na lekko zmodyfikowaną wersję. Zastosowana wersja nie używa relacji rodzic-dziecko, tylko co klatkę ustawia się w punkcie między postacią gracza, a pozycją myszki w świecie gry. Dzięki temu rozwiązaniu gracz jest wstanie widzieć więcej w miejscu, które obecnie go interesuje i w przypadku stworzonego dema technologicznego lepiej celować na dalszy dystans. Podczas testowania zauważono drobną wadę polegająca na utrudnionym stawianiu wież w konkretną kratkę siatki. Rozwiązano ten problem poprzez wyśrodkowanie kamery na postaci gracza kiedy gracz posiada wyświetlone menu budowania.

4.3 Game Loop

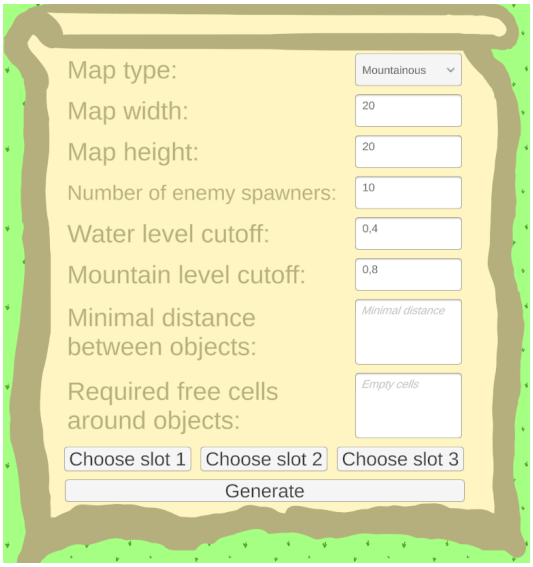
Po rozpoczęciu rozgrywki gracz pojawia się obok bazy - punktu, którego obrona jest jego zadaniem. Dzięki minimapie może szybko zorientować się w terenie i zidentyfikować potencjalne ścieżki, którymi nadejdą przeciwnicy. Po ustawieniu się w dogodnym miejscu gracz ma możliwość zbudo-

wania wież, które pomogą odeprzeć natarcie na bazę poprzez automatyczne atakowanie wrogów znajdujących się w zasięgu. Ponieważ zbudowanie wieży wymaga many - ograniczonego zasobu, gracz musi zrozumieć zalety i wady dostępnych mu rodzajów wież i sprytnie dopasować ich użycie do sytuacji na polu walki. Każdy pokonany oponent nagradza gracza niewielką porcją many - umożliwiając budowę nowych konstrukcji obronnych. Pokonanie wszystkich wrogów obecnych na mapie daje graczowi krótką chwilę na przemyślenie dalszej strategii oraz wybranie karty ulepszenia wieży. W momencie zakończenia przerwy między falami pojawiają się nowi adwersarze - silniejsi, w większej liczbie, a czasem i w zupełnie nowej formie, zmuszając gracza do adaptacji, budowy nowych wież oraz ulepszenia obecnie istniejących, co powoduje zamknięcie pętli rozgrywki.

4.4 Przykład działania generatorów na poszczególnych etapach rozgrywki

Po wejściu w menu główne gracz może rozpocząć nową grę. Tam może wybrać parametry generacji mapy, na której będzie toczyć się rozgrywka. Udostępnione graczowi parametry (Rysunek 4.1) to:

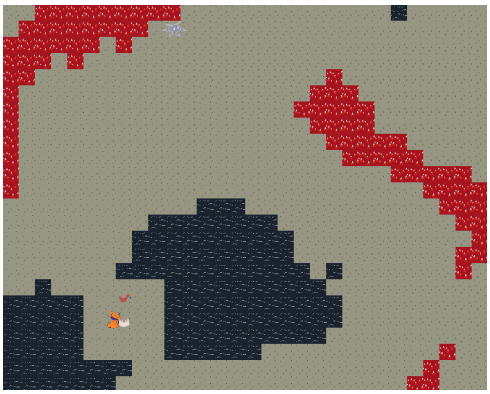
- Typ mapy (które zostały przedstawione w rozdziale 3.3.2)
- wymiary mapy (wysokość i szerokość)
- liczba spawnerów
- gęstość kafelek gór i wody
- minimalna odległość między spawnerami, a celem obrony
- wielkość pustej przestrzeni wokół celu obrony



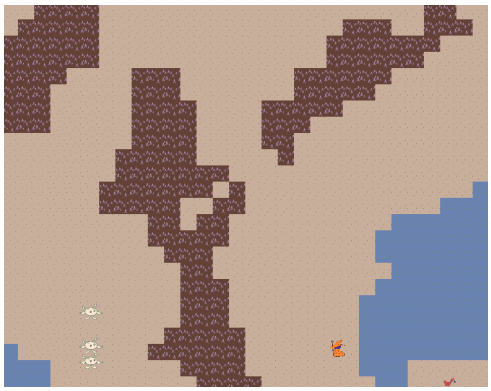
Rysunek 4.1: Menu wybierania parametrów generacji graczy zaimplementowane w grze

Jeśli gracz nie zmodyfikuje pola zostaną wybrane domyślne wartości. Następnie za pomocą generatora mapy udostępnionego z tworzonej biblioteki zostanie stworzona mapa gry, na której zostaną postawione spawnery, obiekt obrony oraz postać gracza. Oprócz tego zostanie wygenerowane ziarno

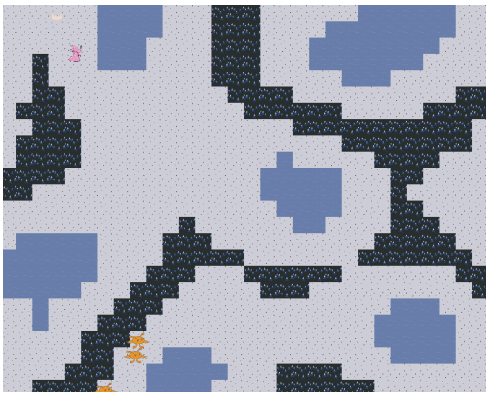
mapy, które będzie wykorzystywane przez obiekt generatora i inne systemy pochodne. Odpowiednie kafelki np. wody zostaną wizualnie wyświetlone na mapie oraz zostanie dobrana losowo wybrana paleta barw (przykładowe wygenerowane mapy zostały przedstawione na rysunkach 4.2 - 4.5.



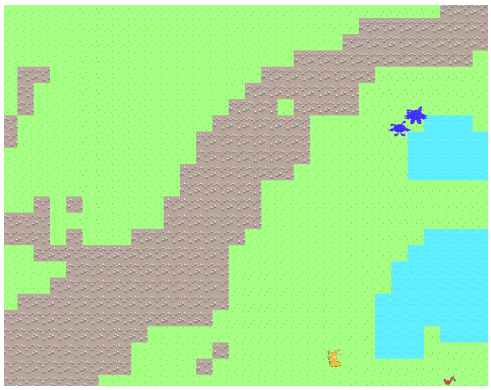
Rysunek 4.2: Mapa wygenerowana wykorzystująca parametr typu "Smooth"oraz o wymiarach 30x30 klatek. Pozostałe wartości zostały domyślne



Rysunek 4.3: Mapa wygenerowana wykorzystująca parametr typu "Smooth"oraz o wymiarach 30x30 klatek. Pozostałe wartości zostały domyślne. Mimo, że wykorzystuje te same parametry co mapa przedstawiona na rysunku (4.2) widać, że różni się paletą barw oraz ułożeniem pól



Rysunek 4.4: Mapa wygenerowana wykorzystująca parametr typu "Mountainous"oraz o wymiarach 30x30 klatek. Pozostałe wartości zostały domyślne

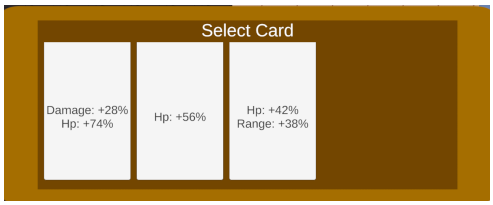


Rysunek 4.5: Mapa wygenerowana wykorzystująca parametr typu "Chaotic"oraz o wymiarach 30x30 klatek. Pozostałe wartości zostały domyślne

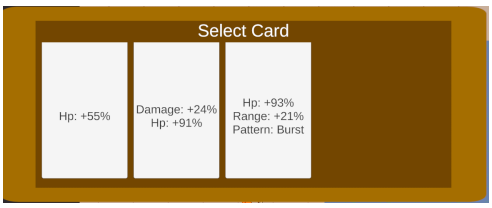
Spawnery co falę będą tworzyć nowych coraz silniejszych przeciwników, którzy dzięki generatorowi skalarów z biblioteki będą posiadali nieznacznie różniące się wartości statystyk. Każdy typ przeciwnika będzie posiadał różne nazwy wygenerowane za pomocą zaimplementowanych łańcuchów Markowa. (który został omówiony w rozdziale 3.3.4) Przeciwnicy wykorzystują zaimplementowany w bibliotece algorytm A* (który został omówiony w rozdziale 3.3.2) do szukania optymalnych ścieżek, które będą aktualizowane kiedy dojdzie do zmiany na mapie ze względu na akcje gracza, który stawia przeszkodę na drodze.

Po zakończeniu fali gracz zostaje nagrodzony kartami ulepszeń, których może używać do ulepszenia wieżyczek. Wartości kart ulepszeń są generowane na podstawie ziarna stworzonego na początku generacji mapy. Podczas tworzenia karty ulepszeń mogą powstać także nowe elementy wykorzystujące system efektów z biblioteki. Pociski wieżyczek posiadające takie elementy po trafieniu w przeciwnika mogą wywołać dodatkowe osłabienia takie jak redukcja prędkości ruchu lub zadawanie obrażeń w czasie (ang: "Damage over time"). Dodatkowo karta ulepszeń może także modyfikować sposób strzelania wieżyczek wykorzystując generator wzorców ataków z biblioteki. Dzięki

temu wieżyczka zamiast strzelać pojedynczym pociskiem może np. w krótkim czasie wystrzelić grad pocisków, a następnie będzie musiała czekać przez okres ustawiony przez generator.



Rysunek 4.6: Otrzymane karty wygenerowane po zakończeniu fali



Rysunek 4.7: Otrzymane inne wartości kart, które zostały wygenerowane w innej iteracji niż pokazano w rysunku 4.7

Rozdział 5

Testy, parametry metod i rezultaty

W celu zapewnienia efektywności działania opracowanych rozwiązań w bibliotece, przeprowadzono badanie ich funkcjonalności pod różnymi parametrami. Celem tych testów było wyznaczenie przedziałów parametrów, które zwracają satysfakcjonujące wyniki, ponieważ gdyby oddać graczowi pełną nieograniczoną przestrzeń wartości, to generacja mogłaby prowadzić do skrajnych lub nieodpowiednich rezultatów. Z uwagi na brak analitycznych metod pozwalających na jednoznaczne określenie parametrów, zostały one wyznaczone w sposób heurystyczny.

Systemy, które wymagały takich testów to:

- Generacja tekstu, a dokładniej mówiąc parametr rzędu w łańcuchach Markowa, aby dowieść utrzymania teoretycznej optymalności rzędu=2 z literatury [14] w implementacji biblioteki TDPG.
- Generacja map, parametry odcięć poziomów terenów na mapie w celu zapewnienia grywalnej, ale nietrywialnej mapy.
- Generacja wideo, parametr tolerancji, podawany przy zmianie kolorów, aby wyznaczyć przedział kolorów w danej parze.

Testy były przeprowadzone pod koniec tworzenia biblioteki, na osobnej gałęzi repozytorium projektu. Każdy z testów polegał na użyciu danego systemu w odizolowanym środowisku i zebraniu uśrednionych wyników, aby wykluczyć z nich losowość. Testy zostały przeprowadzone na dwóch komputerach, pierwszy będący komputerem stacjonarnym z procesorem Ryzen 5 5600X, kartą graficzną GeForce RTX 3060 oraz 32GiB DDR4 RAM. Drugą maszyną był laptop Lenovo Ideapad 5 15ALC05. Wyniki są uśrednione na bazie otrzymanych z obu maszyn, aby minimalizować zależność otrzymanych wyników od maszyny.

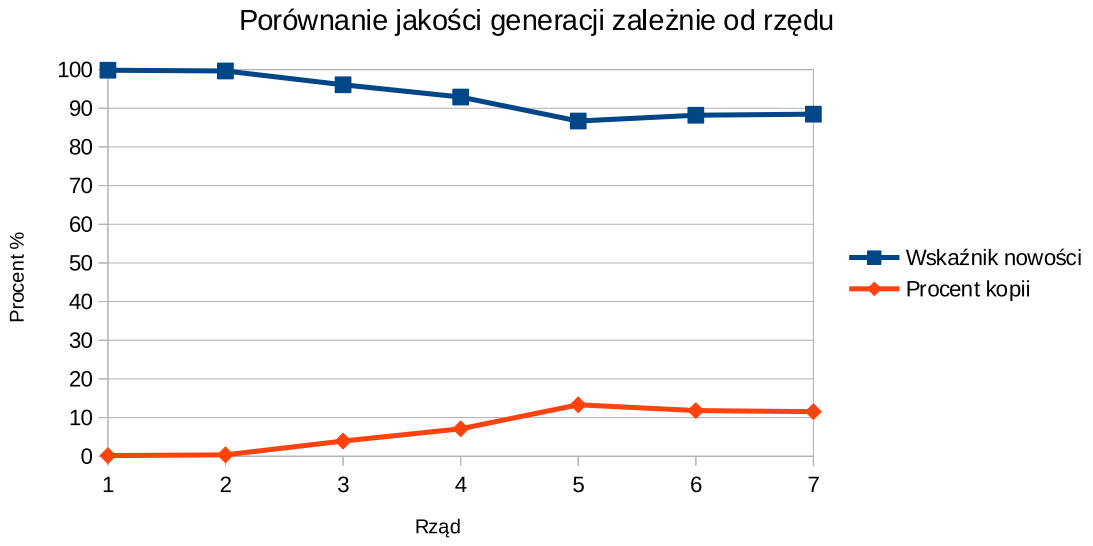
Należy zaznaczyć, że celem testów nie było wyznaczenie bezwzględnych czasów wykonania operacji lub obciążenia procesów, lecz zbadanie relacji i trendów wynikających z danych parametrów wejściowych w generatorach. Przeprowadzenie tych testów sprawia, że znane i zweryfikowane są przedziały, na których generatory działają jak najlepiej, dzięki czemu ich wyniki końcowe są bardziej satysfakcjonujące i odcinane są przedziały prowadzące do błędów.

Generacja tekstu:
W rozdziale 3.3.4 wspomniano, że optymalny rozmiar rzędu dla generatora tekstu w bibliotece, wynosi ok. 2-3. Początkowo była to hipoteza sugerowana przez literaturę [14]. Dodatkowo został przeprowadzony następujący test: Dla rzędów od 1 do 10, 1000 razy wytrenowano generator za pomocą pliku "Names_Fantasy_Clean.txt" następnie wygenerowano na nim dla każdej możliwej litery startowej po jednym wyrazie o długościach od 4 do 10. Następnie to samo na pliku "Elements_Clean.txt". Na wygenerowanych wyrazach sprawdzono:

- Wskaźnik nowości - Procent wygenerowanych wyrazów, które nie występują w danych treningowych (Miara poprawności pod względem kreatywności i generacji)
- Procent kopii - Procent wyrazów wygenerowanych, które mają równy odpowiednik w danych treningowych (można uznać za kopie, "overfitting")
- Wskaźnik jakości fonetycznej - Procent wyników, które przeszły prosty test sprawdzający, czy słowo brzmi naturalnie (stosunek spółgłosek do samogłosek).

Podczas testów zauważono, że rzędy największe (8-10), nie mogły zacząć generacji i potrzebowały korzystać z krótszych rzędów, zgodnie z metodą opisaną w rozdziale 3.3.4, więc ich wyniki sprowadzały się do mniejszych rzędów.

Test miał na celu dowieść, że zaimplementowane rozwiązanie generacji tekstu działa tak jak powinno i potwierdza zgodność rozwiązania w bibliotece z tym z literatury.



Rysunek 5.1: Porównanie jakości generacji. Przedstawiony na wykresie został wpływ wartości parametru rzędu na jakość generacji, zmierzoną jako procent duplikatów w wygenerowanym zbiorze.

Jak przedstawiono na rysunkach 5.1 oraz 5.2, rząd=2 posiada mały procent kopii oraz najwyższą jakość fonetyczną. Przy dużych rzędach szansa na wygenerowanie kopii danych treningowych rośnie, tracona jest też jakość nowo wygenerowanych wyrazów. Do ich niskiej jakości fonetycznej dochodzi z powodu tego, iż była liczona tylko dla słów nowych, takie słowa przy dużych rzędach są nieliczne i zazwyczaj w formie przeskoku między dwoma wyrazami. Dla przykładu na treningu: "Poniatowski" i "Nito" rząd=2 może wygenerować poprawny wyraz "Nitowski", a rząd=5 "Nito", ponieważ musi zakończyć generację wcześniej. Wyraz "Nito" nie ma nawet 5 znaków, by utworzyć klucz. Na podstawie tych wyników podjęta została decyzja o ustawieniu wartości parametru rzędu na 2.

Rysunek 5.2: Porównanie jakości fonetycznej w zależności od wartości parametru rzędu. Wskaźnik jakości fonetycznej określony został jako procent elementów wygenerowanego zbioru, dla których stosunek spółgłosek do samogłosek nie odbiega od oczekiwanej wartości, która wynosi 3:1, o więcej niż wartość odchylenia standardowego.

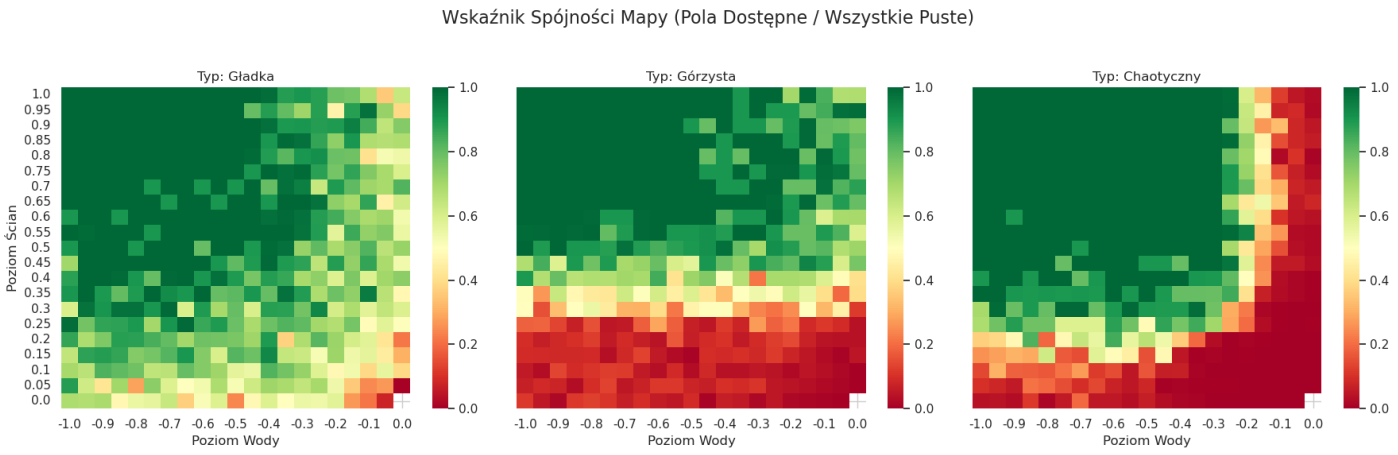
Generacja Map:

Podczas generowania mapy należy wybrać jej rodzaj i parametry odcięć dla poziomów. Wartości w macierzy szumu znajdują się na przedziale od -1 do 1, a ponieważ istnieją trzy poziomy (woda, ziemia, góry), to zakładamy początkowo granicę równą -1 dla wody, 0 dla ziemi, 1 dla gór. Podanie dokładnej granicy jest zostawione w generatorze, jednak zależnie od rodzaju mapy jest ograniczona w innych przedziałach.

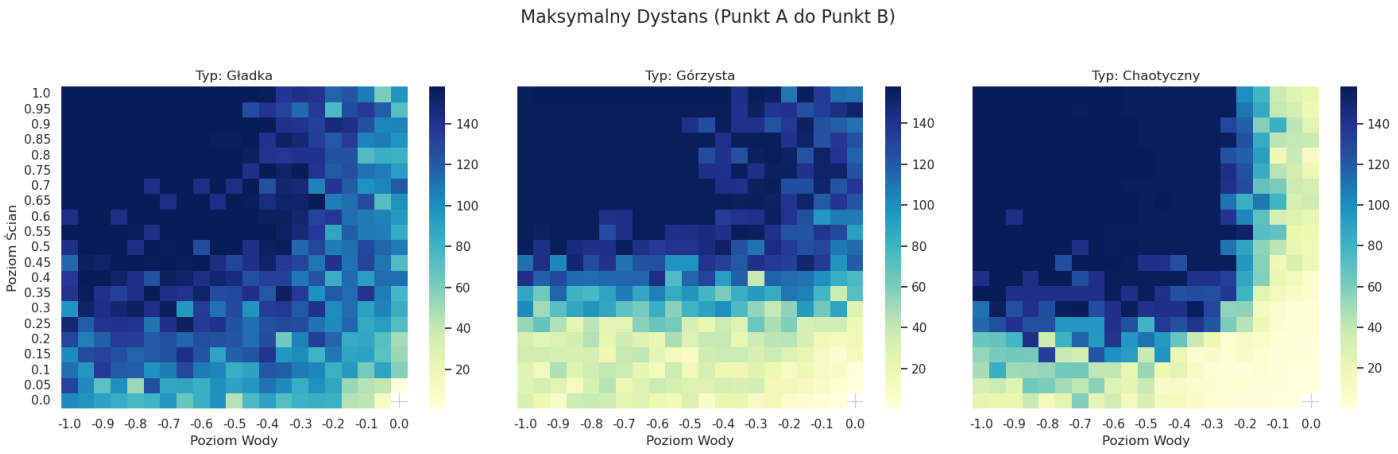
- Smooth - Odcięcie dla wody może być między <-1, -0.001>, a dla gór <0.001, 1>. Do tego rodzaju map jest najmniej ograniczający przedział, ponieważ generacja rzadko zwraca skrajne wartości.
- Chaotic - W tym rodzaju przedziały są limitowane dość mocno, ze względu na odejście od płynnych przejść pól. Jeżeli zostałyby jak w mapie gładkiej, to byłaby niska szansa na poprawną mapę bez użycia algorytmów planu awaryjnego, co niepotrzebnie wydłuża generację z powodu niewiedzy dewelopera lub gracza. Woda: <-1, -0.15>, Góry <0.1, 1>
- Mountainous - Ten sam problem, jak przy mapie chaotycznej. Takie mapy mają najbardziej rygorystyczne przedziały, ze względu na duże zbiory pól. Jeśli powstaną góry, to mogą one stworzyć całe pasmo górskie, bez przejść między nim. Woda: <-1, -0.2>, Góry <0.3, 1>

Granice dla odcięć można uznać za formę planu awaryjnego 0. Jedna iteracja algorytmów naprawy mapy jest mało kosztowna, ale z każdą iteracją poprawki późniejszej ciągną za sobą najpierw pętle innych poprawek.

Wygenerowano 20 map o różnych ziarnach i zbadano je pod względem użyteczności. Zalecanymi wartościami odcięć bezpiecznymi dla wszystkich generatorów, tworzącymi dobre mapy są -0.4 i 0.4. Przez dobrą mapę należy rozumieć taką, która daje dużą ilość pól pustych, ale nie jest cała pusta. Ważny jest też potencjalnie jak największy dystans między dwoma punktami na mapie, ponieważ oznacza to, że cała mapa jest częścią gry. Jeżeli mapa miałaby cel np. w prawym górnym rogu, a wrogowie szliby ze środka mapy, to cały łuk z dolnego prawego rogu do lewego górnego jest



Rysunek 5.3: Mapa ciepła dla wskaźnika spójności map. Wskaźnik spójności oznacza ile na mapie jest pól pustych, z których można dojść do celu. Mapy gładkie mają mniejsze spójności tylko przy odcięciach bardzo bliskich 0, więc powinny w większości sytuacji być dobre na start lub po jednej pętli poprawy.



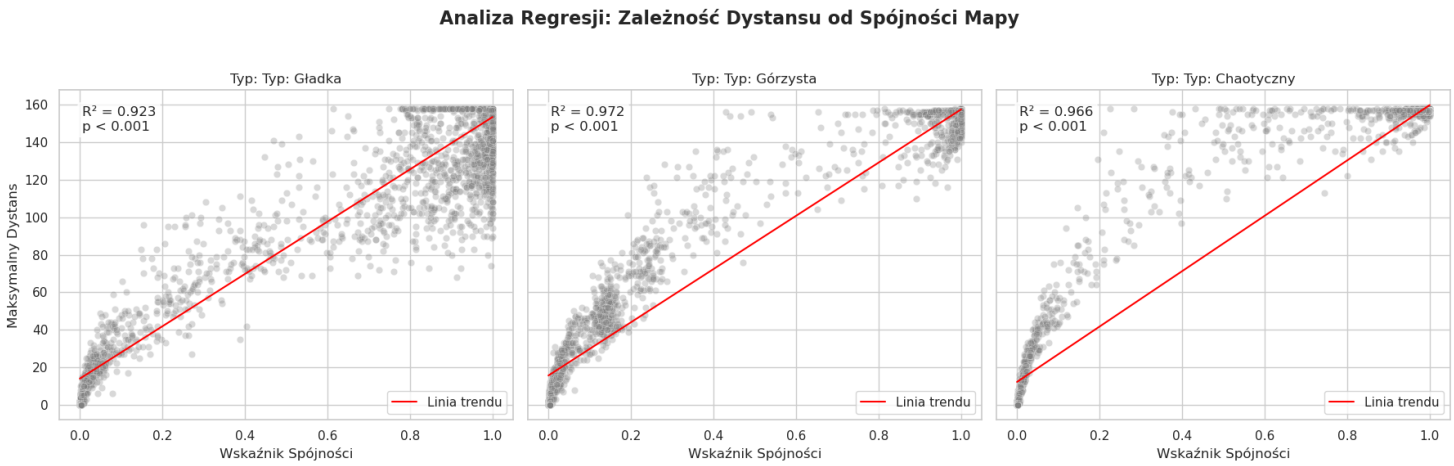
Rysunek 5.4: Mapa ciepła dla maksymalnych dystansów map. Przez maksymalny dystans mapy określa się maksymalny możliwy dystans dwóch punktów na mapie od siebie, między którymi istnieje ścieżka, mierzony poprzez odległość Manhattan, tak jak wytłumaczono w 3.3.2

nieużywany w rozgrywce, ponieważ przeciwnicy nie mają powodu, by tam iść, ani nie jest sensowny dla gracza, gdyż on powinien centrować się wokół celu.

Mapy górzyste natomiast są bardzo zależne od poziomu ścian (ściana będąca wewnętrzną nazwą gór). Z tego powodu zabrania się na poziomie inspektora i kodu aby mapa górzysta była generowana z poziomem ścian poniżej 0.3. Poziom wody także jest ograniczony, ale tutaj z powodu estetyki mapy, bez ograniczenia można tworzyć mapy górzyste gdzie nie ma gór, a jest dużo "jezior".

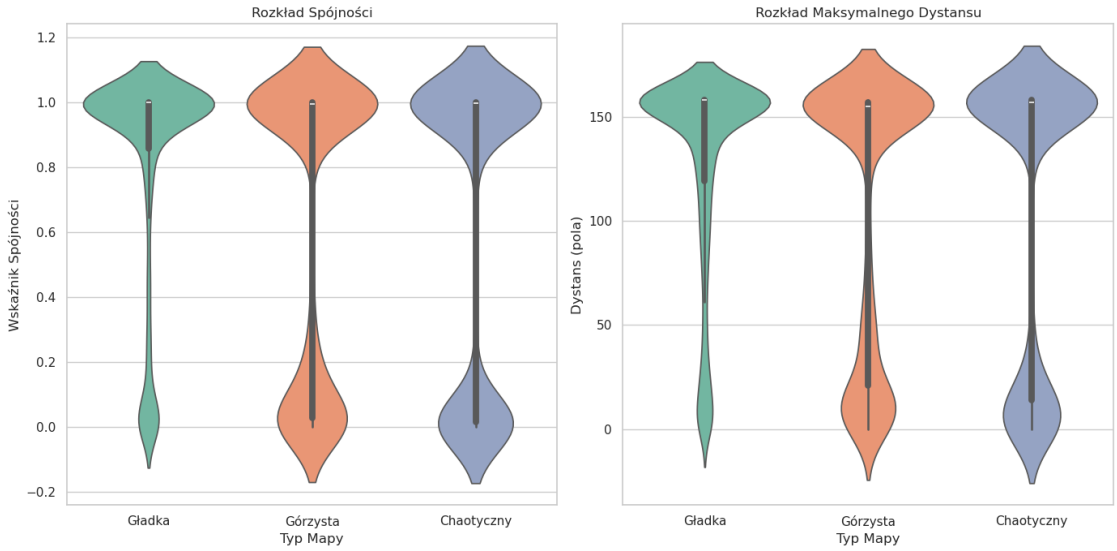
Zakładane jest, że wybierając mapę chaotyczną, wygenerowana mapa ma rzeczywiście wyglądać na wymykającą się schematom. Granice są limitowane do -0.15 i 0.1 właśnie z tego powodu. Mimo występowania pewnej nieoptymalnej spójności na tego rodzaju mapach przy jak najniższych odcięciach, to zostawiana jest ona z pewnym zaufaniem do sensowności deweloperów lub graczy. Granica jest łagodniejsza dla wody, ponieważ przeciwnik może ewentualnie poruszać się po wodzie, ale nigdy po ścianie.

Maksymalne dystanse możliwe na mapach mają prawie identyczny trend, jak użyteczność, więc tym bardziej potwierdzają wartości dobrane na podstawie testów. Podczas badania map silna korelacja widoczna na mapach ciepła zwróciła uwagę autorów, stąd została dodatkowo zbadana korelacja między tymi dwoma parametrami, wykorzystując regresję liniową.



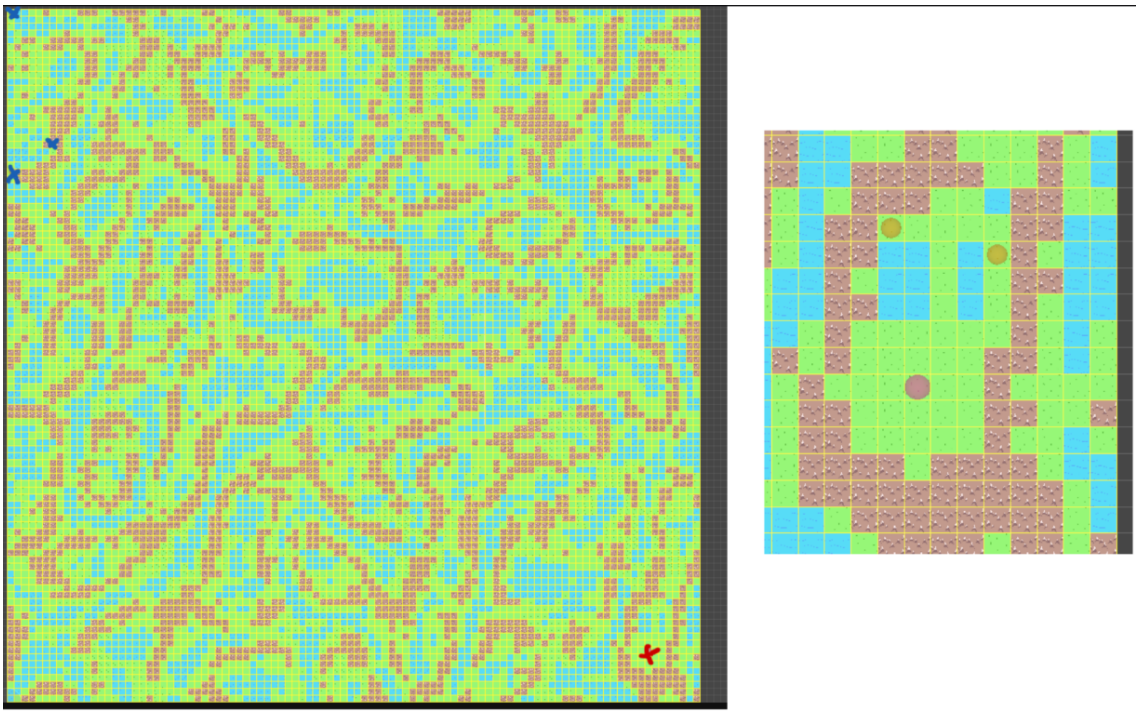
Rysunek 5.5: Wizualizacja regresji liniowej porównująca maksymalny dystans i spójność map.

Przedstawiona na rysunku 5.5 regresja silnie wskazuje na korelację na wszystkich trzech rodzajach map, najsilniej na mapach górzystych. Parametry p oraz R^2 oba mają bardzo skrajne wartości. Analiza regresji liniowej, a dokładniej mówiąc wysokiego parametru R^2 większego od 0.9 oraz parametru $p < 0.001$, na każdym rodzaju mapy wskazuje na silną zależność maksymalnego dystansu o spójności mapy [1]. Potwierdza to przeczucie, że parametry "waterLevel" oraz "wallLevel" nie tylko wpływają na dostępność terenu, ale też na jego poziom komplikacji, a w konsekwencji trudność rozgrywki na mapie.



Rysunek 5.6: Rozkłady wartości spójności mapy oraz maksymalnego dystansu.

Dane w rozkładach gromadzą się przy wartościach skrajnych. Oznacza to, że po pierwszej generacji mapa albo już spełnia wymagania testów, albo ma bardzo małą użyteczność, zazwyczaj spowodowaną poprzez jeden lub tylko kilka pól odgradzających cel od reszty mapy, albo jest mapą o odcięciach zbyt bliskich zeru. Oba te problemy są rozwiązywane przez mechaniki poprawy mapy.



Rysunek 5.7: Przykładowa mapa prezentująca problem jednego pola. Dostęp do celu z dalekich części mapy jest blokowany poprzez jedno pole, którego zwrócony typ uniemożliwia poprowadzenie ścieżki.

Na rysunku 5.7 znajduje się przykładowa wygenerowana mapa, gdzie problem stanowi jedno pole. W przybliżonym fragmencie cel przeciwników (różowe koło) znajduje się w logicznym odpowiedniku doliny, jest otoczony górami. Największe odległości potrzebne do wyznaczenia pozycji spawnerów są zaznaczone jako żółte koła. Ponieważ pole typu woda blokuje wyjście z doliny, to maksymalny dystans jest bardzo mały w skali wielkości mapy. Przy minimalnych poprawkach (zamiana pola wody na pole puste), nowe pozycje dla spawnerów, które są maksymalnie oddalone od celu znajdą się w lewym górnym rogu, wykorzystując przez to o wiele większy procent mapy.

To wszystko silnie wskazuje, że zwiększony wskaźnik spójności mapy pociąga za sobą zwiększony maksymalny dystans na mapie.

Generacja wideo:

W przypadku palette swap nie ma jednej dobrej tolerancji, którą można nałożyć na wszystko. Kontroler właśnie dlatego przyjmuje osobną tolerancję dla każdej z par zmian, zamiast jedną globalną. Każdy rysunek wymaga, aby deweloper zaznaczył pary zmian i ich tolerancje, najlepiej poprzez testowanie na skrajnych kolorach, aby wykryć możliwe pozostałe artefakty, widoczne przy zbyt małej tolerancji.

Działanie tolerancji zostało wytłumaczone w rozdziale 3.3.4. Testowanie, które dowiodło brak pojedynczej tolerancji zostało zauważone podczas implementacji systemu w środowisku testowym dość szybko, więc nie doszło do pełnego testu odnalezienia "idealnej" tolerancji. Jest jednak wspomniane to w obecnym rozdziale, aby wytłumaczyć problem i jego rozwiązanie.

Poza tymi testami, mającymi na celu odnalezienie konkretnych przedziałów wartości, a także demem technologicznym, które samo w sobie pełni rolę testu, przygotowane zostały też testy jednostkowe biblioteki. W wypadku potencjalnych zmian lub rozszerzeń biblioteki, deweloper może się sugerować implementacjami zawartymi w testach.

Rozdział 6

Zakończenie

6.1 Podsumowanie

W ramach pracy zaprojektowano i wykonano bibliotekę, która posiada funkcjonalności: generacji skalarów, wektorów, ziaren oraz wzorców ataków. Uwzględnia też modulacje dźwięków, generator palette swapów, generator tekstu oraz system efektów celem zapewnienia różnorodności w rozgrywce. Biblioteka daje deweloperowi gotowe mechanizmy szablonów przeciwników, wież, siatki oraz jej zarządcy, ale także mechanizm wyszukiwania ścieżek. Opatrzona została też w szczegółową dokumentacje w kodzie, aby deweloperzy korzystający z biblioteki mogli dowiedzieć się jak ją poprawnie wykorzystać.

Oprócz tego stworzono demo technologiczne, które wykorzystuje generatory oraz szablony zaimplementowane w bibliotece. W demie technologicznym zaimplementowano:

- menu główne pozwalające rozpocząć grę i przypisać klawisze.
- generator mapy z biblioteki, którego parametry mogą być wprowadzone przez gracza
- poruszająca się postać gracza
- dynamiczna kamera
- system budowania i wieżyczek
- wieżyczki, które strzelają do przeciwników
- przeciwnicy poruszający się po optymalnej ścieżce z możliwością niszczenia postawionych wież
- system fal generowanych przeciwników
- system generowanych nagród ulepszających wieżyczki gracza
- system zasobów wykorzystywany do budowania wież
- koniec gry w momencie zniszczenia obiektu obronnego

Rozgrywka w środowisku testowym zaowocowała obserwacjami pokazującymi, że systemy znajdujące się w bibliotece rzeczywiście wpływają pozytywnie na grę, np. system ulepszeń jest w stanie pozwolić na obieranie strategii, mogących znacznie ułatwić grę. Wynikło z tego, że konieczne jest wyważenie parametrów w taki sposób, aby przyrost generowanych wartości statystyk przeciwników przewyższał w późniejszym etapie gry przyrost statystyk kart ulepszeń do wieżyczek. Dodatkowo

przy odpowiednio długiej rozgrywce wieże mogą osiągnąć zasięg wielkości całej mapy, co sprawia, że odpowiednie pozycjonowanie wież może stracić znaczenie. Rozwiązanie jest identyczne jak w poprzednim problemie.

Projekt pokazuje, że gry typu "Tower Defense"mogą skorzystać z proceduralnego generowania w wielu aspektach (np. generacja mapy, wzorców ataków, zawartych w grze bonusów). Dzięki temu można zapewnić regrywalność i zachęcić gracza do próbowania nowych kombinacji poprzez narzucanie mu za każdym razem innych sytuacji. Stworzona biblioteka zapewnia już gotowe mechanizmy do implementacji w takich grach.

Celem biblioteki jest ułatwienie pracy deweloperów, aby nie musieli implementować tych samych rozwiązań, co projekt. Dlatego bibliotekę można rozwinąć tak, aby była jeszcze bardziej pomocna dla deweloperów. Jedną z funkcjonalności, jaką można zaimplementować jest system proceduralnego generowania nowych wież w oparciu o algorytm ewolucyjny, bez użycia kart ulepszeń. Oprócz tego bibliotekę można by rozszerzyć o implementację większej liczby powszechnie występujących efektów w obecnym systemie efektów, np. redukcji obrażeń przeciwnika, w celu przyspieszenia pracy deweloperów. Innym systemem, który mógłby się przydać deweloperom jest system wydarzeń, których można by potem używać np. w pociskach wież i wywoływać dodatkowe efekty np. zadawanie obrażeń przeciwnikom w strefie, albo tworzenie się kolejnych pocisków w miejscu trafienia przeciwnika. Do biblioteki można by również zaimplementować system twarzy wektorowych, które pozwoliłyby wizualnie lepiej odróżniać typy przeciwników. System generacji natomiast audio można by rozszerzyć o rozwiązanie binarnego pianina, które zostało przedstawione w projekcie "Binary Music Player" [11].

Literatura

[1] Tomasz Bartuś. Analiza regresji i korelacji dwóch zmiennych. [on-line] https://home.agh.edu.pl/~bartus/index.php?action=dydaktyka&subaction=statystyka&item=regresja_i_korelacja.

[2] Microsoft Corporation. Names of namespaces. [on-line] https://learn.microsoft.com/en-us/dotnet/standard/design-guidelines/names-of-namespaces. [Ostatni dostęp: 26.01.2026].

[3] Anthony Stentz Dave Ferguson, Maxim Likhachev. A guide to heuristic-based path planning. [on-line] https://www.cs.cmu.edu/afs/cs.cmu.edu/Web/People/maxim/files/hsplanguide_icaps05ws.pdf, 2005. [Ostatni dostęp: 28.12.2025].

[4] Godot Engine. Godot documentation. [on-line] https://docs.godotengine.org/. [Ostatni dostęp: 07.1.2026].

[5] Inc. Epyx. Rogue. [on-line] https://store.steampowered.com/app/1443430/Rogue/. [Ostatni dostęp: 24.01.2026].

[6] Firaxis Games. Xcom® 2. [on-line] https://store.steampowered.com/app/268500/XCOM_2/. [Ostatni dostęp: 24.01.2026].

[7] Hello Games. No man's sky. [on-line] https://store.steampowered.com/app/275850/No_Mans_Sky/. [Ostatni dostęp: 24.01.2026].

[8] Supergiant Games. Hades. [on-line] https://store.steampowered.com/app/1145360/Hades/. [Ostatni dostęp: 24.01.2026].

[9] GitLab Inc. Gitlab documentation. [on-line] https://docs.gitlab.com, 2026.

[10] John C. Hart. Perlin noise pixel shaders. [on-line] https://dl.acm.org/doi/epdf/10.1145/383507.383531, 2001. [Ostatni dostęp: 28.12.2025].

[11] Tim Holman. Binary music player. [on-line] https://binarypiano.com/.

[12] Ken Schwaber i Jeff Sutherland. The scrum guide: The definitive guide to scrum: The rules of the game. [on-line] https://scrumguides.org, 2020.

[13] Oetiker Tobias i Partl Hubert i Hyna Irene i Schlegl Elisabeth. *The Not So Short Introduction to LaTeX2e*, 2025. [Ostatni dostęp: 07.01.2026].

[14] Amlan Kundu i Yang He. On optimal order in modeling sequence of letters in words of common language as a markov chain. *Pattern Recognition*, 24(7), 1991.

[15] James Newton-King. Json.net (newtonsoft.json). [on-line] https://www.newtonsoft.com/json/help/html/Introduction.htm. [Ostatni dostęp: 28.12.2025].

[16] Guy L. Steele Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In Andrew P. Black and Todd D. Millstein, editors, *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2014, part of SPLASH 2014, Portland, OR, USA, October 20-24, 2014*, pages 453–472. ACM, 2014.

[17] JGraph Ltd. draw.io. [on-line] https://www.draw.io/. [Ostatni dostęp: 08.01.2026].

[18] George Marsaglia. Seeds for random number generators. COMMUNICATIONS OF THE ACM May 2003/Vol. 46, No. 5, 2003.

[19] Matplotlib Development Team. Matplotlib: Visualization with python. [on-line] https://matplotlib.org/. [Ostatni dostęp: 08.01.2026].

[20] Mossmouth. Spelunky. [on-line] https://store.steampowered.com/app/239350/Spelunky/. [Ostatni dostęp: 24.01.2026].

[21] Mark J. Nelson Noor Shaker, Julian Togelius. *Procedural Content Generation in Games*. Computational Synthesis and Creative Systems. Springer Nature, 2016.

[22] NumPy Developers. Numpy. [on-line] https://numpy.org/. [Ostatni dostęp: 08.01.2026].

[23] Die of Death Games. Rogue tower. [on-line] https://store.steampowered.com/app/1843760/Rogue_Tower/. [Ostatni dostęp: 24.01.2026].

[24] Pandas Development Team. pandas: powerful python data analysis toolkit. [on-line] https://pandas.pydata.org/. [Ostatni dostęp: 08.01.2026].

[25] Amit Patel. [on-line] https://www.redblobgames.com/. [Ostatni dostęp: 28.12.2025].

[26] Jordan Peck. Fast portable noise library. [on-line] https://github.com/Auburn/FastNoiseLite/wiki/Documentation. [Ostatni dostęp: 28.12.2025].

[27] Prineside. Infinitode 2 - infinite tower defense. [on-line] https://store.steampowered.com/app/937310/Infinitode_2__Infinite_Tower_Defense/. [Ostatni dostęp: 24.01.2026].

[28] Python Software Foundation. Python. [on-line] https://www.python.org/. [Ostatni dostęp: 08.01.2026].

[29] QuikGraph Contributors. Quikgraph: Generic graph data structures and algorithms for .net. [on-line] https://github.com/KeRNeLith/QuikGraph/wiki. [Ostatni dostęp: 28.12.2025].

[30] Judea Pearl Rina Dechter. Generalized best-first search strategies and the optimality of a*. Journal of the Association for Computing Machinery, Vol. 32, No. 3, July 1985, pp. 505-536., 1985. [Ostatni dostęp: 28.12.2025].

[31] Andrea Röck. Pseudorandom number generators for cryptographic applications. [on-line] https://www-rocq.inria.fr/secret/Andrea.Roeck/pdfs/dipl.pdf, 2005. [Ostatni dostęp: 28.12.2025].

[32] SciPy Community. Scipy. [on-line] https://scipy.org/. [Ostatni dostęp: 08.01.2026].

[33] David Silva. Kod źródłowy rogue. [on-line] https://github.com/Davidslv/rogue. [Ostatni dostęp: 24.01.2026].

[34] Unity Technologies. Unity documentation. [on-line] https://docs.unity3d.com/. [Ostatni dostęp: 28.12.2025].

[35] Michael Waskom. seaborn: statistical data visualization. [on-line] https://seaborn.pydata.org/. [Ostatni dostęp: 08.01.2026].

[36] Jiahao Liu Ziqian Wu. Perlin noise and its improvements: A literature review. [on-line] https://www.researchgate.net/publication/393342810_Perlin_noise_and_its_improvements_A_literature_review, 2024. [Ostatni dostęp: 28.12.2025].

Słowniczek (Akronimy i pojęcia)

Termin/Skrót	Definicja
Deweloper	W kontekście tej pracy rozumiane jako twórca oprogramowania, mówiąc dokładniej autor dowolnej gry.
Gracz	Użytkownik końcowy, osoba użytkująca z projektu dewelopera.
Grid	(pol. Siatka) Struktura danych reprezentująca teren logiczny rozgrywki.
HP	(ang. Health Points) - Wartość wykorzystywana w grach w celu pokazania ile potrzeba ataków aby zniszczyć obiekt w grze.
NMS	(ang. No Man's Sky) Skrócona nazwa gry. [7]
Noise	(pol. Szum) Dokładniej szum Perlina. Trójwymiarowe pole jednakowych liczb losowych [10].
NPC	(ang. Non-Player Character) Postać występująca w grze, nie kontrolowana poprzez gracza. Może się odnościć do przeciwników ale głównie używane do określenia szeroko postaci neutralnych bądź przyjaźnych graczowi.
Pathfinding	(pol. Wyszukiwanie ścieżek) Algorytm, którego zadaniem jest znalezienie jak najkrótszej ścieżki między dwoma punktami poprzez wyznaczenie listy kroków [3].
PCG	(ang. procedural content generation) Metoda wykorzystująca zbiór wcześniej przygotowanych algorytmów oraz danych służąca do tworzenia dalszych funkcjonalności i zawartości przez komputer przy ograniczonym lub pośrednim udziale człowieka [21].
PRNG	(ang. pseudo-random number generator) Generator liczb pseudolosowych. Algorytmy stosowane do uzyskania wartości liczb pseudo-losowych. Na podstawie jakiegoś źródła entropii zwraca wygenerowany wynik. Nie jest prawdziwie losowe, ponieważ jest deterministyczne względem wartości wejściowych [31].
Seed	Wartość losowa/pseudolosowa używana do zainicjowania generatora [18][31].
SO	(ang. scriptable object) Rodzaj obiektu specjalistyczny dla silnika Unity. Służy jako kontener danych [34].
Spawner	(pol. punkt powstawania) Lokalizacja w grze, stanowiąca domyślną lokalizację, gdzie pojawiają się i wkraczają do rozgrywki przeciwnicy (ewentualnie oprócz przeciwników inne obiekty).
TD	(ang. tower defense) Skrót od nazwy gatunku gier tower defense.
TDPG	(ang. tower defense procedural generation) Nazwa biblioteki stworzonej jako rezultat tej pracy dyplomowej.

© 2026 Jakub Kolasiński, Krzysztof Mańczak, Sebastian Marcinek, Jakub Szybura

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu LaTeX na platformie Overleaf.